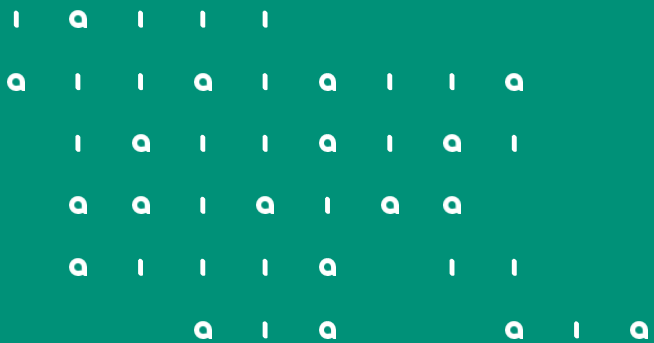
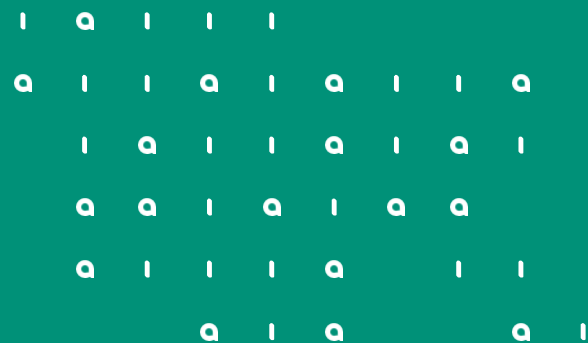




[하나은행] Digital Hana 路
금융서비스개발

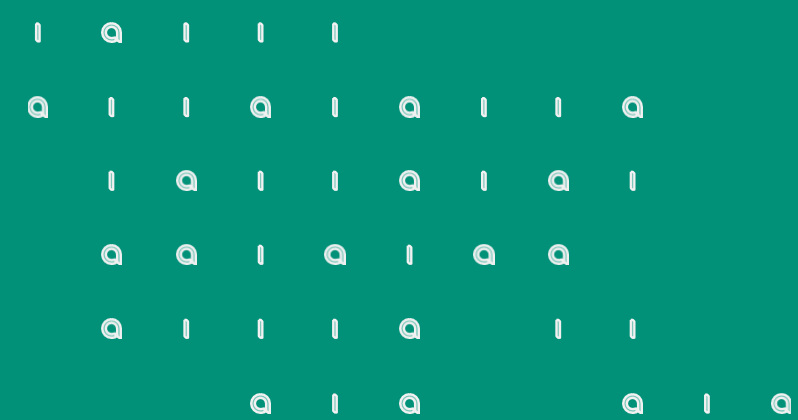
첫 웹페이지, UI/UX,
GitHub, ICT아키텍처



목차



- 01 ----- 웹(WWW)의 동작원리
- 02 ----- HTML5/CSS3 기초 및 브라우저의 작동 원리
- 03 ----- HTML5/CSS3 고급 기법
- 04 ----- Tailwind CSS Framework로 UI/UX 구성하기
- 05 ----- Git 시작하기
- 06 ----- Git 기본 명령어
- 07 ----- 깃 브랜치(Branch)
- 08 ----- 깃 스테시(stash)와 클린(clean)
- 09 ----- 깃 머지(merge)와 리베이스(rebase)
- 10 ----- 깃 리셋(reset), 리버트(revert), 태그(tag)
- 11 ----- 깃 플로우(git-flow)
- 12 ----- Fork & Pull Request
- 13 ----- Best Practice in a Project
- 14 ----- Github Action
- 15 ----- ICT Architecture의 이해

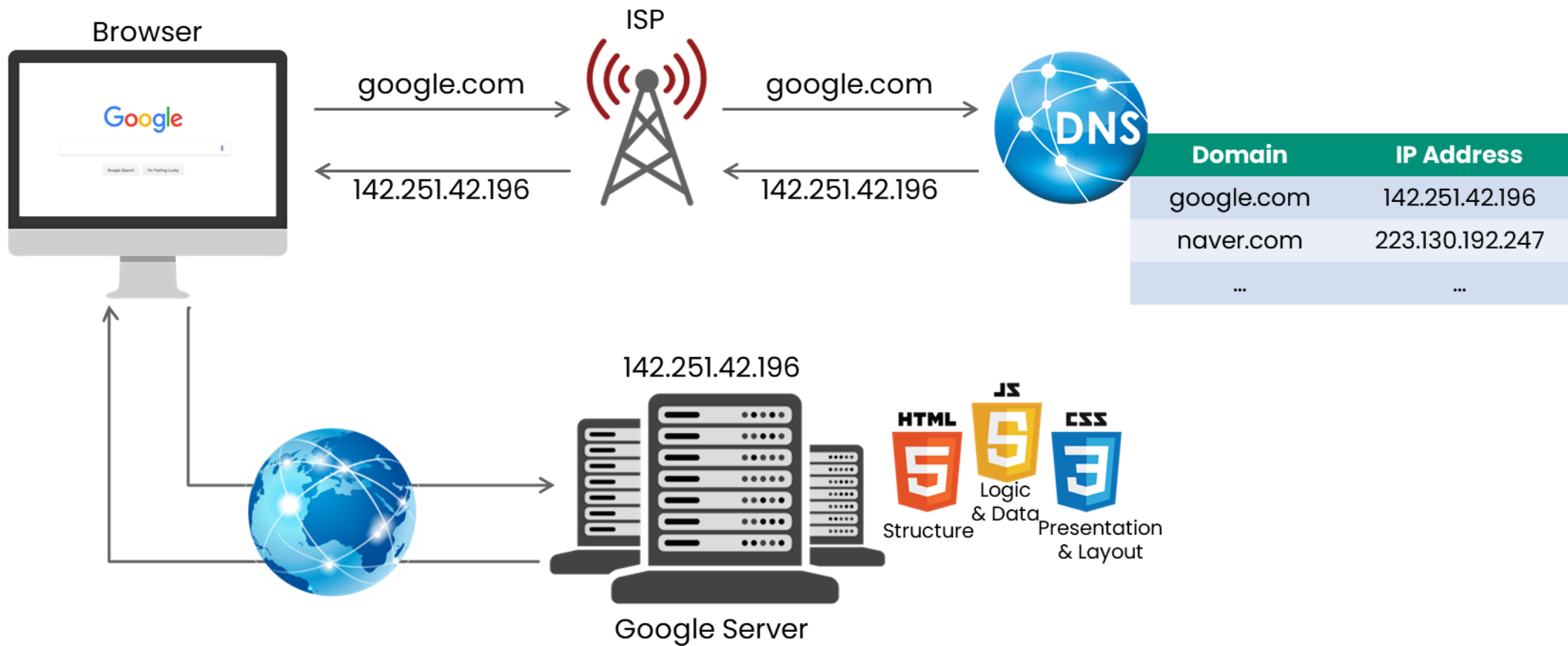


1. 웹(www)의 동작 원리



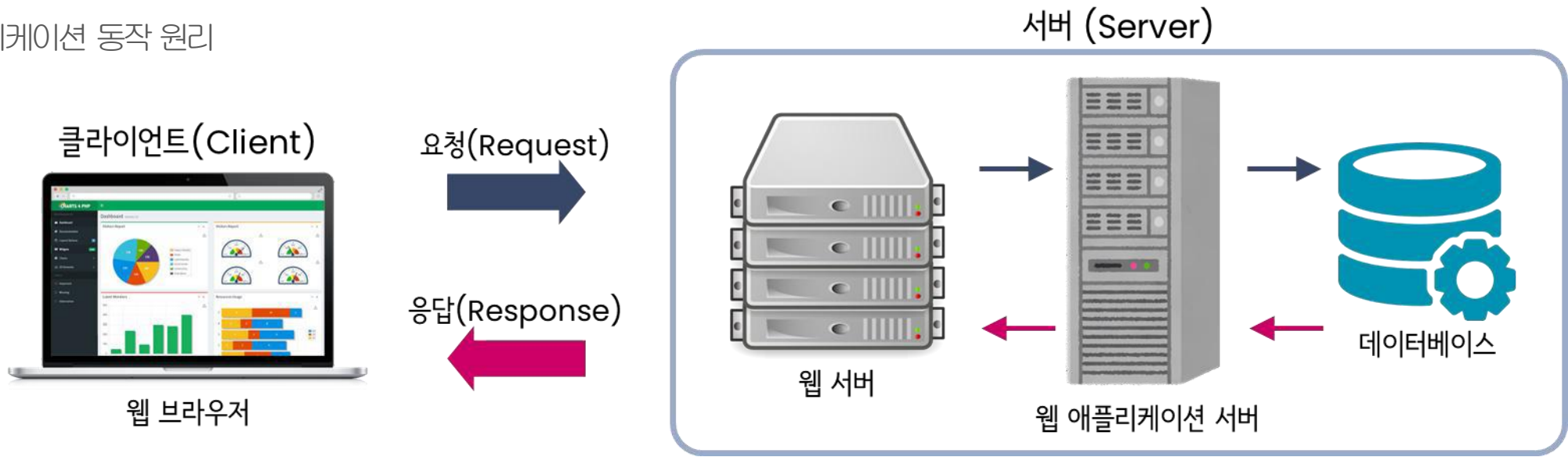
1. 웹(www)의 동작 원리

How the web works



1. 웹(www)의 동작 원리

웹 애플리케이션 동작 원리



구성 요소	설명
웹 브라우저	• 사용자(클라이언트)가 요청한 웹 서버의 결과를 화면에 출력해 보여주는 애플리케이션
웹 서버	• 웹 브라우저의 요청을 받아 들어서 이에 알맞은 결과를 웹 브라우저에 응답해 주는 역할 • 정적인 페이지를 처리를 담당 • 요청에 필요한 페이지의 로직이나 데이터베이스와의 연동을 위해서 애플리케이션 서버에 이들의 처리를 요청 • 대표적인 웹 서버로는 아파치(Apache)와 Microsoft IIS(Internet Information Server) 등이 있음
웹 애플리케이션 서버 (WAS, Web Application Server)	• 요청한 페이지의 로직이나 데이터베이스와의 연동을 처리하는 부분 • 동적인 페이지 처리를 담당 • 대표적인 WAS로는 아파치 톰캣(Tomcat), BEA사의 웹로직(WebLogic), IBM의 웹스파이어(WebSphere) 등이 있음
데이터베이스	• 데이터 정보를 저장하는 공간

1. 웹(www)의 동작 원리

TCP/IP

❖ TCP/IP

- TCP(Transmission Control Protocol)과 IP(Internet Protocol)과 관련되는 프로토콜 군(protocol suite)을 총칭하며, 100개가 넘는 프로토콜이 존재

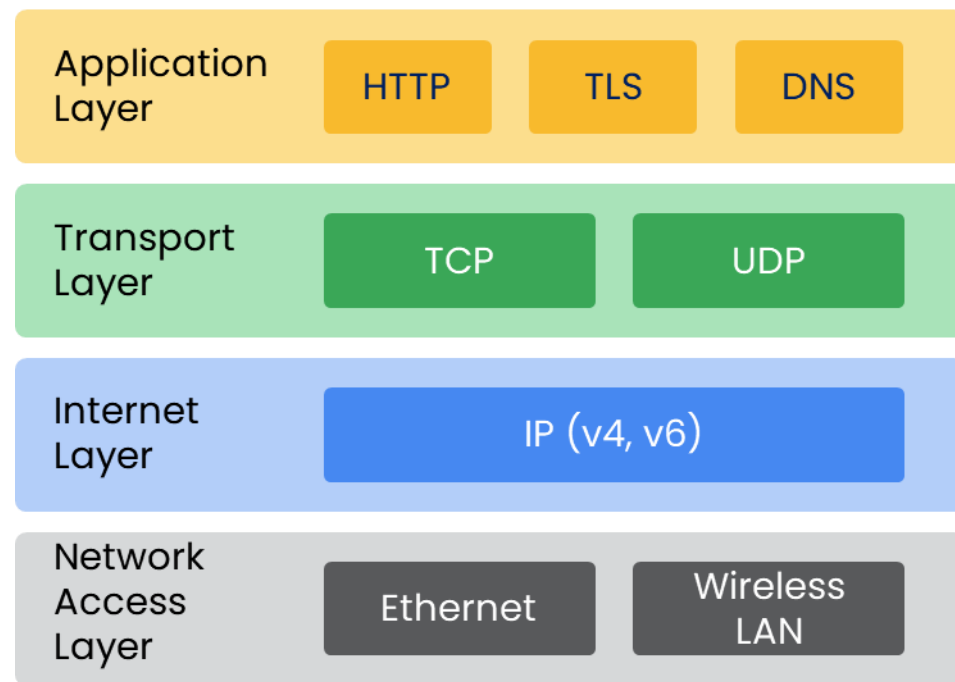
❖ TCP

- 근거리 통신망이나 인트라넷, 인터넷에 연결된 장비들간의 데이터 교환을 안정적으로 교환가능하게 해줌
- OSI 7계층에서 전송계층에 속하며, 네트워크의 정보전달을 통제하는 프로토콜 중의 하나
- 웹 브라우저들이 월드 와이드 웹에서 서버에 연결할 때 사용됨

❖ IP

- 데이터 패킷의 주소를 해석하고 경로를 설정하여 다음 호스트로 전송하는 프로토콜
- 여러 개의 상호 패킷 교환망의 상호연결을 위한 범용 비연결형 프로토콜임
- OSI 7계층에서 네트워크 계층에 해당함

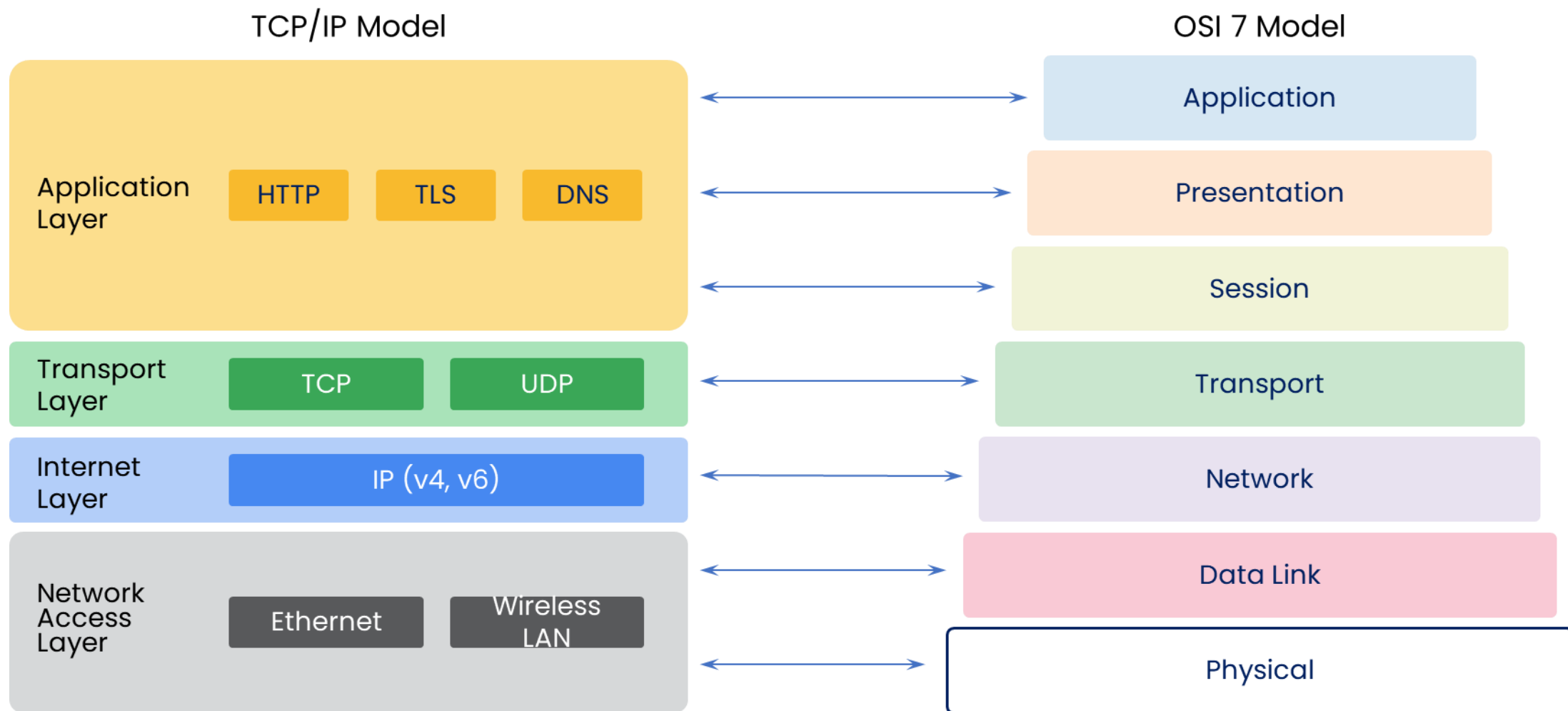
TCP/IP 계층 구조



1. 웹(www)의 동작 원리

TCP/IP

❖ TCP/IP 모델 vs. OSI 7 모델



1. 웹(www)의 동작 원리

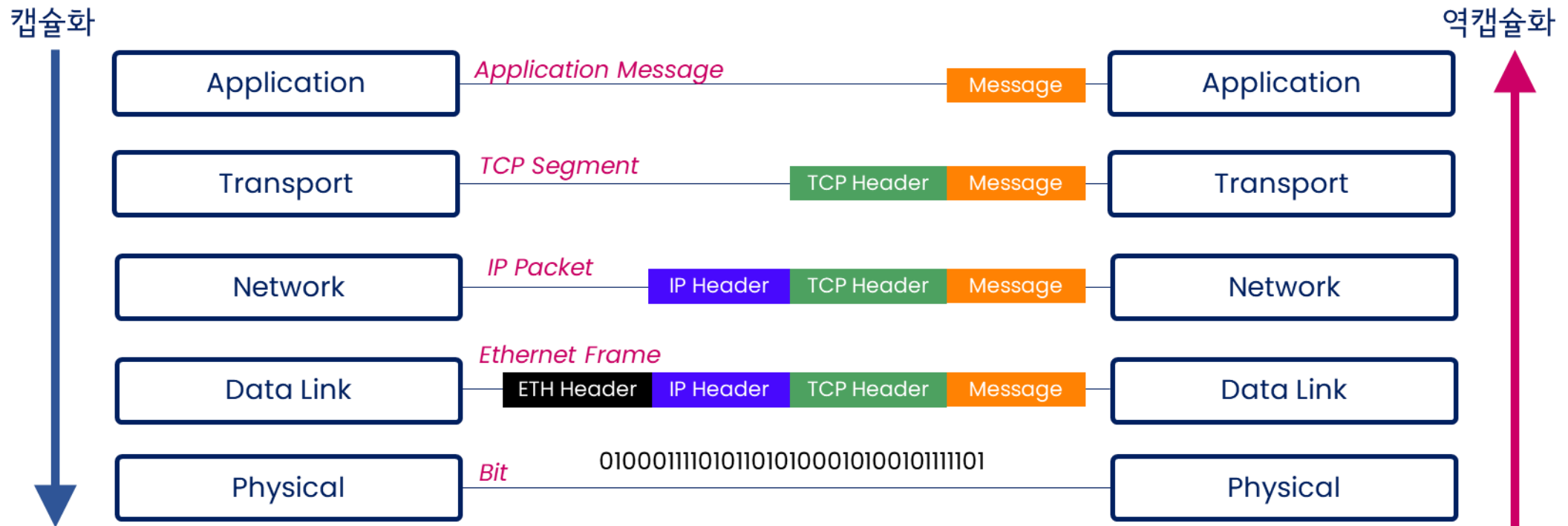
캡슐화와 역캡슐화

❖ 캡슐화(Encapsulation)

- 송신지 Application Layer에서 발생한 데이터를 하위 계층으로 이동시키면서 각계층에서 처리한 결과를 캡슐화하는 과정

❖ 역캡슐화(Decapsulation)

- 수신지의 하위 계층에서 인식한 데이터를 상위 계층으로 이동시키면서 각 네트워크 계층에서 처리 가능한 형태로 디캡슐화하는 과정



1. 웹(www)의 동작 원리

Port 번호

- ❖ 포트 번호는 TCP와 UDP가 상위 계층에 제공하는 주소 표현 방식
- ❖ 컴퓨터 내에서 응용 프로그램을 식별하는 번호 (0~65,535의 번호 사용)
- ❖ 0~1,023(Well Known)은 인터넷주소관리기구가 특정 프로그램에 예약되어 있음
- ❖ 서버 프로그램은 고정된 포트 번호를 사용
 - 예를 들어 메일 서버는 25번, 웹 서버는 80번 포트를 사용
 - 클라이언트 컴퓨터에서 웹 브라우저로 웹 서버에 접속하려면 해당 웹 서버의 IP 주소 외에 80번 포트에 연결을 요청
 - 운영체제는 컴퓨터 내부로 전송된 데이터를 소켓에 분배
 - 수신된 데이터 안에 새겨진 Port 번호를 참조하여 해당 소켓에 데이터 전달

포트 번호	분류
0 ~ 1023	알려진 포트(well-known ports)
1024 ~ 49151	등록된 포트(registered ports)
49152 ~ 65535	동적/사설 포트(dynamic and/or private ports)

Well-known 포트 번호

서비스	포트 번호
FTP(데이터 채널)	20
FTP(제어 채널)	21
TELNET	23
SMTP	25
DNS	53
HTTP	80



1. 웹(www)의 동작 원리

IP (1/2)

❖ IP 주소의 클래스

- A 클래스, B 클래스, C 클래스, D 클래스, E 클래스 가 있으며 각각 첫 번째 바이트로 식별
- 예시) 첫 번째 바이트가 0이면 A 클래스, 10 이면 B 클래스, 110이면 C 클래스, 1110이면 D 클래스, 1111이면 E 클래스
- IP 총 주소 할당은 A 클래스가 50%, B 클래스가 25%, C 클래스가 12.5%, D 클래스와 E 클래스가 6.25%
- 클래스에 따라 IP 네트워크 주소의 지정 범위가 다름

IP 주소 클래스

	바이트 1	바이트 2	바이트 3	바이트 4
A 클래스	0			
B 클래스	10			
C 클래스	110			
D 클래스	1110			
E 클래스	1111			

네트워크 ID와 호스트 ID

	바이트 1	바이트 2	바이트 3	바이트 4
A 클래스	네트워크 ID	호스트 ID		
B 클래스	네트워크 ID		호스트 ID	
C 클래스	네트워크 ID			호스트 ID
D 클래스	멀티캐스트 주소			
E 클래스	차후 사용을 위해 예약됨			

1. 웹(www)의 동작 원리

IP (2/2)

❖ IP 주소의 클래스

■ A 클래스의 주소체계

- ✓ 첫 번째 8비트 즉 1바이트(1옥텟)가 네트워크 ID에, 나머지가 호스트 ID에 할당
- ✓ A 클래스의 네트워크 ID 범위는 0~127이고, 호스트에 할당 가능한 주소는 16,777,214(2²⁴) 개



■ B 클래스의 주소체계

- ✓ 앞의 2바이트가 네트워크 ID에, 나머지 2바이트가 호스트 ID에 할당
- ✓ B 클래스의 네트워크 ID 범위는 128.0~191.255이고, 호스트에 할당 가능한 주소는 65,534(2¹⁶) 개

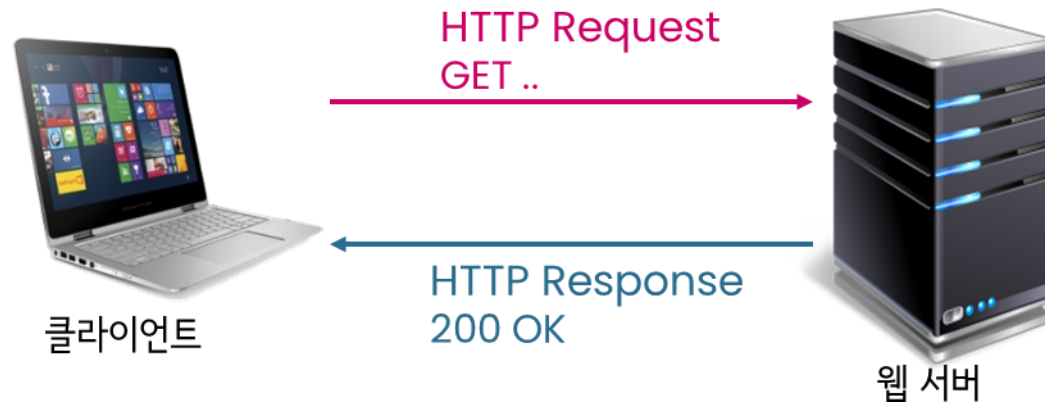


1. 웹(www)의 동작 원리

HTTP

❖ HTTP (Hyper Text Transfer Protocol)

- 인터넷 사용자가 필요한 정보를 자유롭게 검색하게 해주는 하이퍼텍스트 방식의 정보를 교환하기 위한 규칙
- TCP/IP 4계층에서 애플리케이션 계층에 해당
- 전송 계층에서 TCP를 사용하여 웹 브라우저와 웹 서버간에 통신하는 프로토콜
- 무연결(Connectionless) : 응답 후 연결 끊음
- 무상태(Stateless) : 요청마다 다른 연결



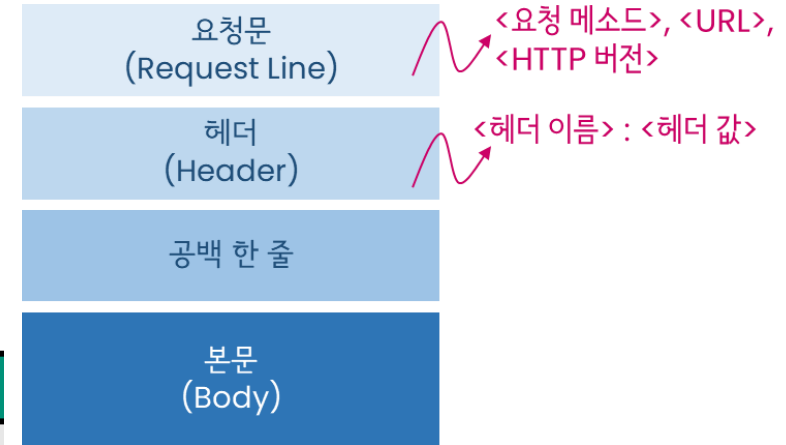
1. 웹(www)의 동작 원리

HTTP 요청

❖ HTTP 요청 메시지

- 클라이언트가 서버에 보내는 요청 메시지는 요청문, 헤더, 바디로 구성
- 요청문의 내용은 <요청 메소드>, <URL>, <HTTP 버전>의 세 부분으로 구성HTTP
- 요청 메소드

요청 메시지의 구조



요청 메시지의 예시

GET /edu/index.html HTTP/1/1

name : content

메소드	설명
GET	- 웹 서버로부터 웹 문서를 요청할 때 사용 - 요청은 URL에 포함된 쿼리 파라미터를 통해 추가 정보를 전달 예) 주소 줄에 URL 직접 입력하거나 하이퍼링크를 클릭하여 요청 페이지로 이동
POST	- 클라이언트에서 웹 서버에 데이터를 전송하여 새로운 리소스를 생성 하거나 데이터를 처리할 때 사용 - 데이터를 body에 포함하여 서버에 전송
PUT	- 서버에 있는 기존 리소스를 전체적으로 업데이트할 때 사용 - 아이디가 포함된 URI를 통해 특정 리소스를 지정
PATCH	서버의 기존 리소스를 부분적으로 수정할 때 사용
DELETE	- 서버에서 리소스를 삭제할 때 사용 - URL로 삭제할 리소스를 지정

1. 웹(www)의 동작 원리

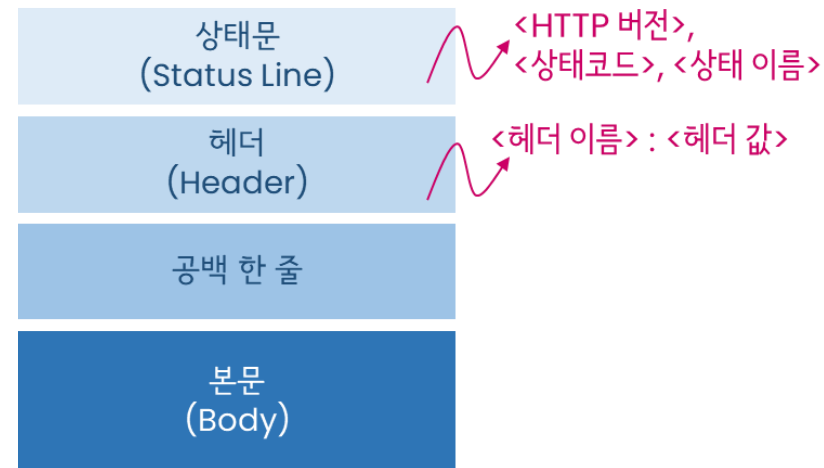
HTTP 응답

❖ HTTP 응답 메시지

- 클라이언트로부터 요청 메시지를 수신한 서버는 해당 요구를 처리한 후에 그 결과를 응답 메시지 형식으로 회신
- 응답 메시지의 구조는 요청 메시지와 거의 동일
- 요청문 대신 처리 결과를 의미하는 상태문이라는 용어를 사용
- HTTP 상태 코드

코드	이름	의미
200	OK	요청이 성공적으로 수행되었음
400	Bad Request	요청 메시지의 내용에 문법 오류가 존재함
401	Unauthorized	클라이언트가 인증되지 않았음을 의미
403	Forbidden	웹 서버가 요청을 이해했지만, 해당 요청에 대한 처리를 거부함. 일반적으로 권한이 부족할 때 발생
404	Not Found	원하는 문서를 찾을 수 없음
500	Internal Server Error	요청을 처리하는 중에 웹 서버 내부에 일반적인 오류가 발생하였음
501	Not Implemented	서버가 요청한 메소드를 지원하지 않거나, 구현되지 않았음

응답 메시지의 구조



응답 메시지의 예시

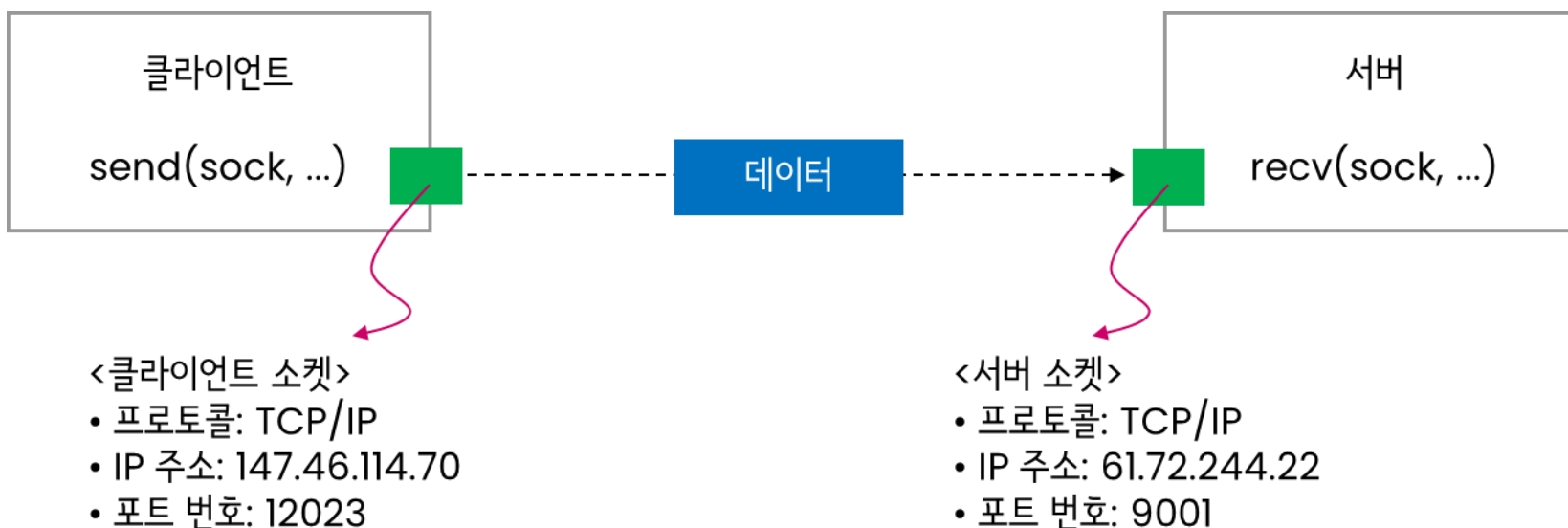
HTTP/1.1 200 OK
name : content

1. 웹(www)의 동작 원리

SOCKET

❖ Socket

- 네트워크를 통해 데이터를 교환하기 위한 인터페이스
- 소켓은 네트워크 상에서 수행되는 두 프로그램 간의 양방향 통신 링크의 끝 단을 의미
- 소켓을 통해 클라이언트와 서버가 양방향 통신을 할 수 있음
- IP 주소와 특정 포트 번호를 기반으로 통신
- 응용 프로그램은 자신의 소켓이 상대방의 소켓과 연결된 것으로 생각하고 데이터를 주고받음



1. 웹(www)의 동작 원리

SOCKET의 종류

❖ 스트림 (TCP)

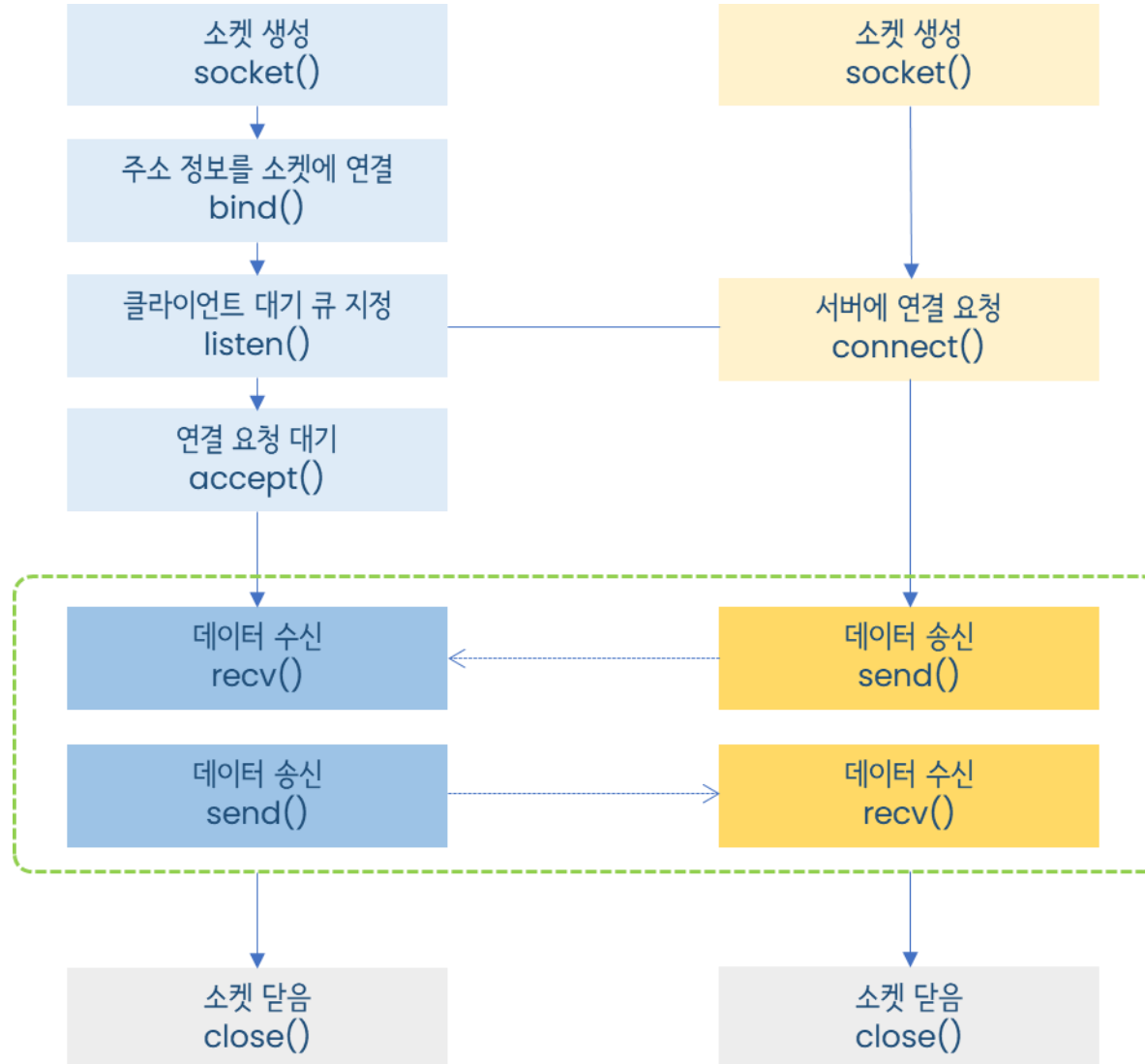
- 양방향으로 바이트 스트림을 전송, 연결 지향성
- 오류 수정, 전송처리, 흐름제어 보장
- 송신된 순서에 따라 중복되지 않게 데이터를 수신 → 오버헤드가 발생
- 소량의 데이터보다 대량의 데이터 전송에 적합 → TCP를 사용

❖ 데이터그램 (UDP)

- 비연결형소켓
- 데이터의 크기에 제한이 있음
- 확실하게 전달이 보장되지 않음, 데이터가 손실돼도 오류가 발생하지 않음
- 실시간 멀티미디어 정보를 처리하기 위해 주로 사용 예)전보

1. 웹(www)의 동작 원리

소켓의 연결형 서비스

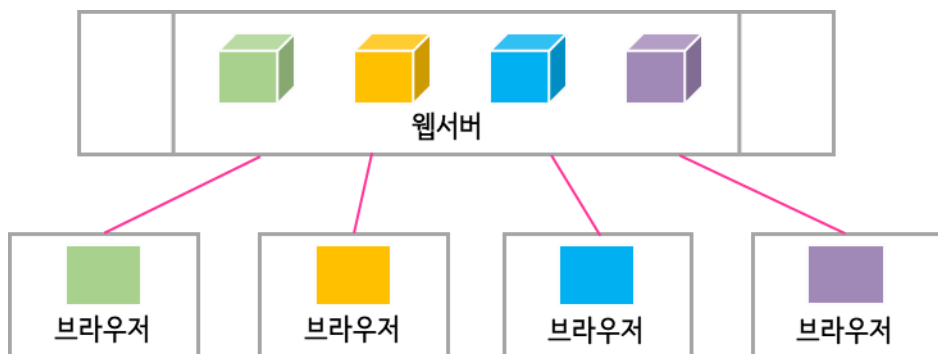


1. 웹(www)의 동작 원리

세션과 쿠키

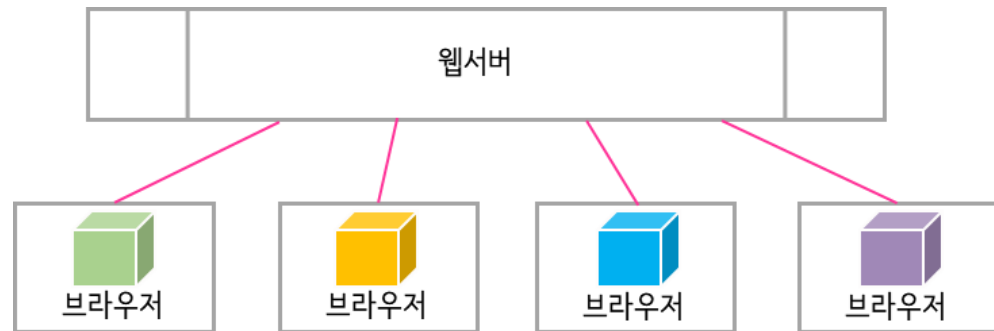
❖ 세션(session)

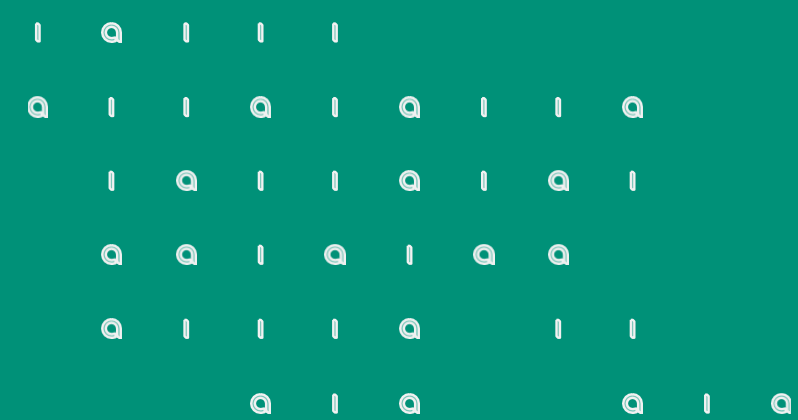
- 클라이언트가 웹 브라우저를 통해 서버에 접속한 후 용무를 처리하고 웹 브라우저를 닫아 서버와의 접속을 종료하는 하나의 단위
- 상태가 없는 프로토콜(stateless protocol)인 HTTP에서 상태에 대한 보전을 위해서 사용
- HttpSession 객체가 생성될 때 클라이언트 정보, 요청 시간 정보 등을 조합한 세션 ID 부여 (세션 ID는 클라이언트 측에 쿠키 기술로 저장)
- 최근에는 세션인증보다는 토큰인증방식을 많이 사용



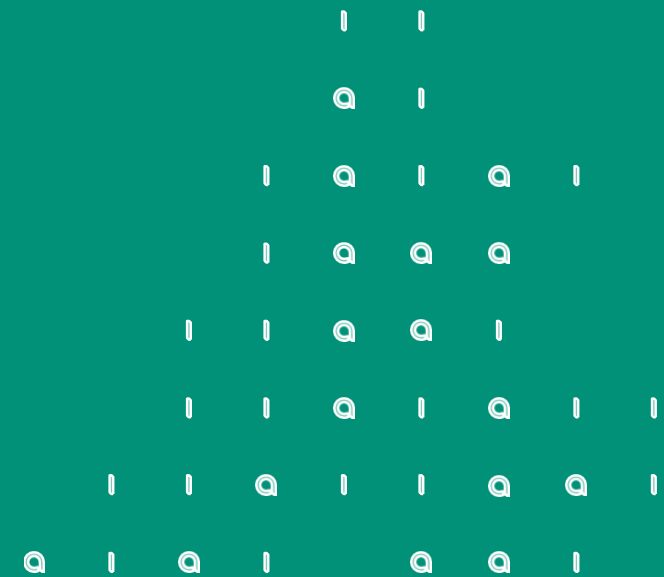
❖ 쿠키(cookie)

- 클라이언트 단위로 상태정보를 유지하는 방법
- 쿠키는 세션과 달리 상태 정보를 웹 서버가 아닌 클라이언트에 저장
- 보안에 취약해서 사용이 많이 축소되었으나 최근에는 토큰기반인증 방식에서 보안을 강화하여 다시 사용이 증가하는 추세





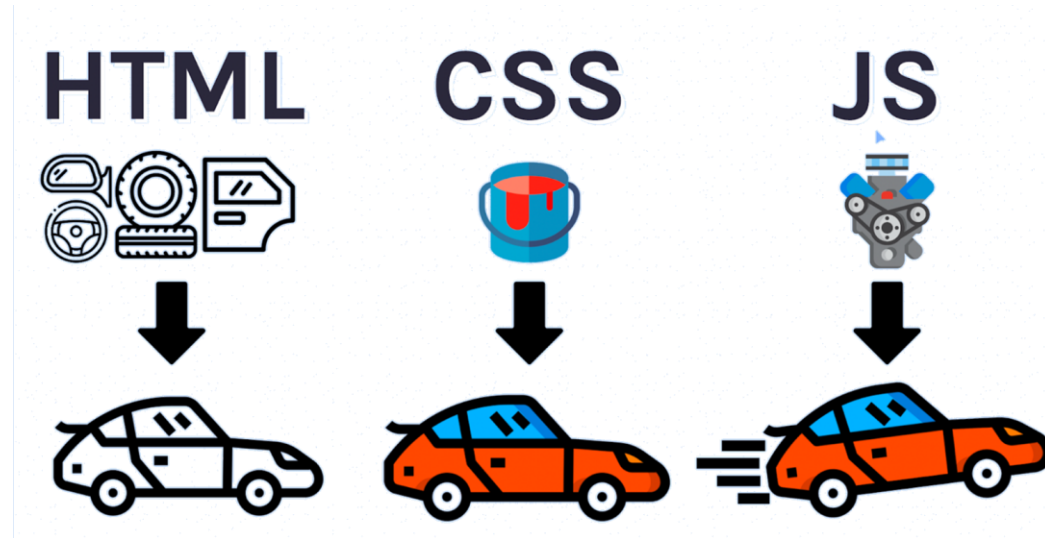
2. HTML5/CSS3 기초 및 브라우저의 작동 원리



2. HTML5/CSS3 기초 및 브라우저의 작동 원리

웹 표준 기술

- ❖ 현대 웹 페이지에 사용되는 표준 기술은 HTML5 표준, CSS3 표준, ECMAScript 표준



HTML, CSS, 자바스크립트의 역할

웹 표준 기술	역할	코드
HTML	웹 페이지 구성를 구성하는 HTML 마크업 언어	<code><input type="button"></code>
CSS	HTML 페이지에 스타일을 지정하는 스타일시트 작성 언어	<code>color: black</code>
자바스크립트	사용자 반응을 처리하는 스크립트 언어	<code>const onClick = ...;</code>

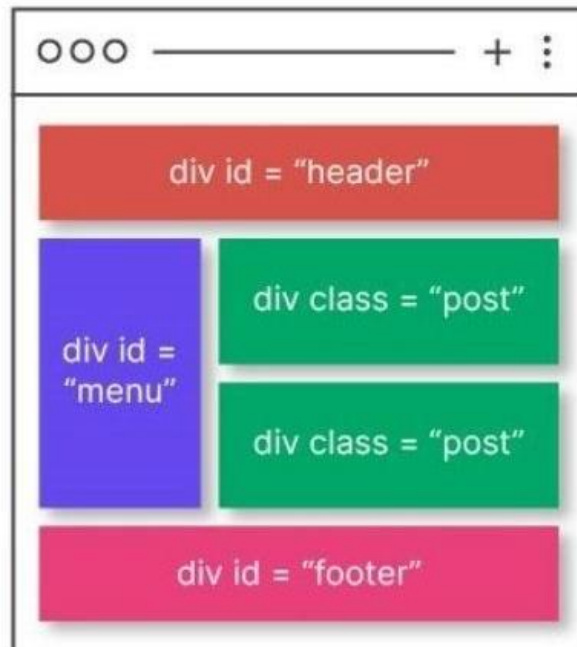
2. HTML5/CSS3 기초 및 브라우저의 작동 원리

HTML vs. HTML5

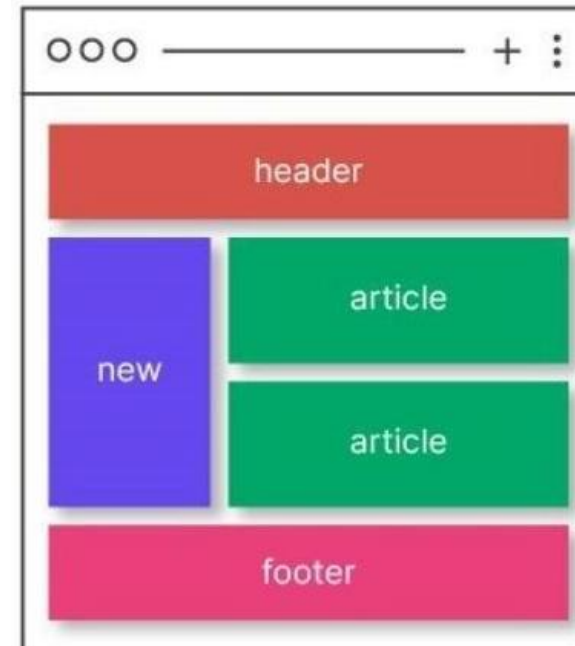
❖ HTML5

- 큰 의미로 CSS3와 자바스크립트를 모두 포함하는 웹 표준 기술을 총칭
- 작은 의미로 웹 페이지를 구성하는 HTML 마크업 언어 자체
 - ✓ 마크업(markup) : 웹 페이지의 서식이나 구조를 표현하는 정보

HTML



HTML5



2. HTML5/CSS3 기초 및 브라우저의 작동 원리

HTML5과 주요 기능

❖ 멀티미디어 관련 태그와 API

- 웹에서 비디오나 오디오를 표현하기 위한 태그와 API를 지원
- 비디오 컨테이너 파일은 오디오 트랙, 비디오 트랙, 추가 메타 데이터로 구성
- avi, flv, mp4, mkv, ogv 등

❖ 그래픽

- 하드웨어의 가속을 받아 2차원, 3차원 그래픽을 구현.
- 예)SVG, CANVAS 등

❖ 장치 접근(DEVICE ACCESS)

- 장치 정보(배터리 잔량, 현재 위치) 가져오거나 장치 기능 (알람 등)을 사용
- 카메라, 동작센서 등의 H/W 기능을 웹에서 직접적으로 제어 가능

❖ 오프라인 및 저장소 (OFFLINE & STORAGE)

- 인터넷이 연결되지 않은 상태에서도 응용 프로그램을 동작시킴

❖ 위치정보 서비스(Geo-Location)

- GPS없이도 지리 및 위치정보를 이용할 수 있는 API를 제공

❖ 드래그 앤 드롭

- 웹 페이지 내 혹은 외부 개체를 웹 페이지로 드래그 또는 드롭 할 수 있는 API를 제공

❖ 시맨틱(Semantic)

- HTML5 표준에서는 시맨틱 태그를 새로 추가
- 시맨틱 웹(Semantic web) : 검색 엔진 같은 프로그램이 정보를 분석하고 자료를 검색하고 처리해서 제공하는 지능형 웹

2. HTML5/CSS3 기초 및 브라우저의 작동 원리

HTML5 호환성

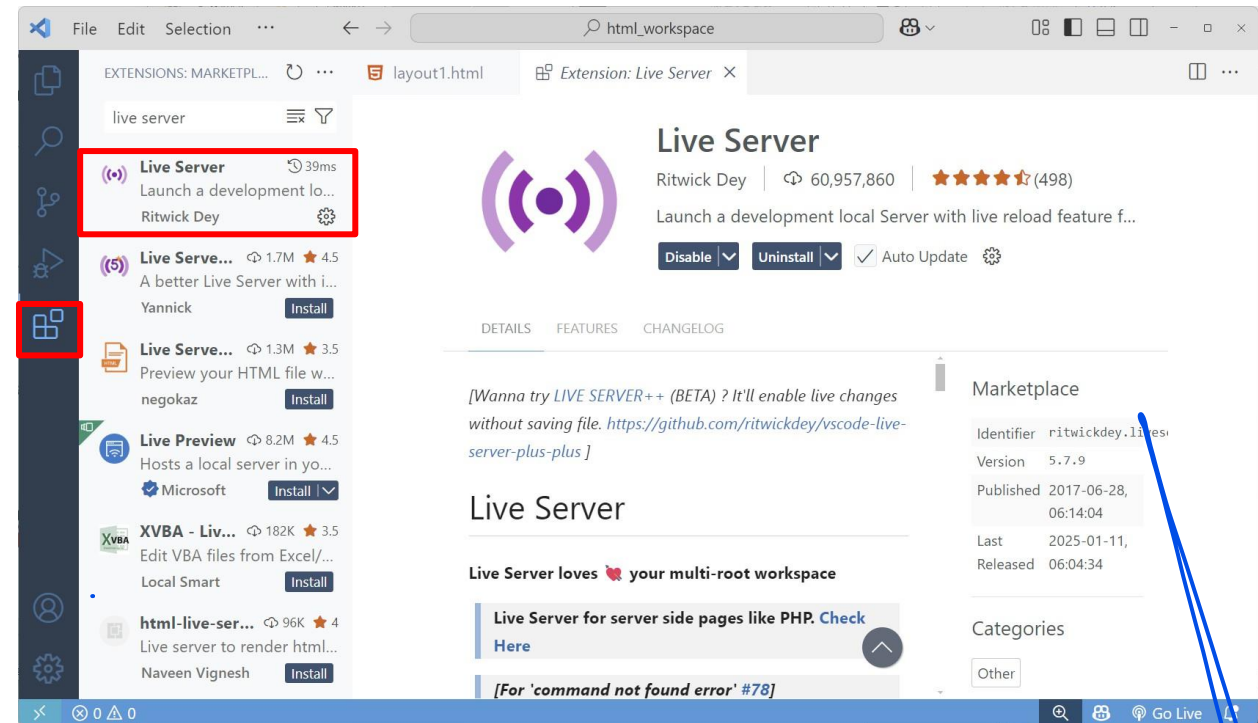
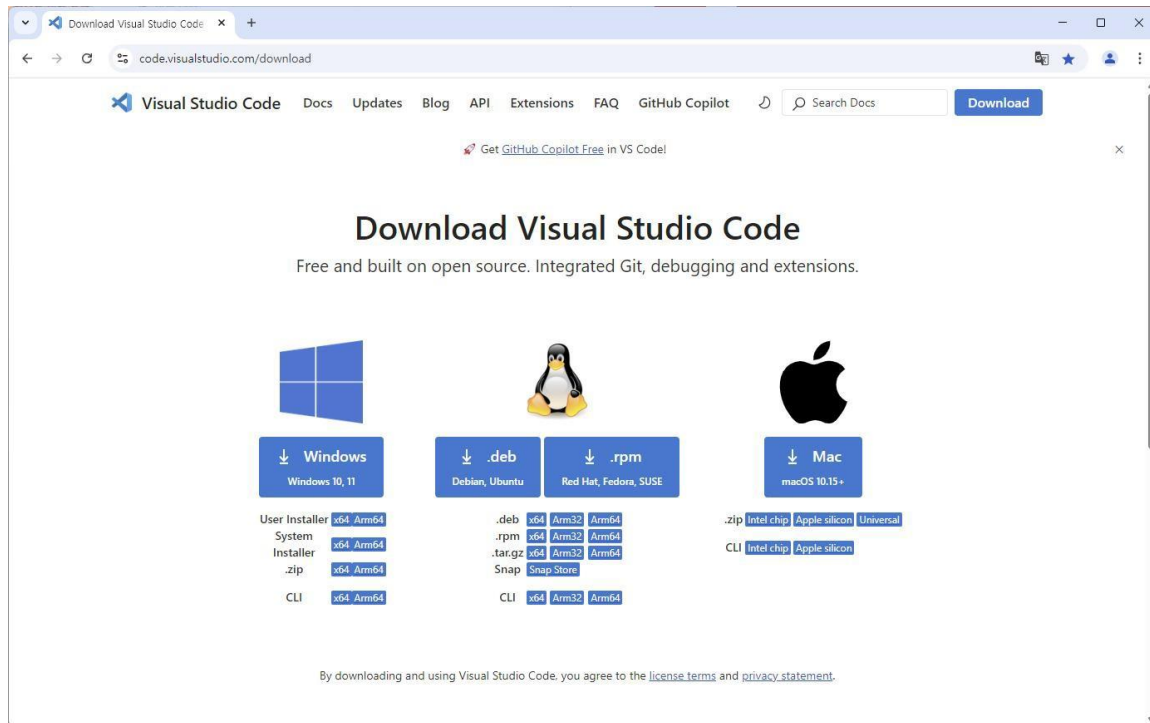
- ❖ 콘텐츠의 호환성 : HTML5 이전 버전으로 제작한 콘텐츠도 문제없이 이용 가능
- ❖ 이전 브라우저와의 호환성 : HTML5 가 지원되지 않는 이전 버전의 브라우저에서도 이용 가능
- ❖ 이용 방법의 호환성 : 기존에 HTML 태그 사용법을 최대한 사용 가능
- ❖ 기능의 재사용 : 각각의 브라우저에서만 사용 가능한 기능들을 통합하여 공통적으로 사용할 수 있음
- ❖ 혁신보다는 발전을 우선 : HTML5는 새로운 언어를 구현하는 것이 아니므로 이미 사용 중인 HTML을 보다 사용하기 쉽도록 재구성

2. HTML5/CSS3 기초 및 브라우저의 작동 원리

실습 환경 구축

❖ 편집기는 atom, intellij, Visual Studio Code 등이 있음

- Visual Studio Code 를 OS에 적합한 방식으로 설치
- Extension에서 live server 설치



2. HTML5/CSS3 기초 및 브라우저의 작동 원리

HTML5 문서의 구조

❖ 헤드 <head></head>

- 웹 페이지에 대한 정보 제공
- 스타일시트나 자바스크립트 제공

❖ 바디 <body></body>

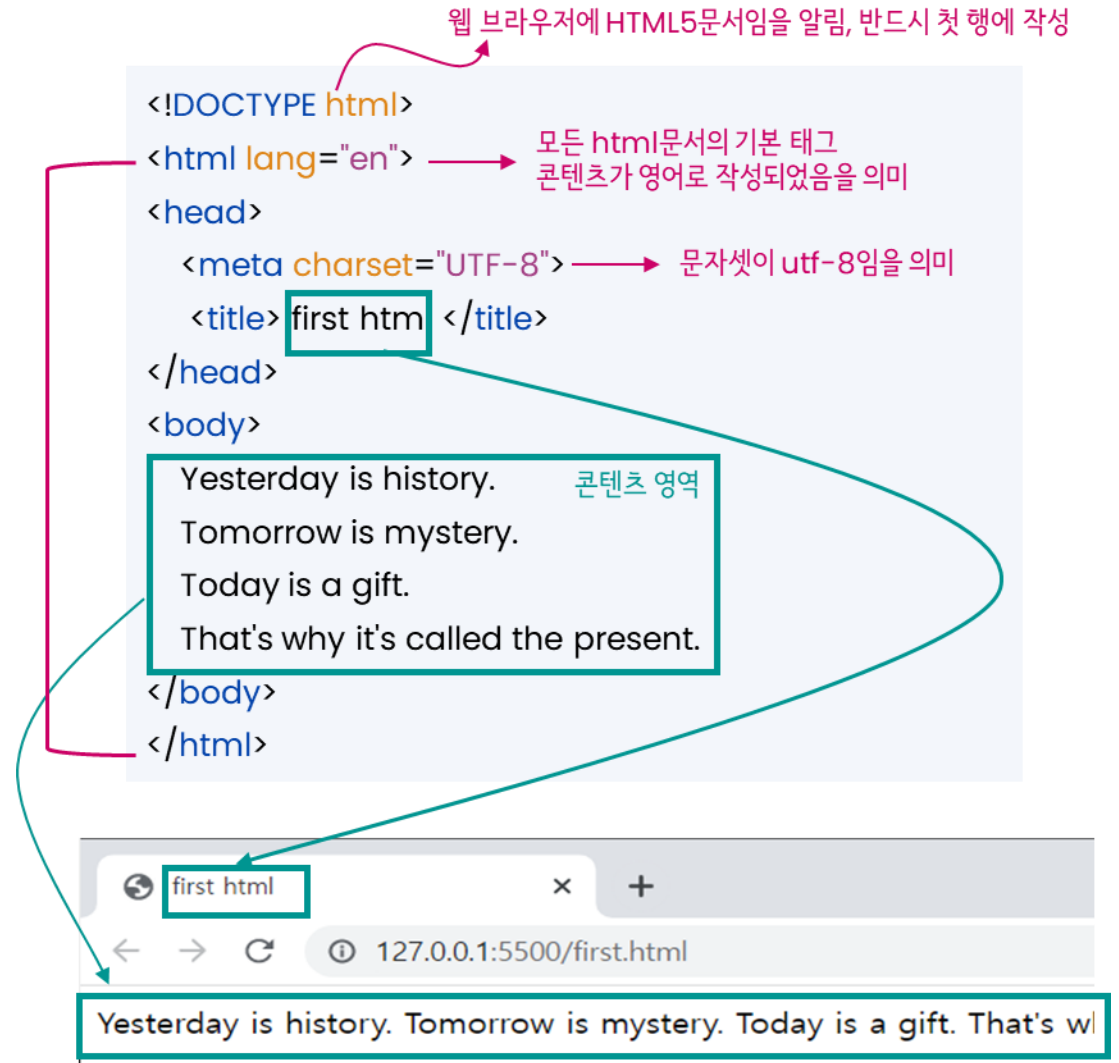
- 사용자에게 실제 보이는 웹 페이지 내용

❖ 태그들

- '<>'로 감쌈
- 태그들은 각각 다른 작업 실행
- 시작태그와 종료태그를 쌍으로 사용 (예외 있음)

❖ 텍스트

- 특별한 태그 없이도 표시할 수 있음
- 모든 텍스트가 연결되어 하나의 줄에 표시
- <p></p>로 단락 표시,
으로 줄 바꿈



2. HTML5/CSS3 기초 및 브라우저의 작동 원리

시맨틱 태그 (Semantic Tag)

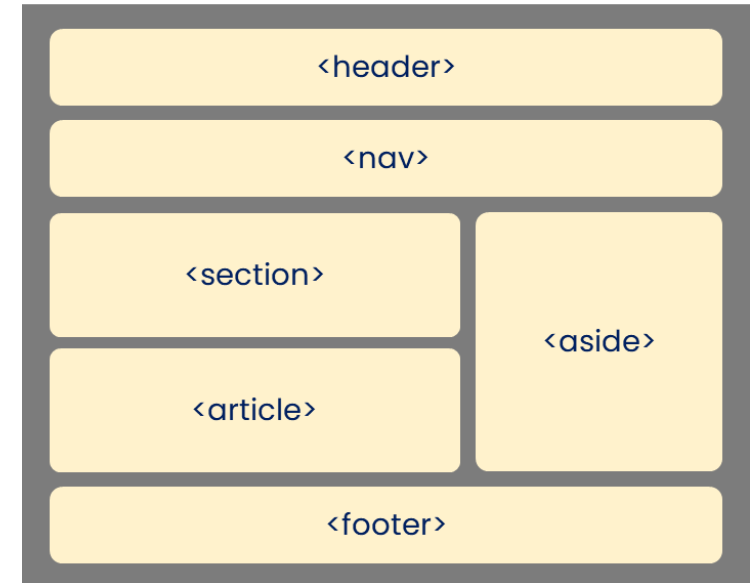
❖ 시맨틱 웹

- 검색 엔진에 문서 정보를 의미 있게 전달할 수 있고 검색 엔진 또한 시맨틱 요소를 이용하여 정보를 효과적으로 불러 올 수 있음

❖ 시맨틱 태그

- 웹 페이지의 태그에 의미를 부여하는 태그
- 브라우저, 검색 엔진, 개발자에게 요소의 의미를 명확하게 설명하는 역할

시맨틱 태그를 이용한 레이아웃 예시

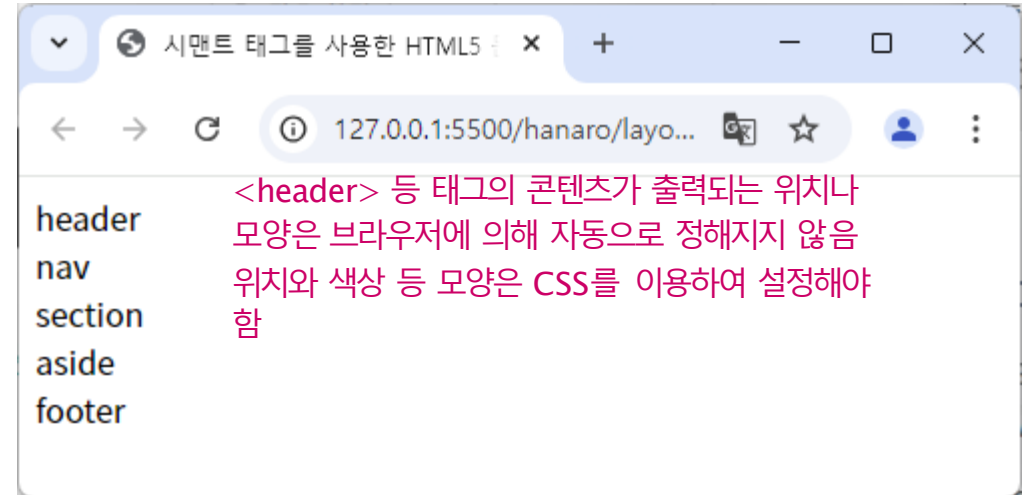


태그	설명
<header>	페이지의 제목, 소개 요소 또는 탐색 링크를 나타냄
<nav>	하이퍼링크들을 모아 둔 내비게이션. 주로 메뉴를 구현
<section>	본문 내용을 포함하는 공간으로 문서의 영역을 구성
<article>	독립적인 콘텐츠를 표시할 때 사용
<aside>	사이드 바처럼 좌우에 위치하는 공간 본문의 흐름에서 벗어나는 노트나 팁과 같은 콘텐츠를 구성할 때 사용
<footer>	문서의 하단을 정의. 카테고리를 요약하거나 저작권을 명시하는 용도

2. HTML5/CSS3 기초 및 브라우저의 작동 원리

HTML5 Layout

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width,
initial-scale=1.0">
  <title>시맨트 태그를 사용한 HTML5 문서 구조</title>
</head>
<body>
  <header>header</header>
  <nav>nav</nav>
  <section>section</section>
  <aside>aside</aside>
  <footer>footer</footer>
</body>
</html>
```



2. HTML5/CSS3 기초 및 브라우저의 작동 원리

Html 태그들

요소	설명
audio, video	멀티미디어 콘텐츠를 표시하는데 사용
embed	Plugin 콘텐츠를 표시할 때 사용
mark	별도로 표시한 문자를 표시하는데 사용
progress	작업 진행상황을 나타낼 때 사용
meter	측정값을 표시할 때 사용
time	날짜, 시간을 표시할 때 사용
ruby, rt, rp	Ruby 언어를 나타낼 때 사용
canvas	Bitmap Graphic을 표시할 때 사용
command	사용자가 호출할 수 있는 명령어를 표시하는데 사용
details	사용자 요청에 따라 얻은 컨트롤이나 추가적인 정보를 표시하는데 사용
datalist	List attribute와 함께 사용하여 combobox를 만들 때 사용
keygen	키쌍(key pair)를 생성할 때 사용
output	스크립트 계산 결과와 같은 실행 결과를 표시하는데 사용

2. HTML5/CSS3 기초 및 브라우저의 작동 원리

Html5 속성

속성	설명
data-*	접두사 “data-”를 가진 속성으로 추후 HTML 버전 충돌 없이 사용자 태그로 이용 하거나 추후 브라우저가 지원하게 되었을 때 사용
role, aria-*	보조 기능을 가리키는데 사용
contenteditable	편집 가능한 영역임을 나타냄
contextmenu	작성자가 제작한 Context Menu를 지정하는데 사용
draggable	Drag & Drop API에서 사용
hidden	요소가 없거나 없을 때 사용
spellcheck	맞춤법 검사 기능을 제공할지 여부를 지정

2. HTML5/CSS3 기초 및 브라우저의 작동 원리

CSS3 소개

❖ CSS의 정의

- CSS는 “Cascading Style Sheets”의 약자로, HTML로 작성된 문서의 레이아웃과 스타일을 정의하는 스타일 시트 언어
- HTML이 문서의 구조를 정의하는데 사용되고, CSS는 문서의 디자인 및 스타일을 담당

❖ CSS3의 역할

- 레이아웃 조정 : 요소들의 배치, 크기, 간격 등을 지정하여 웹 페이지의 레이아웃을 조정
- 텍스트 스타일링 : 글꼴, 색상, 크기 등과 같은 텍스트 스타일을 지정
- 색상 및 배경 관리 : 요소의 배경색, 이미지 또는 동적인 효과 등을 설정하여 디자인을 rots
- 애니메이션 및 트랜지션 효과 : 요소들에 애니메이션이나 트랜지션 효과를 부여하여 웹 페이지를 동적으로 만듦
- 반응형 디자인(responsive design) : 다양한 디바이스와 화면 크기에 대응하여 웹 페이지를 반응형으로 만드는데 사용

❖ CSS3의 구성

- 선택자(Selector): 스타일 시트를 적용할 대상을 지정
- 속성(Property): 어떤 속성을 적용할지 선택
- 속성값(Value): 속성에 어떤 값을 반영할지 선택

The diagram illustrates the structure of a CSS rule. It shows a code snippet: `h1 { color: blue; font-size: 12px; }`. Above the code, five labels with arrows point to specific parts: '선택자' (Selector) points to 'h1', '속성' (Property) points to 'color', '속성값' (Value) points to 'blue', '속성' (Property) points to 'font-size', and '속성값' (Value) points to '12px'.

```
h1 { color: blue; font-size: 12px; }
```

2. HTML5/CSS3 기초 및 브라우저의 작동 원리

CSS 문법과 사용 위치

❖ CSS의 기본 문법

```
div { width: 960px; }
```

```
/* 한 줄로 작성 */  
p { color: blue; background-color: yellow; }
```

```
p {  
    /* 여러 줄로 작성  
    */ color: blue;  
    background-color: yellow;  
}
```

❖ CSS의 사용 위치

■ Inline CSS

- ✓ 태그 내에 style 속성을 이용해서 스타일을 적용

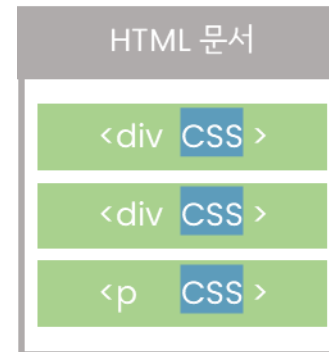
■ Internal CSS

- ✓ html 문서내에 <style>~</style>태그를 이용해 적용

■ External CSS

- ✓ 외부에 css파일을 만들어서 사용
- ✓ <link rel="stylesheet" href="a.css">

인라인 스타일



내부 스타일



외부 스타일



적용 우선 순위

!important > inline > id선택자 > class명 > HTML 태그명 > DOM구조의 상위 상속

2. HTML5/CSS3 기초 및 브라우저의 작동 원리

스타일 적용하기

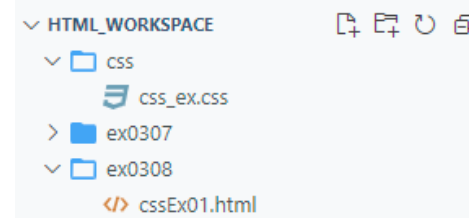
```
p3{  
  color:green;  
  background-color: yellow;  
}
```

CSS/CSS_EX.CSS

```
<head>  
  <meta charset="UTF-8">  
  <meta http-equiv="X-UA-Compatible" content="IE=edge">  
  <meta name="viewport" content="width=device-width, initial-scale=1.0">  
  <link type="text/css" rel="stylesheet" href="../css/css_ex.css"/>  
  <title>Document</title>  
  <style>  
    p2{  
      /* 내부 스타일 시트 적용 */  
      color:blue;  
    }  
  </style>  
</head>  
<body>  
  <p1 style="color:red;">인라인 스타일 시트 적용</p1><br/>  
  <p2>내부 스타일 시트 적용</p2> <br/>  
  <p3>외부 스타일 시트 적용</p3> <br/>  
</body>
```

cssEx01.html

ex0308폴더 내부에 cssEx01.html 파일 위치
css폴더 내부에 css_ex.css 파일 위치



인라인 스타일 시트 적용
내부 스타일 시트 적용
외부 스타일 시트 적용

2. HTML5/CSS3 기초 및 브라우저의 작동 원리

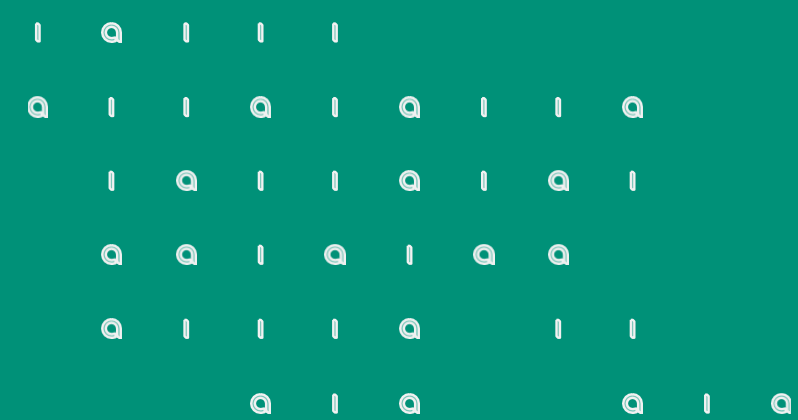
CSS 선택자 (1/2)

종류	사용 방법	설명
전체 선택자	* {속성 선언;}	모든 태그에 스타일 적용 * { margin: 0; padding: 0; }
태그 선택자	태그 {속성 선언;}	지정한 태그에 스타일 적용 p { color: blue; }
클래스 선택자	.클래스 이름 {속성 선언;}	지정한 클래스에 스타일 적용 .highlight { background-color: yellow; }
아이디 선택자	#아이디 {속성 선언;}	지정한 아이디에 스타일 적용 #header { font-size: 24px; }
속성 선택자	[속성] {속성 선언;} [속성=값] {속성 선언;}	지정한 속성 또는 속성값이 있는 태그에 스타일 적용

2. HTML5/CSS3 기초 및 브라우저의 작동 원리

CSS 선택자 (2/2)

종류	사용 방법	설명
후손 선택자	선택자 A 선택자 B	선택자 B가 선택자 A에 반드시 포함되어 있을 경우 <code>article p {</code> <code>line-height: 1.5;</code> <code>}</code>
자식 선택자	선택자 A > 선택자 B	부모 선택자 A의 직계 자식인 선택자 B를 선택 <code>nav > ul {</code> <code>list-style-type: none;</code> <code>}</code>
인접 형제 선택자	선택자 A + 선택자 B	같은 부모를 가진 특정 요소의 다음 인접 형제를 선택 <code>h2 + p {</code> <code>font-style: italic;</code> <code>}</code>
일반 형제 선택자	선택자 A ~ 선택자 B	같은 부모를 가진 특정 요소의 모든 형제를 선택 <code>h2 ~ p {</code> <code>color: green;</code> <code>}</code>
그룹 선택자	선택자 A, 선택자 B	선택자 A와 선택자 B를 모두 선택



3. HTML5/CSS3 고급 기법



3. HTML5/CSS3 고급 기법

form태그

- ❖ form태그안에 input 태그를 두어서 사용자로부터 정보를 입력 받아 서버로 보내는 역할을 담당
- ❖ 주의
 - form태그를 중첩적으로 사용할 수 없음 <form>~</form><form>~</form> 형태로만 가능
 - 보통 하나의 웹페이지에 하나의 form 태그를 배치 (여러 개를 배치할 수는 있음)
- ❖ get방식 전송
 - Input태그의 name속성=값1&input태그의 name속성2=값2&input태그의 name속성3=값3 형태로 전달
 - name속성을 키로 하고 value속성에 값이 저장
 - 예시) username=홍길동&usage=23&address=서울시
- ❖ 태그를 식별하는 속성값이 3개가 있음 (id, name, class)
 - input 태그류만-서버로 정보를 전송할 수 있음
 - input, textarea, select 태그만 name속성을 가짐
 - table, div, p, h ...- id랑 class속성을 가짐

3. HTML5/CSS3 고급 기법

form태그의 주요 속성

❖ action

- 정보를 전송하면 이 정보를 받아서 처리해야 할 서버페이지 url을 지정
- 특별히 지정을 안 하면 자신의 페이지를 호출

❖ method

- 정보를 서버로 보낼 때 보내는 방법을 지정할 수 있음
- get 방식
 - ✓ 1024바이트 이내의 간단한 정보만 보낼 수 있고 url 에 보내는 모든 정보가 표시됨
 - ✓ 보안성이 떨어져서 보통 상세보기나 목록화면 등 간단한 정보만 보내고자 할 때 사용
- post
 - ✓ 사실상 무한대의 정보를 보낼 수 있으나, 대개 20m~50m 내의 정보만을 전송
 - ✓ 데이터를 보내는 방식은 application/x-www-form-urlencoded, text/plain, multipart/form-data 등

❖ enctype: 파일을 보낼때 반드시 필요함

❖ name : 폼객체에 접근할 수 있음. 예) document.폼이름

❖ id : document.getElementById("아이디")

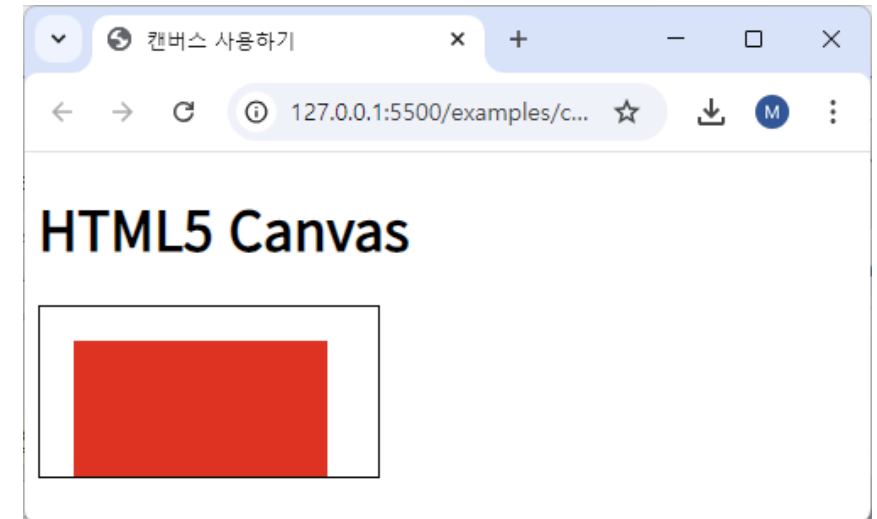
3. HTML5/CSS3 고급 기법

Canvas 사각형 그리기

❖ JavaScript를 사용하여 그래픽을 그릴 수 있는 영역을 제공하는 HTML5 요소임

- <canvas>를 사용하면 동적인 그래픽을 생성하고 제어할 수 있음
- 이와 유사한 요소로 <svg>가 있습니다. 이 요소를 이용해 차트 등을 작성할 수 있음

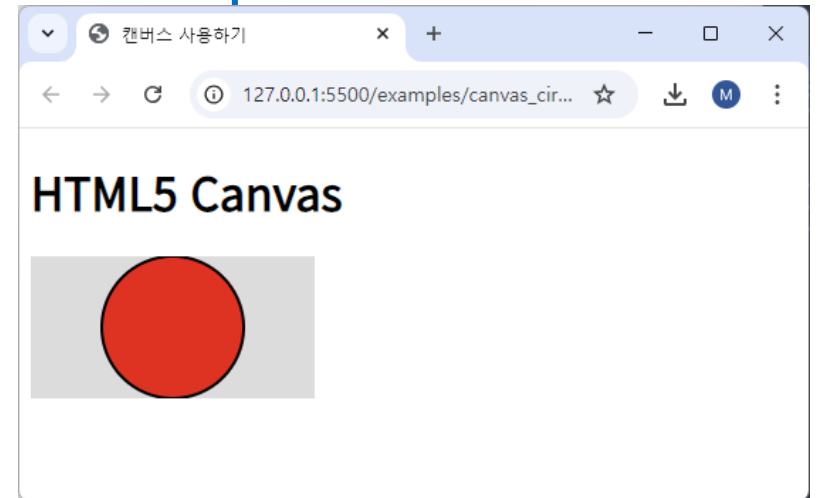
```
<head>
  <meta charset="UTF-8">
  <title>캔버스 사용하기</title>
</head>
<body>
  <h1>HTML5 Canvas</h1>
  <canvas id="myCanvas" width="200" height="100" style="border: 1px solid
  black;">
    Sorry, your browser does not support canvas.
  </canvas>
  <script>
    const canvas = document.getElementById("myCanvas");           //
    캔버스 만들기
    document.body.append
    Child(canvas); const ctx
    = canvas.getContext("2d");
    ctx.fillStyle = "red";
    ctx.fillRect(20, 20, 150, 100);
  </script>
</body>
```



3. HTML5/CSS3 고급 기법

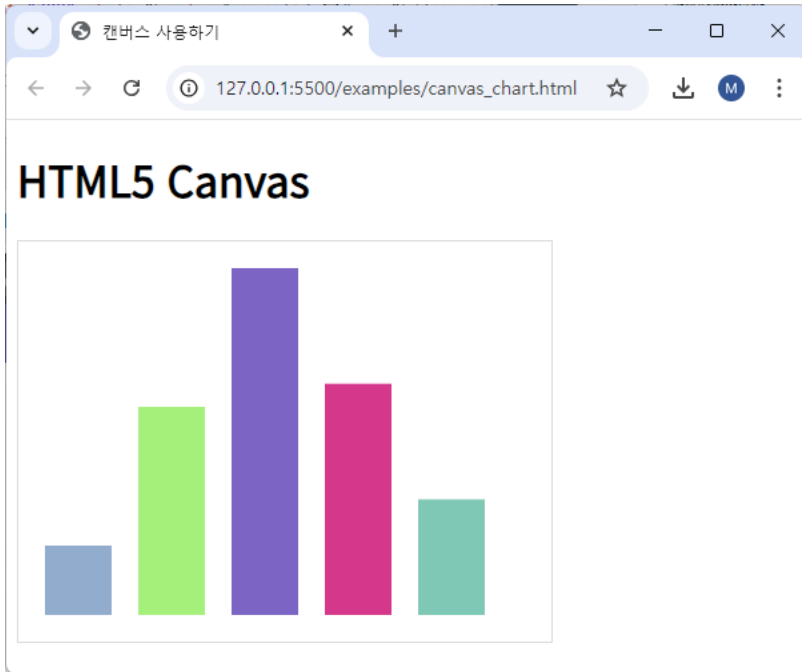
Canvas 원 그리기

```
<body>
<h1>HTML5 Canvas</h1>
<canvas id="myCanvas" width="200" height="100" style="background-color:gainsboro;">
  Sorry, your browser does not support canvas.
</canvas>
<script>
document.addEventListener("DOMContentLoaded", function(){
  const canvas = document.getElementById("myCanvas");    // 캔버스 만들기
  const ctx = canvas.getContext("2d");
  let centerX = canvas.clientWidth/2;    // 영역의 크기 계산
  let centerY = canvas.height/2; // 영역의 중심 위치 계산
  let radius = 50;
  ctx.beginPath();
  ctx.arc(centerX, centerY, radius, 0, 2*Math.PI, false);
  ctx.fillStyle="#FF0000";    // 채우기 색상 설정
  ctx.fill();
  ctx.lineWidth = 2;    // 테두리 두께 설정
  ctx.strokeStyle = "#000";
  ctx.stroke();
  ctx.closePath();
});
</script>
</body>
```

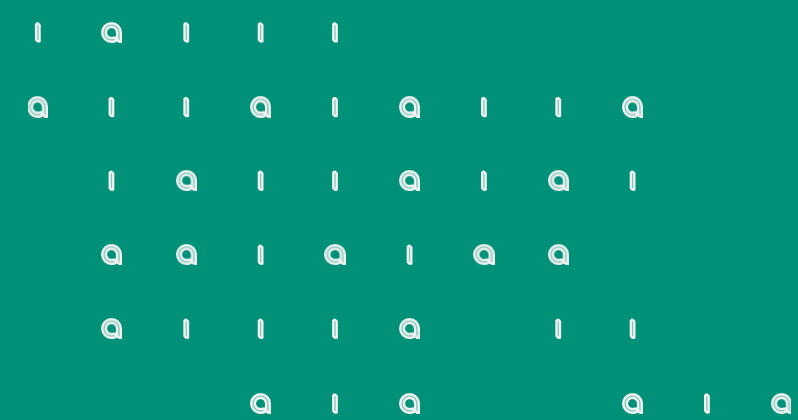


3. HTML5/CSS3 고급 기법

Canvas 차트 그리기



```
<body>
  <h1>HTML5 Canvas</h1>
  <canvas id="barChart" width="400" height="300" style="border:1px solid gainsboro;">
    Sorry, your browser does not support canvas.
  </canvas>
  <script>
    const canvas = document.getElementById("barChart");    // 캔버스 만들기
    const ctx = canvas.getContext("2d");
    let data = [30, 90, 150, 100, 50];
    drawBarChart(data); // 바 차트 만들기
    function drawBarChart(data) {
      // 바 차트 속성 설정
      let barWidth = 50;
      let barSpacing = 20;
      let startX = 20;
      let startY = canvas.height - 20;
      let maxValue = Math.max.apply(null, data);
      data.forEach(function (item) {
        let barHeight = (item / maxValue) * (canvas.height - 40);
        ctx.fillStyle = getRandomColor();
        ctx.fillRect(startX, startY - barHeight, barWidth, barHeight);
        startX += barWidth + barSpacing;
      });
    }
    function getRandomColor() {
      let letters = '0123456789ABCDEF';
      let color = "#";
      for (let i = 0; i < 6; i++) {
        color += letters[Math.floor(Math.random() * 16)];
      }
      return color;
    }
  </script>
</body>
```



4. Tailwind CSS Framework로 UI/UX 구성하기



4. Tailwind CSS Framework로 UI/UX 구성하기

Tailwind CSS

❖ Tailwind CSS

- Adam Wathan과 Adam Pisoni가 개발한 CSS 프레임워크
- Tailwind CSS는 유틸리티 클래스 기반으로 구성
- HTML 요소에 직접 클래스를 추가하여 원하는 스타일을 적용할 수 있음
- 다양한 색상, 글꼴, 크기, 여백, 패딩 등을 위한 미리 정의된 클래스를 제공

❖ 장점

- 별도의 CSS 파일을 만들거나 관리할 필요가 없음
- 코드가 간결하고 명확하며, 코드를 읽고 이해하기 쉬움
- HTML 요소에 직접 클래스를 추가하여 원하는 스타일을 적용하기 때문에 디자인 방식이 직관적임
- 기본 제공되는 클래스 외에도 자체 클래스를 만들거나, 기존 라이브러리를 활용하여 사용할 수 있음

❖ Tailwind CSS 시작하기

- Tailwind 설치(React 과정에서 설치할 예정)
 - ✓ npm 설치가 선행되어야 함
 - ✓ tailwind cli 설치 및 설정

```
PS D:\fullstack_workspace\front\mall> npm install -D tailwindcss
PS D:\fullstack_workspace\front\mall> npx tailwindcss init
```

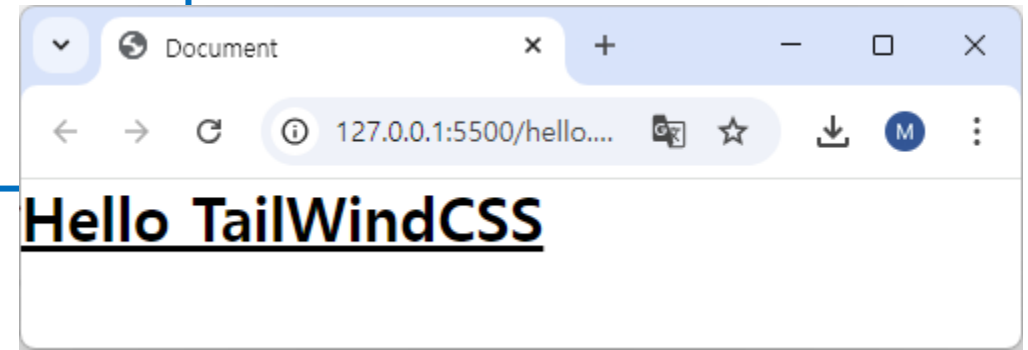
- Tailwind CSS CDN 설치

```
<script src="https://cdn.tailwindcss.com"></script>
```

4. Tailwind CSS Framework로 UI/UX 구성하기

첫 번째 예제(first.html)

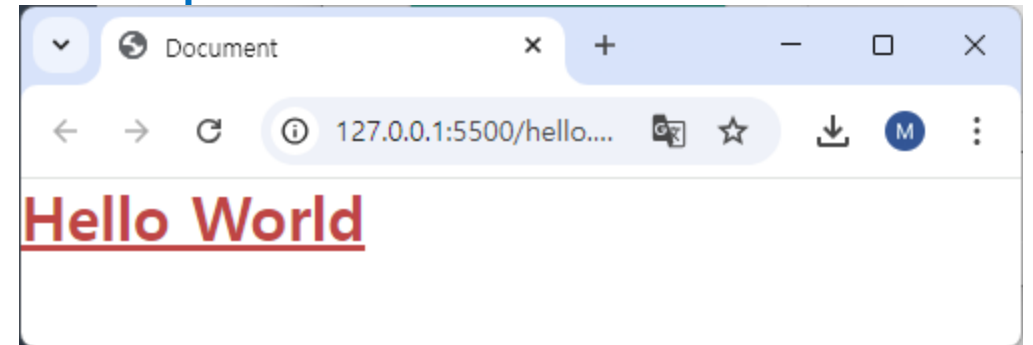
```
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <script src="https://cdn.tailwindcss.com"></script>
</head>
<body>
  <h1 class="text-3xl font-bold underline">Hello TailWindCSS</h1>
</body>
```



4. Tailwind CSS Framework로 UI/UX 구성하기

두 번째 예제(second.html)

```
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <script src="https://cdn.tailwindcss.com"></script>
  <script>
    tailwind.config = {
      theme: {
        extend: {
          colors: {
            clifford: "#da373d"
          }
        }
      }
    }
  </script>
</head>
<body>
  <h1 class="text-3xl font-bold underline text-clifford">
    Hello World</h1>
</body>
```



4. Tailwind CSS Framework로 UI/UX 구성하기

세 번째 예제 (third.html)

<!-- 이전 생략 -->

```
<body class="ml-10 mt-10 mr-10">
```

```
<p class="text-orange-400">
```

Yesterday is history</p>

```
<p class="text-cyan-400 mt-2">
```

Tomorrow is a mystery.</p>

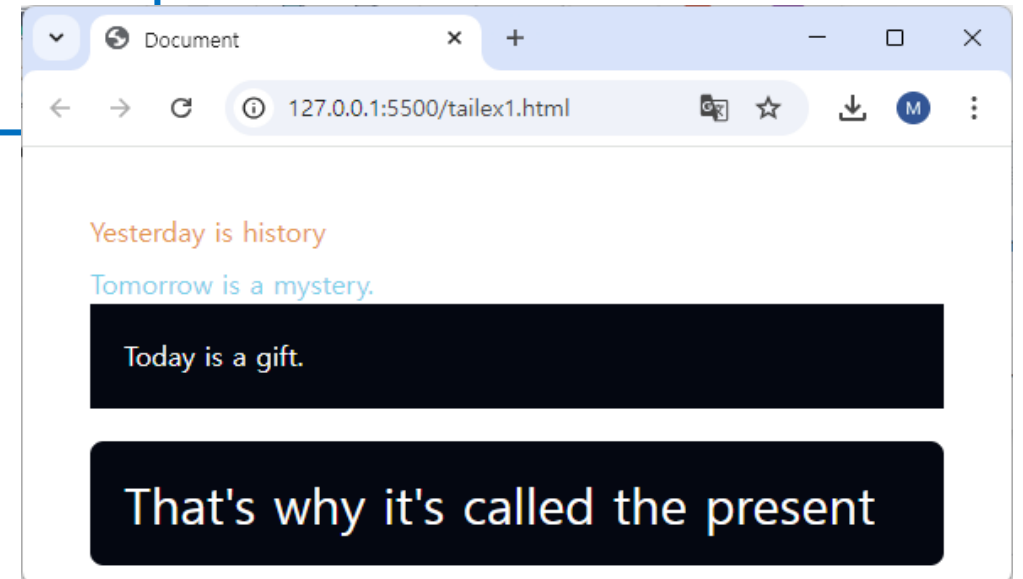
```
<p class="text-white bg-gray-950 p-5">
```

Today is a gift.</p>

```
<p class="text-white bg-gray-950 p-5 text-3xl rounded-lg mt-5">
```

That's why it's called the present</p>

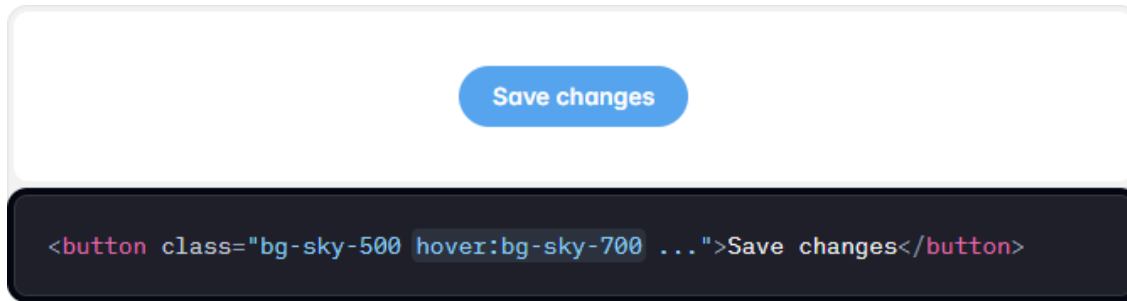
```
</body>
```



4. Tailwind CSS Framework로 UI/UX 구성하기

Hover, focus, and other states

- ❖ Tailwind의 모든 유틸리티 클래스는 대상으로 지정하려는 조건을 설명하는 클래스 이름의 시작 부분에 variant를 추가하여 조건부로 적용할 수 있음



❖ Tailwind의 variant

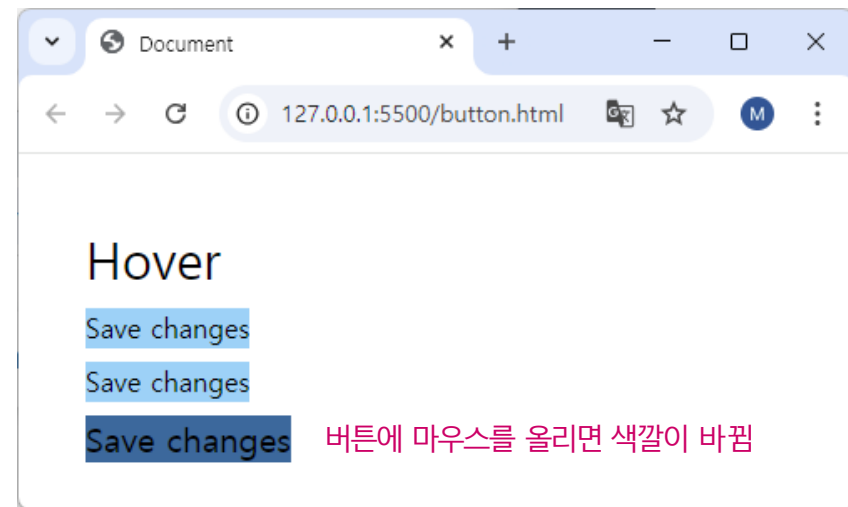
- 의사 클래스 (Pseudo-class)
 - ✓ :hover, :focus, :first-child, :required
- 의사 요소 (Pseudo-element)
 - ✓ ::before, ::after, ::placeholder, ::selection
- 미디어 및 기능 쿼리
 - ✓ 반응형 중단점, 다크 모드, prefers-reduced-motion
- 속성 선택자 (Attribute selector)
 - ✓ [dir="rtl"], [open]
- 자식 선택자 (Child selector)
 - ✓ &, >, *, & *

4. Tailwind CSS Framework로 UI/UX 구성하기

Hover(button.html)

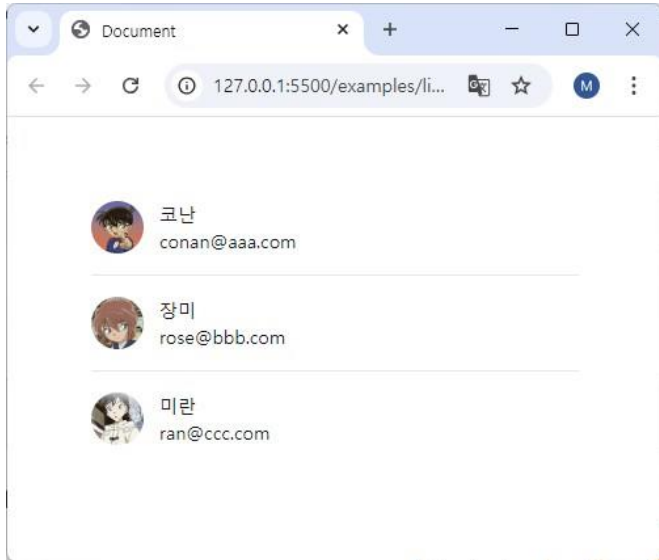
```
<body class="ml-10 mt-10 mr-10">
  <h1 class="text-3xl py-1">Hover</h1>

  <!-- bg-sky-300:배경색 mt-2: margin top -->
  <button class="bg-sky-300 mt-2">Save changes</button>
  <br />
  <button class="bg-sky-300 mt-2 hover:bg-sky-700">Save
changes</button>
  <br />
  <button class="bg-sky-300 mt-2 hover:bg-sky-700 text-xl">Save
changes</button>
  <br />
</body>
```



4. Tailwind CSS Framework로 UI/UX 구성하기

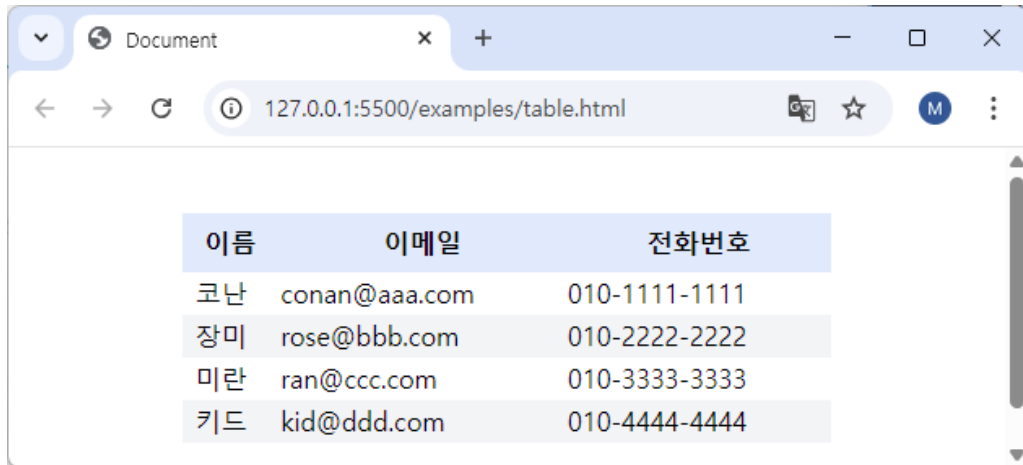
UI태그 : first last



```
<body class="mx-10 my-10">
  <div>
    <ul role="list" class="p-6 divide-y divide-slate-200">
      <li class="flex py-4 first:pt-0 last:pb-0">
        
        <div class="ml-3 overflow-hidden">
          <p class="text-sm font-medium text-slate-900">코난</p>
          <p class="text-sm text-slate-900 truncate"><a href="mailto:conan@aaa.com">conan@aaa.com</a></p>
        </div>
      </li>
      <li class="flex py-4 first:pt-0 last:pb-0">
        
        <div class="ml-3 overflow-hidden">
          <p class="text-sm font-medium text-slate-900">장미</p>
          <p class="text-sm text-slate-900 truncate"><a href="mailto:rose@bbb.com">rose@bbb.com</a></p>
        </div>
      </li>
      <li class="flex py-4 first:pt-0 last:pb-0">
        
        <div class="ml-3 overflow-hidden">
          <p class="text-sm font-medium text-slate-900">미란</p>
          <p class="text-sm text-slate-900 truncate"><a href="mailto:ran@ccc.com">ran@ccc.com</a></p>
        </div>
      </li>
    </ul>
  </div>
</body>
```

4. Tailwind CSS Framework로 UI/UX 구성하기

table 태그 응용 : odd, even



이름	이메일	전화번호
코난	conan@aaa.com	010-1111-1111
장미	rose@bbb.com	010-2222-2222
미란	ran@ccc.com	010-3333-3333
키드	kid@ddd.com	010-4444-4444

```
<div class="mt-3">
  <table class="table table-auto w-9/12 mx-auto">
    <thead>
      <tr class="bg-blue-100 h-9">
        <th>이름</th>
        <th>이메일</th>
        <th>전화번호</th>
      </tr>
    </thead>
    <tbody>
      <!-- Use different background colors for odd and even rows -->
      <tr class="odd:bg-white even:bg-gray-100">
        <td class="pl-2">코난</td>
        <td>conan@aaa.com</td>
        <td>010-1111-1111</td>
      </tr>
      <tr class="odd:bg-white even:bg-gray-100">
        <td class="pl-2">장미</td>
        <td>rose@bbb.com</td>
        <td>010-2222-2222</td>
      </tr>
      <tr class="odd:bg-white even:bg-gray-100">
        <td class="pl-2">미란</td>
        <td>ran@ccc.com</td>
        <td>010-3333-3333</td>
      </tr>
      <tr class="odd:bg-white even:bg-gray-100">
        <td class="pl-2">키드</td>
        <td>kid@ddd.com</td>
        <td>010-4444-4444</td>
      </tr>
    </tbody>
  </table>
</div>
```

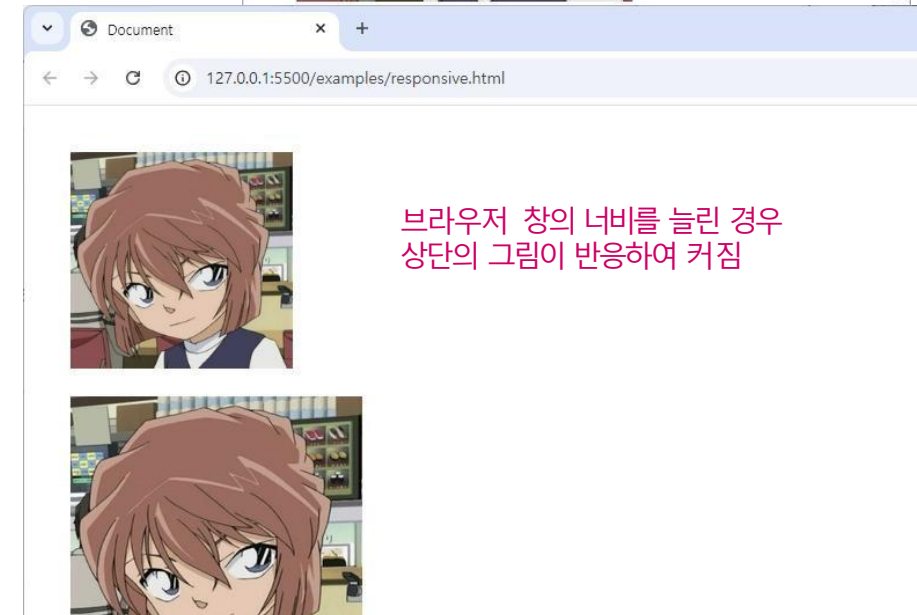
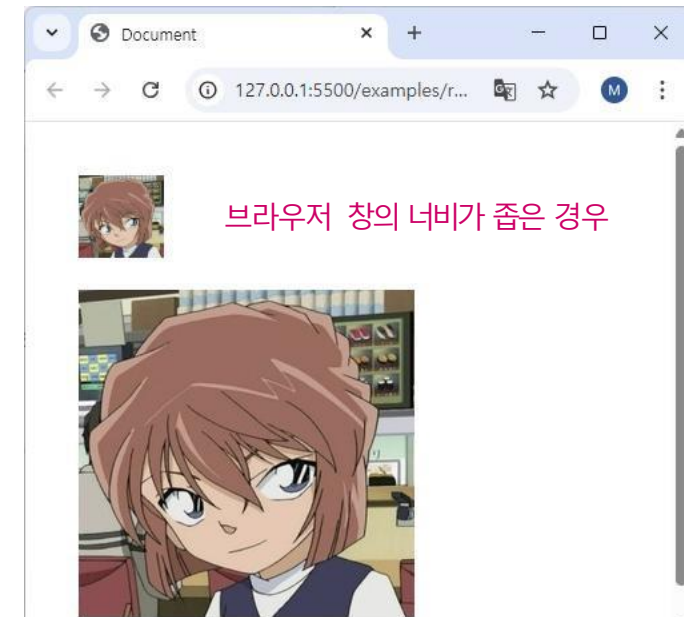
4. Tailwind CSS Framework로 UI/UX 구성하기

반응형 디자인 (1/2)

- ❖ Tailwind의 모든 유틸리티 클래스는 다양한 중단점에서 조건부로 적용할 수 있어서, 반응형 인터페이스를 쉽게 생성할 수 있음

```
<body class="mx-10 my-10">
  <!-- Width of 16 by default, 32 on medium screens, and 48 on large screens -->
  <br />
  
</body>
```

Breakpoint prefix	Minimum width	CSS
sm	40rem (640px)	@media (width >= 40rem) { ... }
md	48rem (768px)	@media (width >= 48rem) { ... }
lg	64rem (1024px)	@media (width >= 64rem) { ... }
xl	80rem (1280px)	@media (width >= 80rem) { ... }
2xl	96rem (1536px)	@media (width >= 96rem) { ... }



4. Tailwind CSS Framework로 UI/UX 구성하기

반응형 디자인 (2/2)

```
<body class="mx-10 my-10">
  <div class="max-w-md mx-auto bg-white rounded-xl shadow-md overflow-
hidden md:max-w-2xl">
    <div class="md:flex">
      <div class="md:shrink-0">
        
      </div>
      <div class="p-8">
        <div class="uppercase tracking-wide text-sm text-indigo-500 font-semibold">
홍장미</div>
        <a href="#" class="block mt-1 text-lg leading-tight font-medium text-black
hover:underline">
          장미의 나무위키 읽기
        </a>
        <p class="mt-2 text-slate-500">명탐정 코난의 서브 히로인이자 주인공 에도가와 코
난을 짝 사랑하고 있는 쿨하고 ....</p>
      </div>
    </div>
  </div>
</body>
```



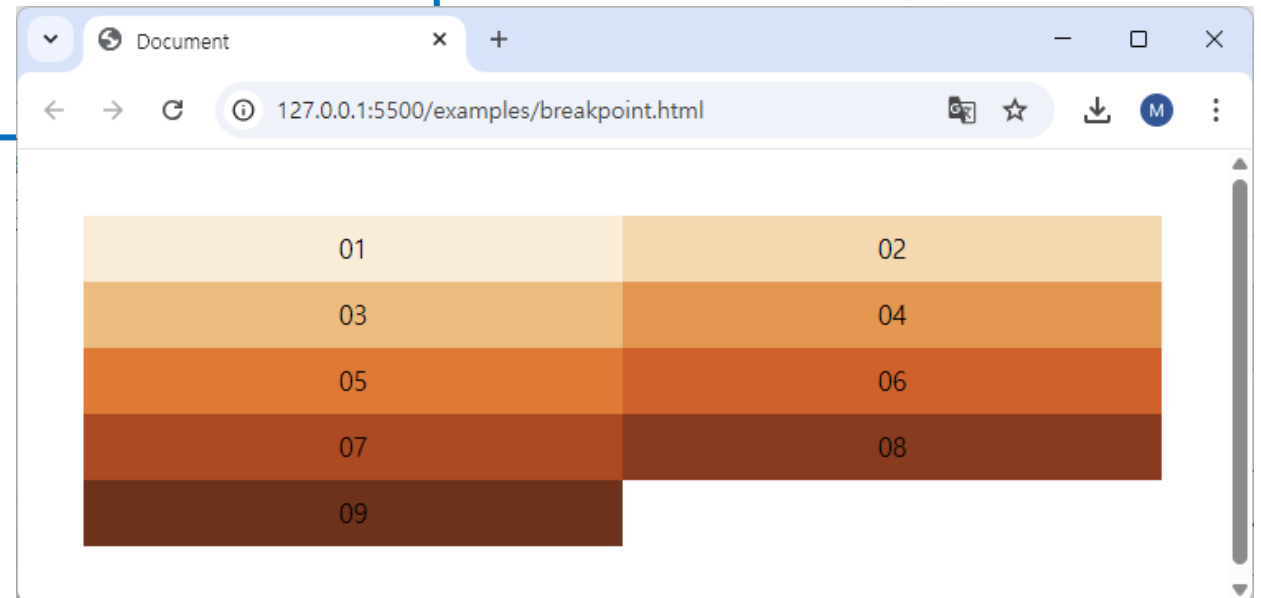
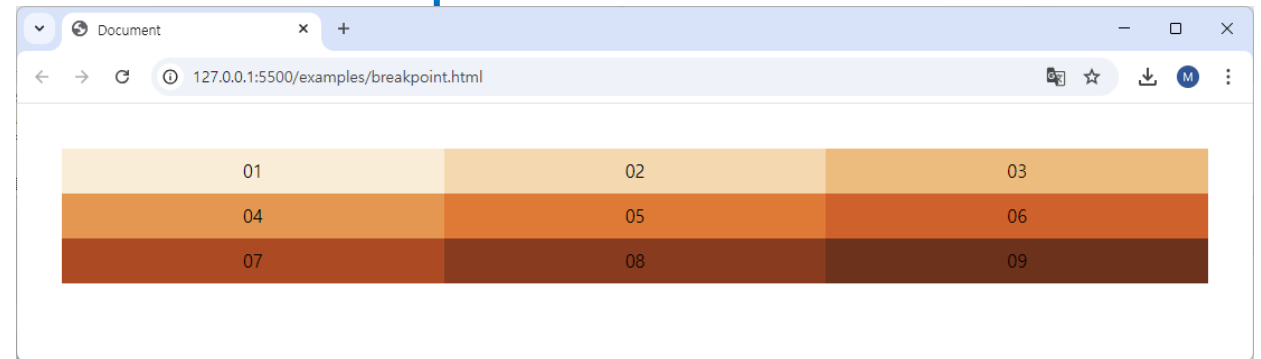
브라우저 창의 너비를 조절하면
레이아웃이 반응형으로 동작

4. Tailwind CSS Framework로 UI/UX 구성하기

사용자 정의 중단점

```
<body class="mx-10 my-10">
  <div class="grid grid-cols-1 tablet:grid-cols-2 laptop:grid-cols-3 desktop:grid-cols-4">
    <div class="bg-orange-100 text-center p-2">01</div>
    <div class="bg-orange-200 text-center p-2">02</div>
    <div class="bg-orange-300 text-center p-2">03</div>
    <div class="bg-orange-400 text-center p-2">04</div>
    <div class="bg-orange-500 text-center p-2">05</div>
    <div class="bg-orange-600 text-center p-2">06</div>
    <div class="bg-orange-700 text-center p-2">07</div>
    <div class="bg-orange-800 text-center p-2">08</div>
    <div class="bg-orange-900 text-center p-2">09</div>
  </div>
</body>
```

브라우저 창의 너비를 조절하여 중단점 확인하기

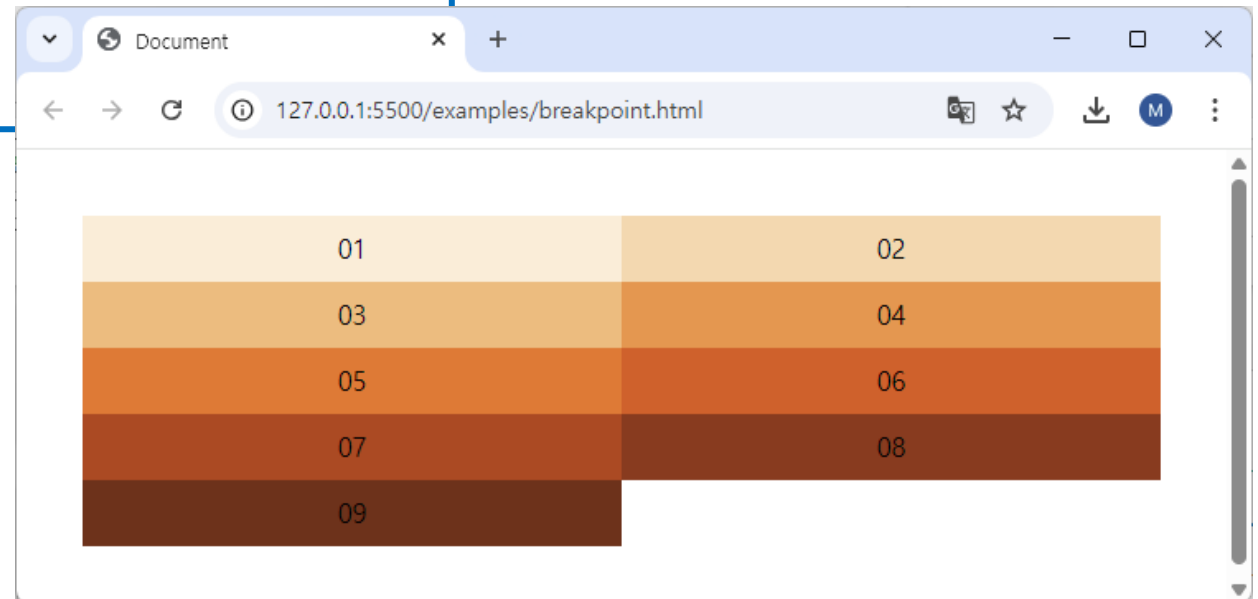
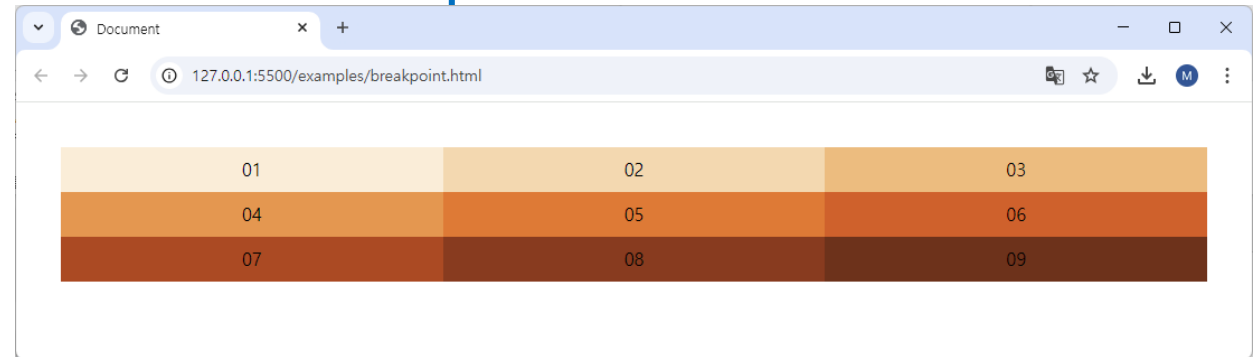


4. Tailwind CSS Framework로 UI/UX 구성하기

darkmode

```
<body class="mx-10 my-10">
  <div class="grid grid-cols-1 tablet:grid-cols-2 laptop:grid-cols-3 desktop:grid-cols-4">
    <div class="bg-orange-100 text-center p-2">01</div>
    <div class="bg-orange-200 text-center p-2">02</div>
    <div class="bg-orange-300 text-center p-2">03</div>
    <div class="bg-orange-400 text-center p-2">04</div>
    <div class="bg-orange-500 text-center p-2">05</div>
    <div class="bg-orange-600 text-center p-2">06</div>
    <div class="bg-orange-700 text-center p-2">07</div>
    <div class="bg-orange-800 text-center p-2">08</div>
    <div class="bg-orange-900 text-center p-2">09</div>
  </div>
</body>
```

브라우저 창의 너비를 조절하여 중단점 확인하기



연습문제

용도	중량	개수	가격
선물용	3KG	11~16과	3,5000원
	5KG	18~26과	52,000원
가정용	3KG	11~16과	3,5000원
	5KG	18~26과	52,000원

선물용과 가정용 상품구성

용도	중량	개수	가격
선물용	3KG	11~16과	3,5000원
	5KG	18~26과	52,000원
가정용	3KG	11~16과	3,5000원
	5KG	18~26과	52,000원

선물용과 가정용 상품구성

- 1.html과 css를 이용하여 다음과 같은 표를 작성하세요.
- 2.TailWindCSS를 이용하여 다음과 같은 표를 작성하세요.
3. 로그인 화면을 TailwindCSS로 만들어 보시오.

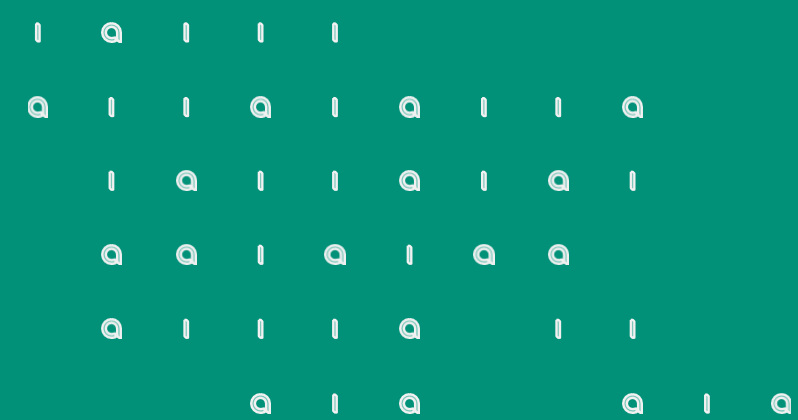
Login

Email

Password

[Regist](#)





5. Git 시작하기



5. Git 시작하기

DVCS (Distributed Version Control System)

Git: 2005 by **Linus** Benedict **Torvalds**

Mercurial: Python

BitKeeper: 1998, 유료

코드 공유 / 이력관리 / 협업 ⇒ 책임과 기록

cf. SCCS(1970), RCS(1980), **CVS**(1986), **SVN**(2000)

test repo: <https://github.com/indiflex>

5. Git 시작하기

Settings

Download Git Client

- <https://git-scm.com/downloads>

SCM Service Site

- [GitHub](#) : 무료는 public만 가능, 안정적(2022~ 모두 무료)
- [BitBucket](#) : public/private 모두 무료, 안정적인 편
- 기타: [GitLab](#), [Codebase](#), [CloudForge](#), etc

(참고) windows에서 GitHub에 한글 깨질때

- cmd"명령프롬프트"> **chcp 65001**

Free

- > Unlimited public/private repositories
- > 2,000 CI/CD minutes/month
Free for public repositories
- > 500MB of Packages storage
Free for public repositories
- > 120 core-hours of Codespaces compute
- > 15GB of Codespaces storage
- > Community support

<https://training.github.com/downloads/ko/github-git-cheat-sheet/>

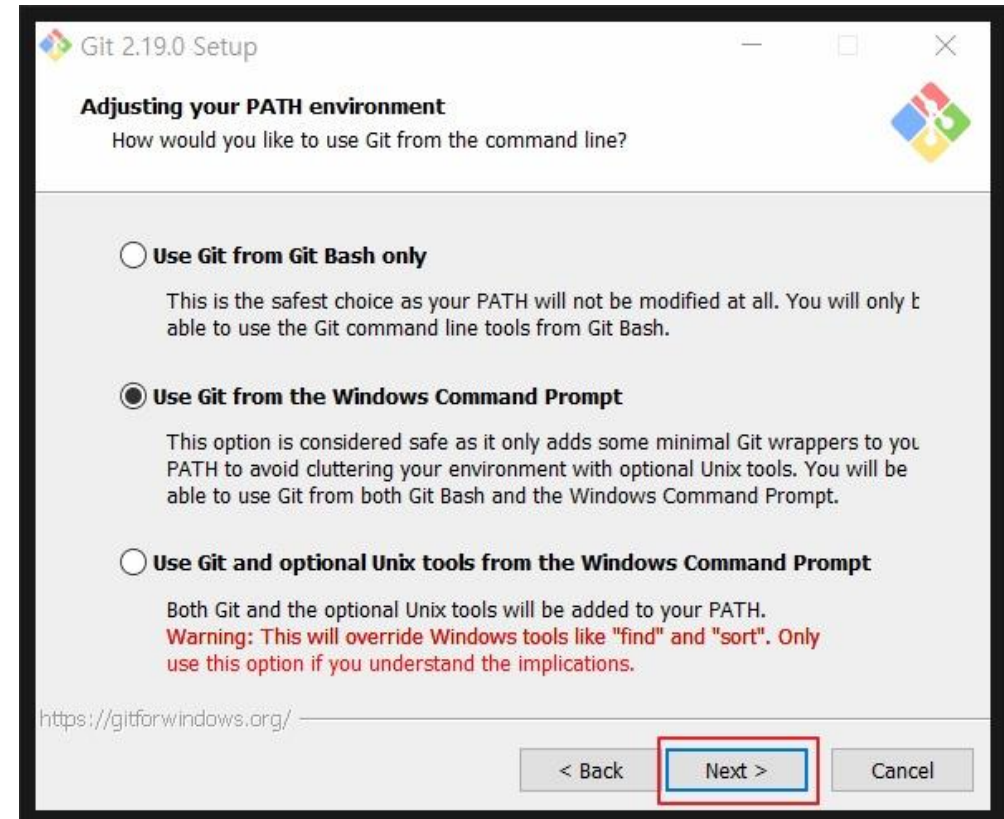
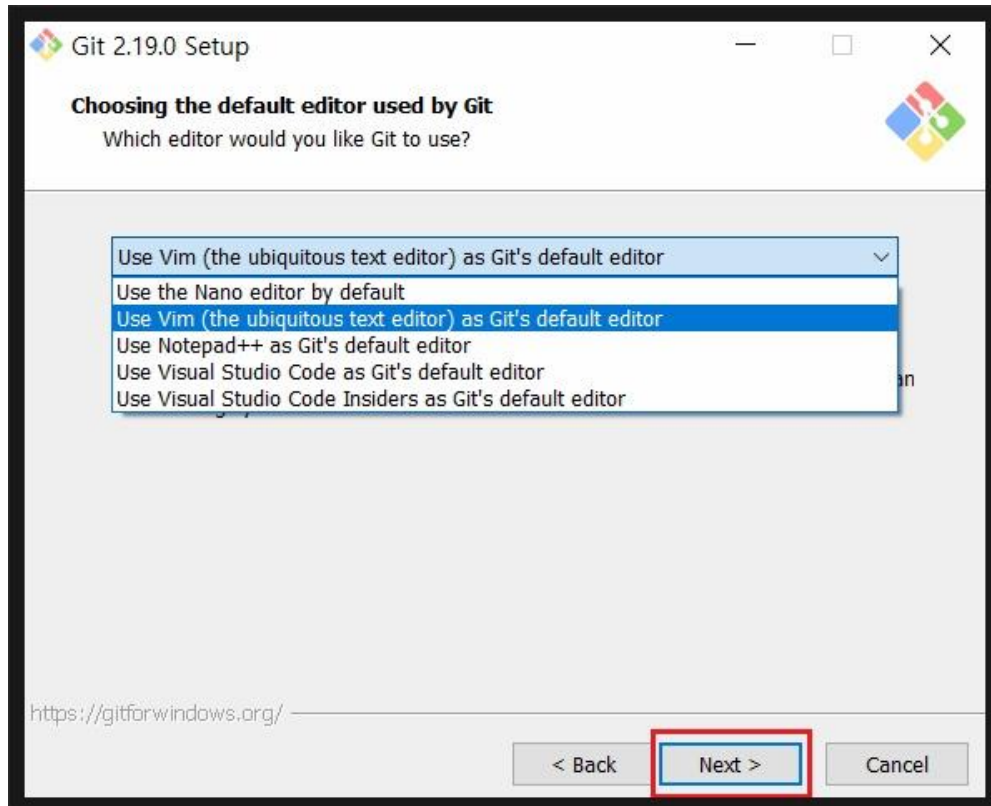
<http://www.blklime.com/cheatSheet.html>

```
# bash에서 prompt에서 branch 보이게
parse_git_branch() {
  git branch 2> /dev/null | sed -e '/^[^*]/d' -e 's/* \(.*\)/(\1)/'
}

export PS1='\e[0;33m[\t:\u \w]$ \e[m'
alias Ps2="export PS1='\e[0;33m[\W]$ '"
alias PsGit="export PS1='\[\033[32m\]\w\[\033[33m\]\$(parse_git_branch)\[\033[00m\] $ '"
```

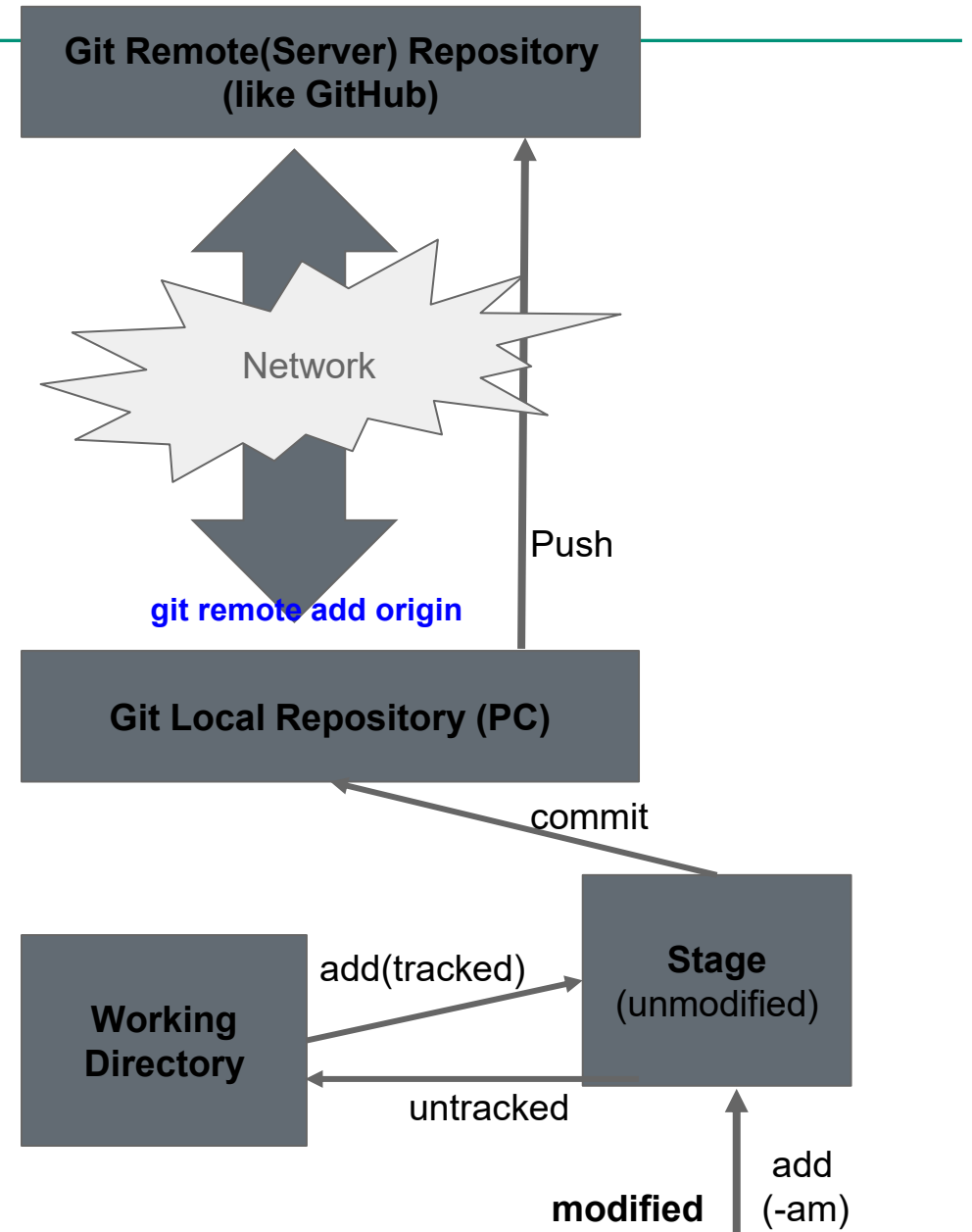
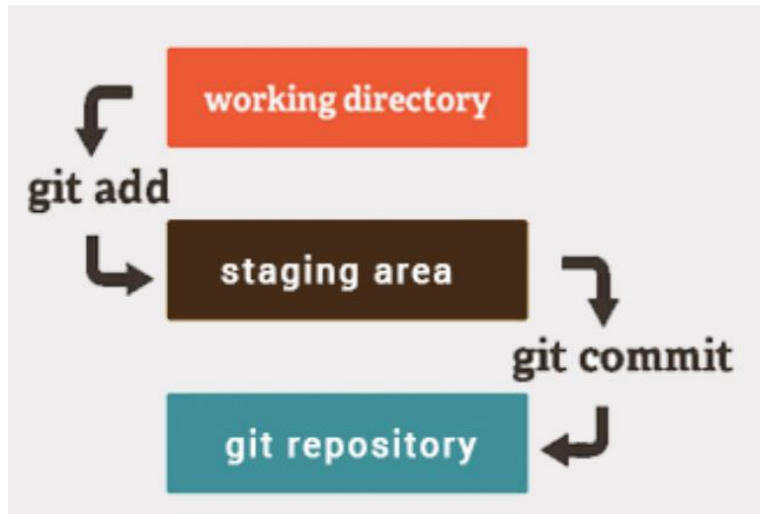
5. Git 시작하기

Settings



5. Git 시작하기

git structure



5. Git 시작하기

workingDirectory - stage - repository

working directory (working tree)

stage: 임시 저장 공간 (git status 또는 git ls-files --stage)

repository (local / remote)

cf. **untracked** vs **tracked**

(빈폴더 / .gitignore)

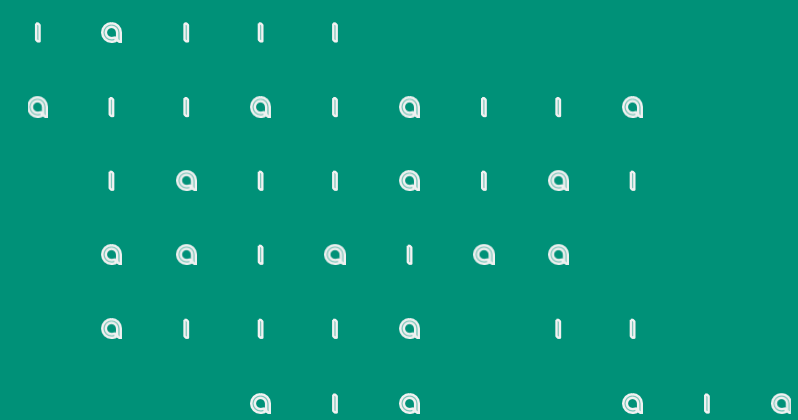
staged vs **unstaged**

git rm --cache 파일

modified vs **unmodified**

now working

★ 파일수정 → **modified(WD)** → **add** → **modified(Stage)** → **commit** → **unmodified(Repo)**



6. Git 기본 명령어



6. Git 기본 명령어

Git Config

1. # 바탕화면 `git bash` 실행
2. `git config --list`
`git config user.name`
`git config user.name "xxx"`
`git config --global user.name <github-name>`
4. `git config user.email`
`git config --global user.email <email>`
`git config --global credential.helper cache` # github 인증정보 cache 활성화
5. # `git config` 한눈에 보기 (`git` 활성화된 디렉터리)
`cat .git/config`
6. # unset (remove)
`git config --unset user.name`

6. Git 기본 명령어

로컬 저장소(Local Repository) 만들기

```
.vscode 1. cd <work-dir>
node_modules
*.local 2. git init          # .git 디렉터리 생성
3. vi .gitignore          # git 제외 대상 파일 (touch .gitignore && code .)
4. echo "ttt" > test.txt
5. git ls-files           # test.txt 없음(:untracked)
6. git add --all          # working dir ⇒ stage   git add -A 또는 git add .
   cf. git add <file명>   또는   git add .
7. git ls-files           # test.txt 보임(tracked file list): git이 알고있는 파일
   git status
8. git commit -m "first commit message"
9. git log                # commit logs
   git lg                 # logs graph + oneline + decorate
10. git status            # staging 상태 확인
```

```
# make unstaged (remote only)
$> git rm -r --cached htmlcss

# directory 용량 확인
$> du -sh htmlcss/tw/node_modules
```

recursive
(directory)

```
# .zshrc   .bash_profile
$> vi ~/.bashrc
# i 키 누르기
alias ll='ls -al'
# esc 키 누르기
:wq
```

```
git config --global alias.lg "log --color --graph --pretty=format:'%Cred%h%Creset -%C(yellow)%d%Creset %s %Cgreen(%cr) %C(bold blue)<%an>%Creset' --abbrev-commit --"
```

6. Git 기본 명령어

리모트 저장소 (Remote Repository)

indiflex

view Repositories 58 Projects Packages Stars 2

Find a repository...

Type Language Sort

Create a new repository

A repository contains all project files, including the revision history. Already have a repository elsewhere? [Import a repository.](#)

Required fields are marked with an asterisk (*).

Repository template my profile > settings > repositories

No template

Start your repository with a template repository's contents.

Owner * Repository name *

indiflex /

Great repository names are short and memorable. Need inspiration? How about...

Description (optional)

1. `cd <work-dir>`
`github remote repository` 생성 선행
2. `git remote add origin <git-remote-url>`
`cat .git/config`
`git remote show [origin]`
3. `git branch -M master` # main → master
4. `git push -u origin master`
`git push` # -u 옵션으로 파이프라인 이후!
5. 오류시 처리
`git pull origin master` # pull 먼저
`git pull origin master --allow-unrelated-histories`
6. `git log`
7. `git status` # staging 상태 확인
`git ls-files -s` # `git ls-files --help`
8. `mv test.txt ttt.txt`
`git ls-fiels -m` # -c, -i, -o
`git ls-fiels -d`

6. Git 기본 명령어

remote 명령어

Local | HTTP(s) | SSH | Git

git remote -v

```
$> git remote add <별칭:origin> https://github...
```

```
$> git remote rename <before> <after> // 별칭 변경
```

```
$> git remote rm origin // origin 저장소 연결 해제
```

```
$> git (push|pull) [-u] origin <branch> // -u: set-upstream
```

```
$> git remote show origin // origin 정보 보기
```

6. Git 기본 명령어

git clone / push / pull

Remote repo와 클론하기

git clone <github-remote-url> [dir-name]

#git pull : 서버의 최신 소스를 다운받기

#git push : 서버에 local repo 올리기

1. **git pull**

2. touch nnn.txt # modified or new

git **diff** --name-only # diff for HEAD

git **diff** HEAD~1 # 이전 커밋과의 diff

3. **git add --all** # git add -A

4. **git commit -m "message"**

git commit -am "msg" # modified → stage → repository

5. **git push -u origin master**

cf. git push

(참고: 서버와 연결된 repo 끊기)

git remote rm origin

6. Git 기본 명령어

restore (make unstage)

원본 —수정→ modified(WD)
—git add→ staged

staged —restore --staged→ modified(WD)
—restore→ 원본!

1. `echo "bb" >> ttt.txt` # modified(WD)
2. `git status`
3. `git add -A` # modified(staged)
4. `git status`
5. **`git restore --staged ttt.txt`**
6. `git status` # modified(WD)
`git ls-files`
7. **`git restore ttt.txt`** # unmodified(WD)
(sameAs **`git checkout ttt.txt`**)
"수정 전 상태(HEAD)로 돌아옴"

cf. `git checkout <CommitID>`

6. Git 기본 명령어

rm (make untracked)

tracked — **rm --cache** →
untracked(WD)
& deleted(Stage)

— **restore --staged** → unmodified(Stage)
원위치(취소됨)!

```
$> git ls-files -s # staged file list
$> git ls-files -u # conflicted file list
```

1. `echo "ttt" > ttt.txt` # new file(untracked)
2. `git add .` # tracked & staged
3. `git commit -m 'ttt'` # committed
4. **git show** # last commit log
5. `git log` # commit log
git log --raw # with filename
`git log --graph` # git log --oneline
`git reflog --raw` # with HEAD pointer
6. **git rm --cache** ttt.txt
파일 원본은 나두고, cache에서만 삭제해줘!
7. `git status`
8. **git ls-files** # tracked file list

주의!) **--cache option**을 붙이지 않으면 완전 삭제됨!!

git rm aaa.txt # deleted(완전삭제!)

복구 방법)

restore --stage ⇒ deleted(WD) ⇒ **restore**

Local

Remote

Working Directory

Stage

Local Repository

Remote Repository



hello.txt

1. 파일 생성

untracked file : hello.txt

add



hello.txt

2. tracked 시작

new file : hello.txt

commit



hello.txt

3. 작업완료

working tree clean

push



hello.txt

4. 원격저장소에 업로드

add
2. 다시 staging
modified : hello.txt

case 1)
hello.txt 의 내용을 수정한 경우

working directory로 이동

changes
not staged
modified : hello.txt

working directory에 파일 복구

cache 옵션을 사용하면, 파일 원본은 남기고
stage에는 deleted 상태로 만들수 있다.

case 2)
hello.txt 을 삭제한 경우

git rm --cache hello.txt

git restore hello.txt

Untracked files:
hello.txt

deleted : hello.txt

Changes not staged for commit:

deleted : hello.txt

git restore --staged hello.txt

deleted : hello.txt

git rm hello.txt

add

commit

push

연습문제

Try This -restore

실수로 tracked 파일을 수정했다.
restore를 사용하여,
다음 각 상황별로 원위치해보자.

1. 아직 add를 하지 않았다.
2. add 까지 했다(staged)

1. `echo "777" >> nnn.txt`

⇒ `modified`

⇒ 수정 취소하기

`git restore nnn.txt`

2. `echo "888" >> nnn.txt`

`git add nnn.txt`

⇒ 수정 취소하기

`git restore --staged nnn.txt`

`git restore nnn.txt`



연습문제

Try This -rm

commit된 파일을 untracked하려다
실수로 --cache 옵션을
붙이지 않아 원본이 삭제되었다.

이를 복구해보자.

`git rm --cache nnn.txt` 해야 할 것을 실수로
`git rm nnn.txt` 하였다. 이를 복구해보자.

1. `git rm nnn.txt`

2. 방법1

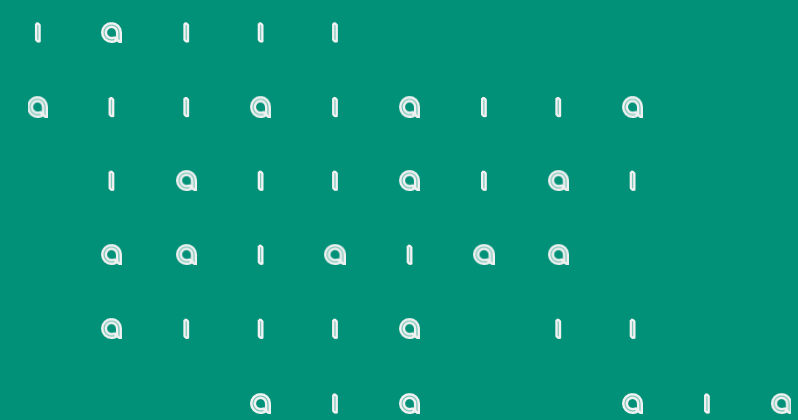
`git restore --staged nnn.txt`

`git restore nnn.txt`

3. 방법2

`git checkout HEAD nnn.txt`





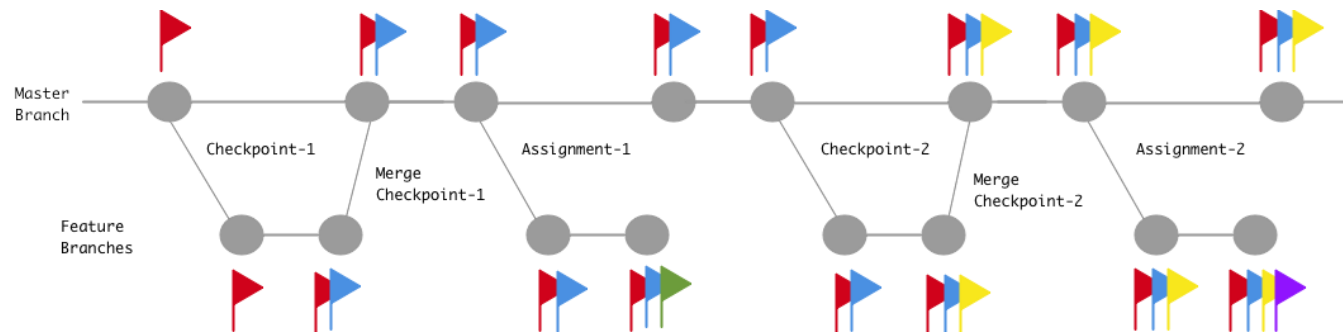
7. 깃 브랜치(Branch)



7. 깃 브랜치(Branch)

Branch

- * 기본(head) branch는 master(or main)
(`git config --list` ⇒ `init.defaultbranch`)
- * 가상 폴더 생성 (가상 폴더에는 해당 branch의 commit 들이 존재)
 - 특정 commit pointer로 사본(가상폴더) 생성
 - master branch는 HEAD pointer 유지
- * 독립적인 작업 공간(merge하지 않는 한 별개의 소스, 독립적인 commitID들로 구성)
 - checkout(switch)하면 WD도 해당 브랜치로 변경!
- * 빠른 동작(41B Blob - Binary Large Object)



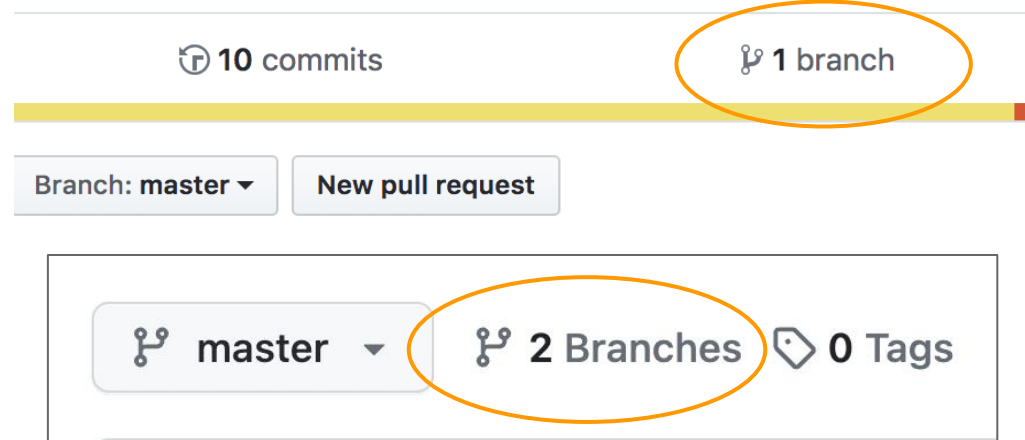
7. 깃 브랜치(Branch)

branch 생성

`git branch <Branch> [CommitId]`

test code

[Manage topics](#)



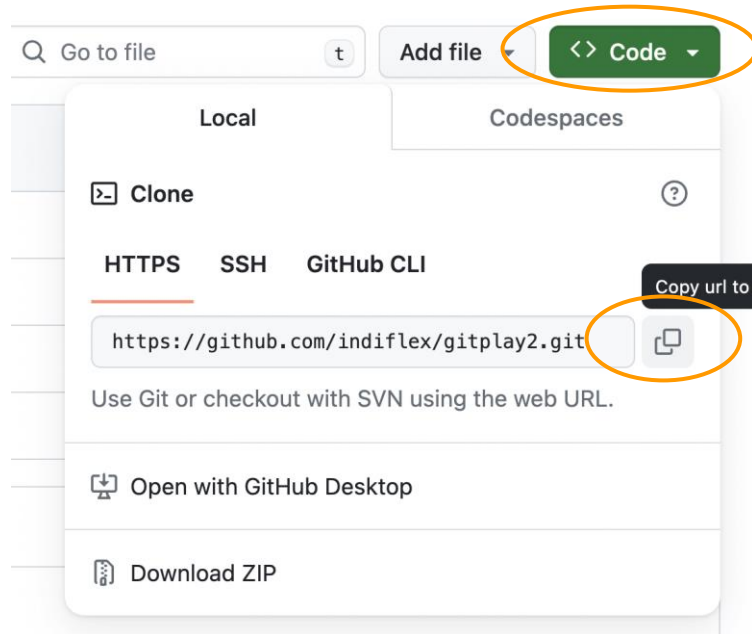
1. `git branch <branch>` # branch 만들기
2. `git branch` # branch 전체 보기
`git log -1` # HEAD log only
3. `git checkout <branch>` # 전환
`git switch <branch>` # 전환
`git checkout -` # 이전 브랜치
4. `git checkout -b <branch>` # 생성 + 이동
`git switch -c <branch>` # 생성 + 이동
5. 소스 수정 & add & commit # branch 실제로 생성!
6. `git push origin <branch>`
`git push -u origin <branch>`
7. `git branch -v` # 또는 `-vv`
8. `git branch -r` # remote
9. `git branch -a` # local + remote
10. `git log <branch>`
`git log <branch> --graph [--all]`
11. `git rev-parse <commit/branch>`
`git rev-parse HEAD^1` # show commit HASH

7. 깃 브랜치(Branch)

branch with remote

merge 취소!

\$> git merge --abort



`git branch <branch> origin/<branch>`

1. `git remote -v` # 원격 저장소 링크 + tracking
`git remote show origin` # 링크 + 정보
`git branch -[av | rv | avv]`
2. `git push -u origin <branch>`
`git push origin <org-name>:<new-name>`
3. `git config pull.rebase false` # merge def.
4. 다른폴더> `git clone <remote-url>` # master only
OR

다른동료폴더> `git pull origin <branch>`

`git branch <branch> origin/<branch>`

`git checkout -b <branch> origin/<branch>`

`git checkout --track origin/<branch>`

cf. `git pull --rebase origin develop`

`git clone -b <branch> <remote-url> [dir]`

ex) A 브랜치 내용을 로컬의 B 브랜치로 merge

B> `git merge A` # local A → B

B> `git fetch` # fetch before merge

B> `git merge origin/A` # remote A → B

7. 깃 브랜치(Branch)

commit & log

HEAD: 변화 내용에 대한 snapshot

AHEAD **vs** **BHEAD** (behind)

(**local** before push) (**remote** before pull)

```
$> git commit -m [또는 -am] "message"
```

```
$> git commit -F <file> -m [또는 -am] "message"
```

```
$> git commit --allow-empty-message -m ""
```

```
$> git log [--pretty=[short|online] | -p | --stat | --graph --all]
```

```
$> git (show|diff) <CommitID 또는 head>
```

cf. `$> git diff HEAD~3..HEAD d.txt`

7. 깃 브랜치(Branch)

branch 삭제

```
git branch -d <branch>  
git branch -D <branch>
```

```
git push -d origin <br>  
git fetch -p
```

```
* remote origin에서 삭제된 branch 정리(sync)하기  
$> git remote prune origin
```

```
* remote origin에서 삭제된 branch 되살리기  
github> restore branch
```

1. **git checkout -b xx** # xx 브랜치 생성 + 이동
git branch xx # 생성
2. **git branch -d xx** # xx에서 xx를 지울 수 없음!
3. **git checkout -** # git checkout master
4. **git branch --delete xx** # 삭제 됨
5. **git branch -D xx**
merge 또는 commit 할게 남았다면 -D로 **강제 삭제!!**

ex) push를 이용하여 remote branch 삭제하기

```
$> git push --delete origin <branch>
```

```
$> git push -d origin <branch>
```

```
$> git fetch -p # prune  
# 다른 폴더에서는 fetch -p해야 적용!
```

7. 깃 브랜치(Branch)

cf. github bug repo history 완전 삭제

BGF 설치

- <https://rtyley.github.io/bfg-repo-cleaner/>

삭제 작업

1. 파일 삭제
2. add & commint & push
3. git clone --mirror <url>
4. **bfg** --delete-files
5. push

```
$> git clone --mirror git@github.com:indiflex/hello.git
$> cd hello.git
$> java -jar ../bfg.jar --delete-files bigquery.json .
$> git reflog expire --expire=now --all && git gc --prune=now
--aggressive
$> git push
```

소스 저장소로 이동후!!

```
$> git pull --allow-unrelated-histories
```

연습문제

Try This -branch#1

다음 두 개의 브랜치를 만들고 각각 test.txt를 수정한 다음 remote에 push 하시오. (br22는 local에서는 삭제)

* branch start commit

br11: master의 HEAD

br22: master의 HEAD~1

1. **br11 branch 생성**
master>
2. **br22 branch 생성 (HEAD^)**
master>
3. **br11, br22에서 각각 test.txt 수정**
br11> vi test.txt
br22> vi test.txt
4. **push br11, br22 to remote**
master>
5. **br22 삭제 in Local**
master>

```
git branch -a
git branch br11
git branch br22 HEAD~1
git log --oneline
git log --oneline -3
git checkout br11
git log --oneline -3
git checkout br22
git log --oneline -3
git push -u origin br22
git checkout -
git push -u origin br11
git branch -d br22
git branch -a
```

연습문제

Try This -branch#2

이전 페이지에서 만든 **br22** 브랜치를 다른 폴더(others)에서 clone 하고, ppp.txt를 **생성**하고 commit & push하시오.
(단, br22외 다른 브랜치(master 등)는 clone하지 마시오!)

```
git fetch --all  
git checkout -b br11 origin/br11  
git checkout -b br22 origin/br22
```

1. 다른 폴더에서 clone (폴더는 자동 생성)

others>

2. ttt.txt 생성

3. commit & push

```
git clone -b br22 https://github.com/indiflex/kc4.git kc4br22  
cd kc4br22/  
cat .gitignore  
echo "ppp" > ppp.txt  
git add .  
git commit -m "first ppp"  
git push
```

연습문제

Try This -branch#3

develop에서 br33 브랜치를 만들고,
br11과 br22를 merge 한 후,
br11과 br22는 모조리 삭제하시오.
(local & remote & every directory)

```
master> git checkout -b br33 origin/br22
br33> git merge origin/br11
br33> git checkout master
master> git branch -D br11
master> git branch -D br22
master> git push origin -d br11
master> git push origin -d br22
```

1. br33 생성 from develop
2. br11과 br22를 br33으로 merge
3. br11과 br22 모두 삭제

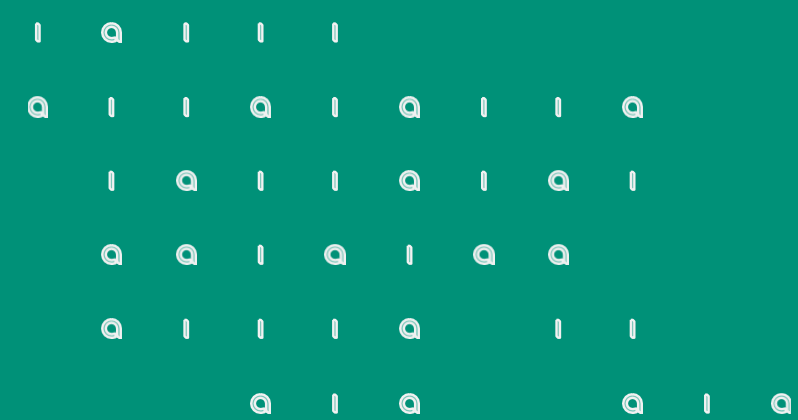
* remote origin에서 삭제된 branch 정리(sync)하기
\$> **git remote prune origin**



연습문제

1. develop 브랜치에서 feature 브랜치를 생성하고, ppp.txt를 수정한 후, add & remote에 push하시오.
2. 다른 폴더에서 feature 브랜치만 clone 하시오.
(폴더명은 feature)
3. feature 폴더에서 aaa.txt 파일을 생성 & commit & push
4. 원래 폴더로 돌아와 develop 브랜치에 feature 브랜치를 merge 하시오.
5. feature 브랜치를 모두(원래폴더 및 feature 폴더) 삭제하시오.
(remote에서도 삭제)

1. develop> git checkout -b feature
2. feature> vi ppp.txt
feature> git commit -am "xxx"
3. feature> git push -u origin feature
4. other-dir> git clone -b feature
<https://github.com/<id>/kc4.git> ff
5. other-dir> cd ff
6. ff> echo 'aaa' > aaa.txt
ff> git add . & git commit -m "xx"
7. ff> git push
8. cd <org-dir>
9. feature> git pull
10. feature> git checkout develop
11. develop> git merge feature
12. develop> git branch -d feature
13. develop> git push origin -d feature



8. 깃 스테시(stash)와 클린(clean)



8. 깃 스테시(stash)와 클린(clean)

stash (임시 저장)

작업중인데 갑자기 긴급 수정사항이?!
(아직 commit하면 안되는데...)

작업중....

git stash **save** (임시저장,작업내역제거)
긴급 작업 & commit

git stash **pop** (임시저장내역 다시적용)

WD와 stage는 branch들의 공유 공간!

* **Cannot switch branch before working-tree clean!**
(commit전에는 branch switch 하면 안됨)

* commit하기 전에 갑자기 긴급 수정사항이 있다면?!
⇒ 현재 작업중인걸 임시 저장하고 패치하자!

* **git stash [save "label"]** // STACK 구조

Saved working directory and index state WIP on
<branch>: <CommitID> <Last Commit Message>

⇒ 작업중이던 내용은 **stash**로 임시저장되고 사라짐!

⇒ 이제 switch 가능!

* 여러개의 stash가 stack에 저장(save & pop)

WIP(Work In Progress)는 stash@{num}형태

* **cat .git/refs/stash**

git stash list

8. 깃 스테시(stash)와 클린(clean)

stash (Cont'd)

```
git stash list
      show
      -p <stash WIP>
      pop
      drop
      branch
      apply
```

```
$> git stash [save "label"]    # stash 저장
```

```
$> git stash [--include-untracked]
```

```
$> git stash list              # stash 목록
```

```
$> git stash show -p          # stash log
```

```
$> git stash show -p <WIP>    # 상세 log
```

이제 패치(patch) (switch & edit & commit ... 모두) 가능

```
$> git stash pop              # LIFO (auto-merge!)
```

```
$> git stash drop            # conflict시 삭제안된 내역까지
삭제(Last In First Drop)    "stash 취소"
```

8. 깃 스테시(stash)와 클린(clean)

stash branch

(안전한 복구를 위해 갈아타기!)

git stash branch <branch>

현재 작업중인 내역으로 새로운 브랜치 생성

```
$> git stash # stash 저장
```

```
# stash 저장된 내역으로 branch 생성
```

```
$> git stash branch <new-branch-for-stash>
```

```
wbr> git stash branch st1 # stash로 st1 만들
```

```
st1> git commit -am "working..." # wbr의 작업내역 사라짐!
```

```
st1> git branch -vv
```

```
st1> git stash list # 저장된 stash drop됨
```

```
st1> git switch wbr # 작업할 br로 이동
```

```
wbr> ... edit & commit... # 작업
```

```
wbr> git switch st1
```

```
st1> # commit하지 못한 작업 이어서 하기
```

8. 깃 스테시(stash)와 클린(clean)

stash apply

(특정 작업중 상태로 복원!)

git stash **apply** [stashID]

어느 브랜치건 stash로 임시 저장해 놓은
작업중인 상태로 복원하기
(커밋된 내역은 유지하고, 다만 작업중이던 내역만 복원)

```
$> git stash [save "label"]    # stash 저장  
⇒ 작업 전(마지막 commit)상태로 돌아감  
(주의: git stash 할 때마다 stashID는 변경됨)
```

```
$> git stash list
```

```
$> git stash apply stash@{1}    # label 상태로 복원  
⇒ stash는 사라지지 않고 그 상태로만 복원
```

```
$> git stash drop              # 마지막 stash 삭제
```

```
$> git stash drop stash@{1}    # 특정 stash 삭제
```

8. 깃 스테시(stash)와 클린(clean)

git clean

(untracked file 삭제)

git clean **-n** (내역만 확인, dry-run)

git clean **-d** (untracked files only)

git clean **-x** (with ignore too)

git clean **-X** (only ignore files)

```
$> git ls-files
```

```
$> git ls-files -o
```

```
$> git ls-files -o | grep -v node_modules
```

```
$> git clean -h # clean 도움말(옵션 보기)
```

```
$> git clean -n # clean 시 삭제 될 파일 목록 출력  
⇒ .gitignore에 정의된 파일은 대상이 아님
```

```
$> git clean -di # 안전하게 interactive하게!  
⇒ ignore file외 untracked 모두 삭제(하위폴더포함)
```

```
$> git clean -df # 묻지도 따지지도 않고 삭제
```

```
$> git clean -xi # 안전하게 interactive하게!  
⇒ ignore file(x)포함 모든 untracked 삭제
```

```
$> git clean -xf # 묻지도 따지지도 않고 삭제
```

8. 깃 스테시(stash)와 클린(clean)

```
~/workspace/kc4(develop) $ git clean -di
Would remove the following items:
  aaa.txt  bbb.txt
*** Commands ***
  1: clean          2: filter by pattern    3: select by numbers    4: ask each
  5: quit           6: help
What now> 2
  aaa.txt  bbb.txt
Input ignore patterns>> b*
  aaa.txt
Input ignore patterns>>
Would remove the following item:
  aaa.txt
*** Commands ***
  1: clean          2: filter by pattern    3: select by numbers    4: ask each
  5: quit           6: help
What now> 1
Removing aaa.txt
```

l a l l l
a l l a l a l l a
l a l l a l a l
a a l a l a a
a l l l a l l
a l a a l a

9. 깃 머지(merge)와 리베이스(rebase)

l l
a l
l a l a l
l a a a
l l a a l
l l a l a l l
l l a l l a a l
a l a l a a l

9. 깃 머지(merge)와 리베이스(rebase)

merge (병합)

* 병합의 종류

- Fast-Forward merge
- 3-Way merge
- Rebase

1. Fast-Forward merge

시간의 흐름대로 커밋된 내역을 병합

충돌(conflict) 발생 없고 100% Auto-merge

merge 후 모든 commit이 복제

2. 3-Way merge

두 개 이상의 브랜치로 파생된 커밋을 병합

충돌(conflict) 가능성 있음

병합 메시지(merge commit) 존재

3. Rebase

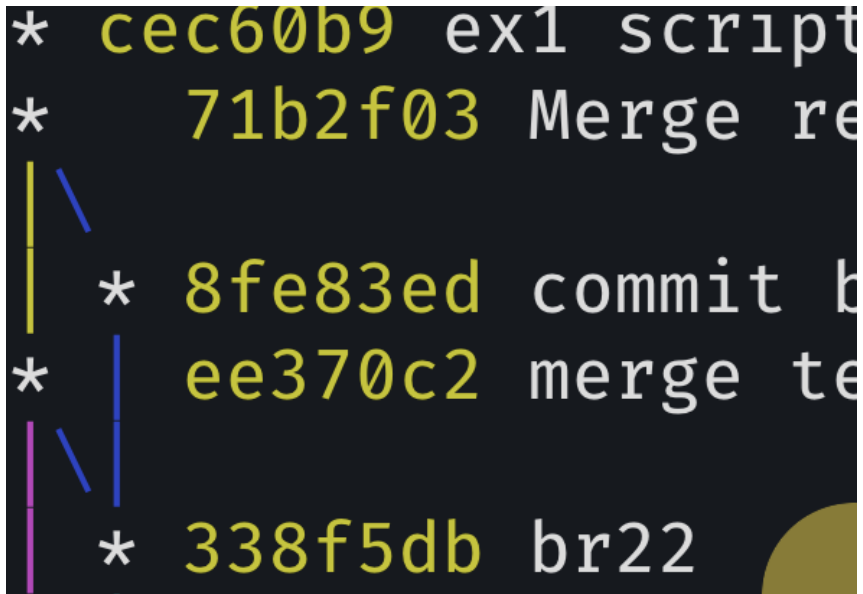
공통조상(base) 병합

3-Way ⇒ Fast-Forward 화

9. 깃 머지(merge)와 리베이스(rebase)

Fast-Forward merge

\$> git log --graph --oneline



```
* cec60b9 ex1 script
* 71b2f03 Merge re
|
| * 8fe83ed commit b
| * ee370c2 merge te
|
| * 338f5db br22
```

```
master> git checkout -b br11 # 작업 branch
```

```
br11> edit & commit # 작업(커밋1)
```

```
br11> edit & commit # 작업(커밋2)
```

```
br11> git switch master # 기준 branch
```

```
master> git merge br11 # merge
```

⇒ master로 커밋1, 커밋2 모두 복제

cf. git merge --abort # 병합 취소

```
master> git log --graph # 확인
```

```
master> git branch --merged # merge 확인
```



```
git5 (master) $ git branch --merged
br11
* master
```

```
master> git branch -d br11 # 완료 제거
```


9. 깃 머지(merge)와 리베이스(rebase)

3-Way merge

- 공통 조상(base) [1]
- 작업 브랜치(feature) [2]
- 또 다른 작업 브랜치(master) [3]

\$> git log --oneline -5 --graph

```
git5 (master) $ git log --graph
*   commit a7cc21b5b7f1ad4e9986d5c4c9705ccda8e990dd (HEAD -> master)
   \
    Merge: b7533b9 3eb52b1
    Author: indiflex <indiflex1@gmail.com>
    Date:   Tue Jan 2 14:15:23 2024 +0900

        Merge branch 'br11'

*   commit 3eb52b1e3a4b5d6da4532ea7fb79598898c6ed1e (br11)
    Author: indiflex <indiflex1@gmail.com>
    Date:   Thu Dec 28 19:08:53 2023 +0900

        aaa

*   commit b7533b92d1916c23ce5097067c4e2d2b117fb913 (origin/master)
   /
    Author: indiflex <indiflex1@gmail.com>
    Date:   Thu Dec 28 19:16:00 2023 +0900

        aa
```

develop> **git checkout -b feature** # 작업 branch 생성

feature> **edit & commit** # 작업(커밋F1)

feature> **edit & commit** # 작업(커밋F2)

feature> **git switch master** # 기준 branch

⇒ 아직 feature를 merge하지 않아 master는 수정 전.

develop> **edit & commit** # 작업(커밋M)

⇒ master에서도 작업

develop> **git merge feature [-e | --edit]**

3-way(F1 + F2 + M) merge

develop> **git log --graph** # merge 확인

develop> **git ls-files -u** # 충돌확인(unmerged)

⇔ **grep -R --exclude-dir=node_modules "<<< HEAD" ***

develop> **git branch -d feature** # 완료 제거

cf. **git merge --abort** # merge 취소!

9. 깃 머지(merge)와 리베이스(rebase)

conflict & merge

```
README.md ! x
i README.md > abc # 11111
You, 24 seconds ago | 2 authors (You and others)
1  ## git test
2  11111
3  22222
Accept Current Change | Accept Incoming Change
4  <<<<<< HEAD (Current Change)
5  33-local
6  =====
7  333-remote
8  >>>>>>
c4b83aaa34a455a050931d25d13f3418d7
```

cf. **conflict** 발생 시 비교

```
$> git diff --ours
```

```
$> git diff --theirs
```

```
$> git diff --base
```

- local과 remote 동시 수정
git pull # commit before merge!!
git commit -am "333-local"
git pull # conflict!!
cf. **git config pull.rebase false**

- **conflict**

```
<<<< HEAD
```

```
local(me, ours) 변경 내역
```

```
=====
```

```
remote(others, theirs) 변경 내역
```

```
>>>> sha1 id
```

- **git ls-files -u** # conflict files
- **editor에서 수정 & 저장 후** 아래 명령 수행
git commit -am "merge!"
- **git log --graph --oneline -5**

```
* cfd4413 (HEAD -> master) mege!
| \
|  * c4b83aa (origin/master) Update README.md
|  * 29b1262 3-local
```

9. 깃 머지(merge)와 리베이스(rebase)

cf. merge by fetch

- **pull** by **auto-merge**
- **fetch** by **manual-merge**

pull vs fetch

* **pull = fetch + merge**

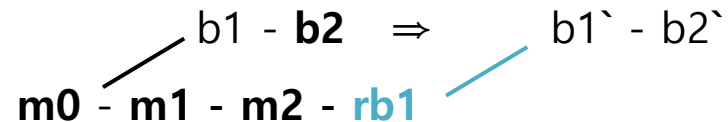
1. github에서 README.md 추가(또는 수정)
2. **git pull** # **fetch & auto-merge**
cat README.md
3. github에서 README.md 수정(변경)
4. **git fetch -p** # 임시 브랜치로 pull
cat README.md # 변경 내역 안보임!
5. git log **origin/master** README.md
show remote log (fetch이후에만 보임!)
6. **git merge origin/master**
7. git log README.md
cat README.md

9. 깃 머지(merge)와 리베이스(rebase)

Rebase

(내 base를 master HEAD로!)

- re-base(base를 변경) : `한 줄로 만들기`


b1 - b2 ⇒ b1` - b2`
m0 - m1 - m2 - rb1

- 3-way ⇒ ⇒ ⇒ fast-forward 化
 - base br의 최신 CommitID로 변경
 - br2는 최종적으로 base로 merge 해야 함!
- (주의) br을 push했다면 revert사용(:ID 변경)

(주의) push 전에 rebase하자!

```
develop> git checkout -b br # 작업 branch (m0)
```

```
br> edit & commit # 작업(커밋b1)
```

```
br> edit & commit # 작업(커밋b2)
```

```
br> git switch develop # 기준 branch
```

```
develop> edit & commit # 작업(커밋m1)
```

```
develop> edit & commit # 작업(커밋m2)
```

```
develop> git checkout br # 작업 br로 이동
```

```
br> git rebase develop # rebase
```

⇒ m2까지는 그대로, br1과 br2는 새로운 ID로 변경!!

```
br> git add -A # mark merge
```

```
br> git rebase --continue # rebase 마무리
```

```
br> git checkout develop
```

```
develop> git merge br # b1, b2 병합
```

```
develop> git branch -d br
```

9. 깃 머지(merge)와 리베이스(rebase)

Rebase conflict!

(rebase시 conflict 발생!)

- 동일 파일(d.txt) 수정했다면!
conflict 나면서 br의 commitID 사라짐!

git rebase --abort # rebase 취소
--skip # master 소스 우선 (내 것 무시)
--continue # 둘 다 수용

```
interactive rebase in progress; onto 0a46129
Last command done (1 command done):
  pick e91868c d by br11
No commands remaining.
You are currently rebasing branch 'br11' on '0a46129'.
(fix conflicts and then run "git rebase --continue")
(use "git rebase --skip" to skip this patch)
(use "git rebase --abort" to check out the original branch)

Unmerged paths:
  (use "git restore --staged <file>..." to unstage)
  (use "git add <file>..." to mark resolution)
    both modified:   d.txt
```

```
master> git checkout -b br # 작업 branch (m0)
```

```
br> edit d.txt # d.txt 수정
```

```
br> git commit -am "br" # e91868c e91868c (HEAD -> br11)
```

```
br> git switch master # 8e6e065 ddd  
6110aef aa
```

```
master> edit d.txt # d.txt 수정
```

```
master> git commit -am "master" # 0a46129
```

```
br> git rebase master # rebase
```

```
⇒ d.txt conflict!!
```

```
Auto-merging d.txt
CONFLICT (content): Merge conflict in d.txt
error: could not apply e91868c... d by br11
hint: Resolve all conflicts manually, mark them as resolved with
hint: "git add/rm <conflicted_files>", then run "git rebase --continue".
hint: You can instead skip this commit: run "git rebase --skip".
hint: To abort and get back to the state before "git rebase", run "git rebase --abort".
Could not apply e91868c... d by br11
```

```
br> git log --oneline # br11의 e91868c 사라짐!
```

```
0a46129 (HEAD, master) d by master
8e6e065 ddd
6110aef aa
```

```
brX> edit merge & git add # merge editing & add
```

```
brX> git rebase --continue # 마무리
```

```
br>
```

```
7ba9e3b (HEAD -> br11) d by br11
0a46129 (master) d by master
8e6e065 ddd
6110aef aa
```

9. 깃 머지(merge)와 리베이스(rebase)

cf. `$> git diff HEAD~3..HEAD d.txt`

cf. rebase의 또 다른 기능 (commit 합치기)

- `git rebase -i HEAD~n`
(n: 작업할 포인트 개수)

m0 - m1 - m2 - m3 - m4

`$> git rebase -i HEAD~3`

```
pick      m2
squash    m3
squash    m4
=> m0 - m1 - m2 - m3 - m4 - m5
           (11)  (22)  (33)  (11)
```

`pick 8e6e065 ddd`
`squash 0a46129 d by master`
`squash 7ba9e3b d by br11`

m4 7ba9e3b (HEAD
m3 0a46129 d by
m2 8e6e065 ddd
6110aef aa
51bd2ac aa

da19e1d (HEAD -> master) ddd
6110aef aa
51bd2ac aa

1. 마지막 n개의 커밋을 하나로 합치기

`git log -5 --oneline`

2. `git rebase -i HEAD~3` # interactive모드
HEAD~3 이전까지

`squash(s)` **합칠** 커밋ID # HEAD포함 3개

`pick` **남길(기준)** 커밋ID # 과거 ID 남기기

```
# This is a combination of 3 commits.
# This is the 1st commit message:

ddd

# This is the commit message #2:

d by master

# This is the commit message #3:

d by br11
```

기준(pick)이 없다면 다음 에러! ⇒ `$> git rebase --edit`

error: cannot 'squash' without a previous commit

9. 깃 머지(merge)와 리베이스(rebase)

cf. rebase the remote branch

```
br> git fetch -p # prune
br> git rebase origin/develop
no-br> merge editing for conflict
no-br> git add -A
no-br> git rebase --continue
br> git status
```

cf. rebase로 합치기 후,
최종 merge 시 conflict

연습문제

Try This -rebase

develop 브랜치에서 br 브랜치를 생성하고,
develop에서 2회의 commit,
br에서 2회 commit 을 각각 수행한 후,
br에서 develop을 rebase 하고,
br에서 commit을 한번 더 수행한 후,
br에서 commit을 HEAD~3까지 합치고,
develop에서 br을 merge하시오.
(같은 파일을 수정하여 conflict 내시오)

1. `git checkout -b br`
2. br에서 2번의 `commit` 수행
3. develop에서 2번의 `commit` 수행
4. br에서 develop을 `rebase`
5. br에서 추가 `commit` 수행
6. br에서 `commit` 합치기 (HEAD~3까지)
7. develop에서 br merge
8. br 제거




```

I  Q  I  I  I
Q  I  I  Q  I  Q  I  I  Q
    I  Q  I  I  Q  I  Q  I
Q  Q  I  Q  I  Q  Q
Q  I  I  I  Q      I  I
      Q  I  Q      Q  I  Q

```

10. 깃 리셋(reset), 리버트(revert), 태그(tag)

```

                I  I
                Q  I
            I  Q  I  Q  I
            I  Q  Q  Q
        I  I  Q  Q  I
        I  I  Q  I  Q  I  I
    I  I  Q  I  I  Q  Q  I
Q  I  Q  I      Q  Q  I

```

10. 깃 리셋(reset), 리버트(revert), 태그(tag)

reset (복귀, 되돌리기)

- soft** : repo → stage
- mixed** : repo → WD(M)
- hard** : repo → 초기화! (수정전)
- merge** : merge 취소

(주의) Commit ID를 변경하므로, **push된 ID는 reset하면 안됨!!**
(push된 커밋은 reset후 pull 요구)

10. 깃 리셋(reset), 리버트(revert), 태그(tag)

reset options

Repo — **reset --soft** → Stage

Repo — **reset --mixed** → WD(Modified)

Repo — **reset --hard** → WD(Unmodified)
: 수정내역 삭제!!

cf.

alias gl5='git log --oneline -5'

\$> git reset [options] <target-HEAD>

1. **vi ttt.txt** # 수정
2. **git commit -am "c1"** # commit (C1)
3. **edit & commit -am "c2"** # commit (C2)
edit & commit -am "c3" # commit (C3)
4. **git reset --soft HEAD~1** # Repo → Stage
5. **git log -5 --oneline**
git status # C3 제거되고 staged
6. **git reset --mixed HEAD~1** # Repo → WD(M)
7. **git reset --hard HEAD~1** # Repo → WD(UM)
⇒ 수정내역 모두 사라짐!!

cf. reset을 reset(취소)하고 싶을 때 뒷 면 참조!!

10. 깃 리셋(reset), 리버트(revert), 태그(tag)

cf. reset을 reset 하다!

```
git reset HEAD@{X}
```

```
5018baf HEAD@{12}: reset: moving to HEAD
5018baf HEAD@{13}: rebase (finish): returning to refs/heads/xx2
5018baf HEAD@{14}: rebase (pick): xx2-2
4498dd0 HEAD@{15}: rebase (continue): xx1
316f720 (develop) HEAD@{16}: rebase (start): checkout develop
646fd94 HEAD@{17}: checkout: moving from develop to xx2
316f720 (develop) HEAD@{18}: commit: dd
88963aa (origin/develop) HEAD@{19}: checkout: moving from xx2 to develop
646fd94 HEAD@{20}: commit: xx2-2
8a0b114 HEAD@{21}: commit: xx1
88963aa (origin/develop) HEAD@{22}: checkout: moving from develop to xx2
88963aa (origin/develop) HEAD@{23}: checkout: moving from master to develop
```

만약, 위에서 rebase 전으로 가고 싶다면,
git reset HEAD@{16}

```
~/workspace/tmp/xx(xx2) $ git log --oneline -5
316f720 (HEAD -> xx2, develop) dd
88963aa (origin/develop) basic grammar
d5968dc aa
```

1. `git reflog` # 전체 action log
2. `git reset HEAD@{1}` # 방금 reset 취소!
(소스는 모두 남음)
3. `git reset --hard HEAD@{16}`
⇒ 그 시점으로 소스와 커밋을 되돌림
(rebase 이전으로 가면 rebase 이전 상태로 돌아감)

10. 깃 리셋(reset), 리버트(revert), 태그(tag)

reset의 또 다른 기능

- stage(add) 초기화
⇒ git reset --mixed
- merge(병합) 취소
⇒ git reset --merge

```
$> git config --global alias.lg "log --color --graph --  
pretty=format:%Cred%H%Creset %C(yellow)%d%Creset %s  
%Cgreen(%cr) %C(bold blue)<%an>%Creset' --abbrev-commit"
```

1. Stage 초기화

```
edit ttt.txt & aaa.txt  
git add -A                # make staged  
git reset [file]         # --mixed HEAD 생략  
⇒ stage → → → WD(modified)  
git reset --hard          # 수정 내역까지 삭제!
```

2. merge 취소

```
master> git checkout -b hotfix    # 생성  
hotfix> edit & commit             # 작업  
hotfix> git checkout master  
master> git merge hotfix          # 병합  
master> git reset --merge HEAD~1 # 병합 취소!  
master> git lg                  # merge log
```

10. 깃 리셋(reset), 리버트(revert), 태그(tag)

revert

기존 Commit 남겨두고 되돌리기.
즉, push된 Commit 되돌리기!

cf. 마지막 커밋 msg 수정 (CID 변경)

`git commit --amend -m 'xx'`

```
You are currently reverting commit b9aba2f.
(all conflicts fixed: run "git revert --continue")
(use "git revert --skip" to skip this patch)
(use "git revert --abort" to cancel the revert operation)
```

```
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   ttt.txt
```

1. `vi ttt.txt`
2. `git commit -am "before revert"` # (가)
3. `git log -3 --oneline` # 지난 3 commits
4. `git revert HEAD` # 방금 전 commit 취소(가 제거)
5. `(edit & commit) x 2 times` # (나)
6. `git revert HEAD~2` # conflict!
두 커밋 전(가)와 같은 라인 (**both-modified**)
7. `vi ttt.txt` # 수동 merge
8. `git add ttt.txt` # make staged(merged)
modified (in stage)
9. `git revert --continue` # 수동 merge
cf. `git revert CommitID1..CommitID2`

```
~/workspace/gitplay/gitpp2 (master) $ git revert --continue
[master fb1e8ab] Revert "Revert "5555555""
1 file changed, 4 insertions(+)
~/workspace/gitplay/gitpp2 (master) $ git log --oneline -3
fb1e8ab (HEAD -> master) Revert "Revert "5555555""
dc6c4d0 (origin/master) r22
```

10. 깃 리셋(reset), 리버트(revert), 태그(tag)

cf. VSCode GitLens after revert

```
dsfdsakfjkals;dsfdsakfjkalssfdas
<<<<<<< HEAD
a
sdf
dsfdsakfjkalssfdasdfasd
dsfdsakfjkalsdsa
f
dsfdsakfjkalssfdasdfasdf
as
dsfdsakfjkals
dsfdsakfjkalsdsa
f
ads
d
sasdfa
=====
DATE=`date +%Y-%m-%d` "%H:%M`
MSG="$DATE lesson"
if [ $# -gt 0 ]; then
    MSG="$DATE - $1"
fi
git add --all
git commit -am "${MSG}"

git push
>>>>>> parent of cacf34a... LAST
```



GitLens — Git supercharged v14.7.0

GitKraken gitkraken.com | 28,855,755 | ★★★★★ (761)

Supercharge Git within VS Code — Visualize code authorship at a glance ...

[Disable](#) [Uninstall](#) [Switch to Pre-Release Version](#) [Settings](#)

This extension is enabled globally.

현재 변경 사항 수락 | 수신 변경 사항 수락 | 두 변경 사항 모두 수락 | 변경 사항 비교

<<<<<< HEAD (현재 변경 사항)

```
a
sdf
dsfdsakfjkalssfdasdfasd
dsfdsakfjkalsdsa
f
dsfdsakfjkalssfdasdfasdf
as
dsfdsakfjkals
dsfdsakfjkalsdsa
f
ads
d
sasdfa
```

```
=====
DATE=`date +%Y-%m-%d` "%H:%M`
MSG="$DATE lesson"
```

```
if [ $# -gt 0 ]; then
    MSG="$DATE - $1"
fi
```

```
git add --all
git commit -am "${MSG}"
```

```
git push
```

>>>>>> parent of cacf34a... LAST (수신 변경 사항)

10. 깃 리셋(reset), 리버트(revert), 태그(tag)

cf. 수정 및 복구

1. 마지막 커밋 msg 수정 (CID 변경)

```
$> git commit --amend -m 'xx'
```

2. 완전 삭제된 브랜치 복구

```
$> git reflog # 여기서 삭제된 브랜치의 최종 CommitID확인
```

```
$> git checkout -b 'new-br-name' HEAD@{n} # 최종 CommitID로 새 브랜치 생성
```


10. 깃 리셋(reset), 리버트(revert), 태그(tag)

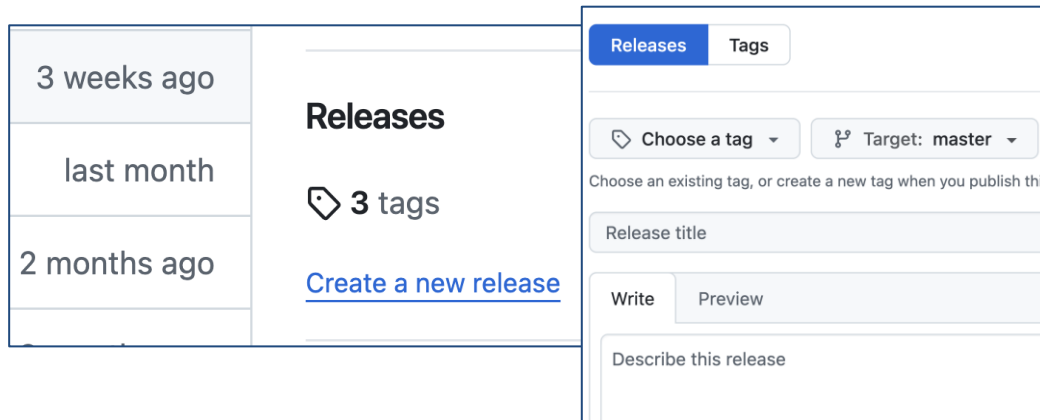
tag (배포 꼬리표, 특정 커밋 태깅)

- * Annotated: 태그 + 정보
- * Lightweight: 태그 이름만!

- git tag -a <tag>
- git tag -l/d

cf. SemVer(Semantic Versioning)

release
in github

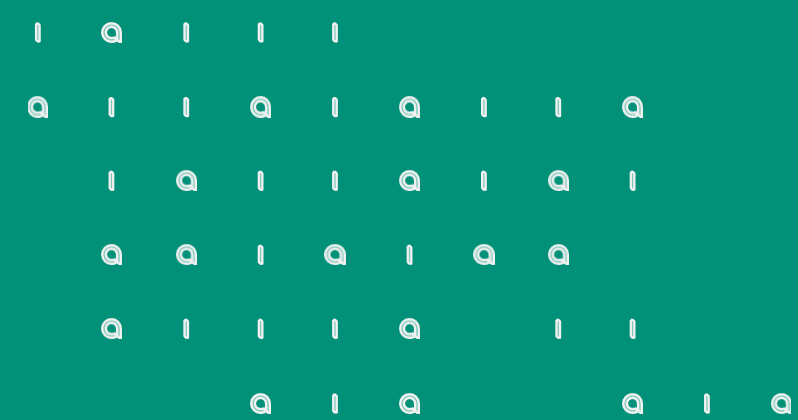


1. `git tag [-l]` # tag list
2. `git tag -a <tag> [ID]` # make tag annotated-tag
release note 작성 (edit tag info)
`git tag -a 1.0.0 -m "msg"`
`git tag -a 1.0.0` # editor
3. `git log` # tag 정보 보임
`git tag -l` # tag list
4. `git show <tag>` # 해당 tag 변경 사항
5. `git tag -d <tag>` # 특정 tag 삭제
6. `git checkout <tag>` # 특정 tag로 이동
`git checkout -b <br> <tag>` # br 생성
7. `git push origin <tag>` # push
`git push origin 1.0.0`
cf. `git push --delete origin 1.0.0`

master

1 Branch

1 Tags



11. 깃 플로우(git-flow)



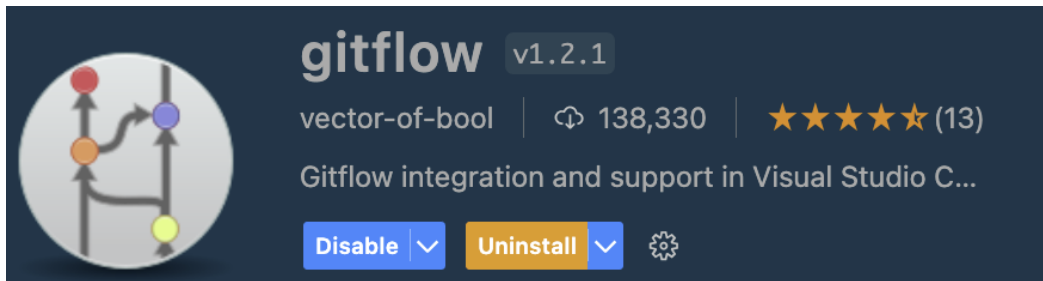
11. 깃플로우(Git-Flow)

Git Flow Concepts

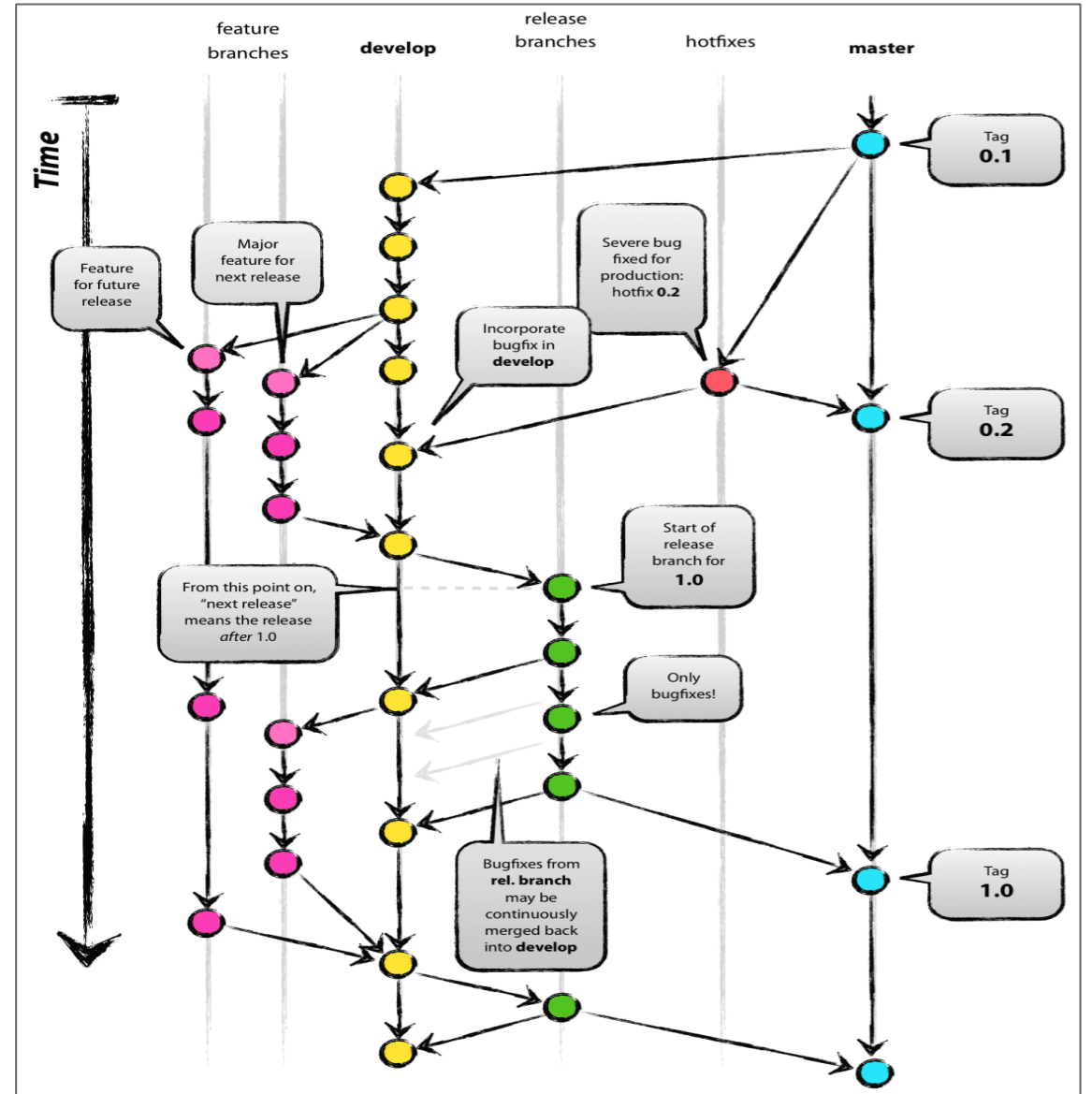
- **master (main)**
- **develop**
- **feature/** (feature 생략)
- **release/**
- **hotfix/**
- **support/** (only git flow cli)

강의영상

<https://youtu.be/VY8K9FguM-E>



<https://nvie.com/posts/a-successful-git-branching-model/>



11. 깃플로우(Git-Flow)

Install Git Flow CLI

Git Flow Settings

1. Install **gitflow** VSCode extension
2. <https://github.com/nvie/gitflow>



Windows

[guide](#)

1. Install MSysGit
<https://gitforwindows.org/>
2. download dll files
libiconv-1.9.2-1-bin/bin/libiconv2.dll
libintl-0.14.4-bin/bin/libintl3.dll
util-linux-ng-2.14.1-bin/bin/getopt.exe
3. copy to msysgit's bin
(ex. C:\Program Files (x86)\Git\bin)

```
$> git clone --recursive git://github.com/nvie/gitflow.git
$> cd gitflow
$> git config --global url."https://github".insteadOf git://github
```

```
C:\gitflow> contrib\msysgit-install.cmd "[path to git installed folder]"
$> ln -s "/C/Program Files (x86)/Git/bin/git-flow" git-flow
```

Mac

[guide](#)

```
$> brew install git-flow
```

Linux

[guide](#)

```
$> sudo -
$> dnf install git -y
$> curl -OL
https://raw.githubusercontent.com/nvie/gitflow/develop/contrib/gitflow-installer.sh
$> chmod +x gitflow-installer.sh
$> ./gitflow-installer.sh
```

11. 깃플로우(Git-Flow)

Git Flow Commands

```
$> git flow init # gitflow: ...
$> git push origin --all
$> git flow feature start <feature-branch>
$> git flow feature list
$> git add .
$> git commit -m
# push & pull-request(to review)
$> git flow feature finish <feature-branch>
$ other-feature> git merge --no-ff develop
$> git flow release start <tag>
$> git flow release finish <tag>
$> git tag -l
$> git push origin --all --follow-tags
$> git flow hotfix start <tag>
$> git flow hotfix finish <tag>
```

```
>git
```

```
Gitflow: Initialize repository for gitflow
```

example scripts

```
$> git flow init
$> git push origin --all
$> git flow feature start login
$> git merge --no-ff develop
```

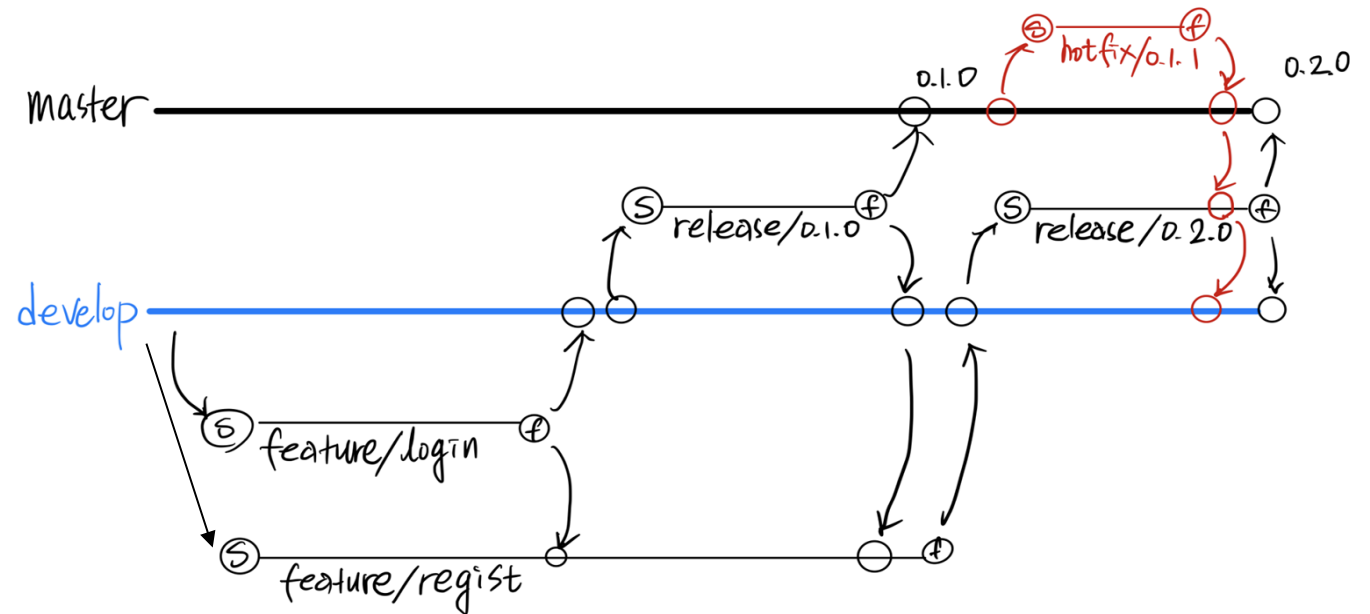
```
~/workspace/worki/admx (main) $ git flow init

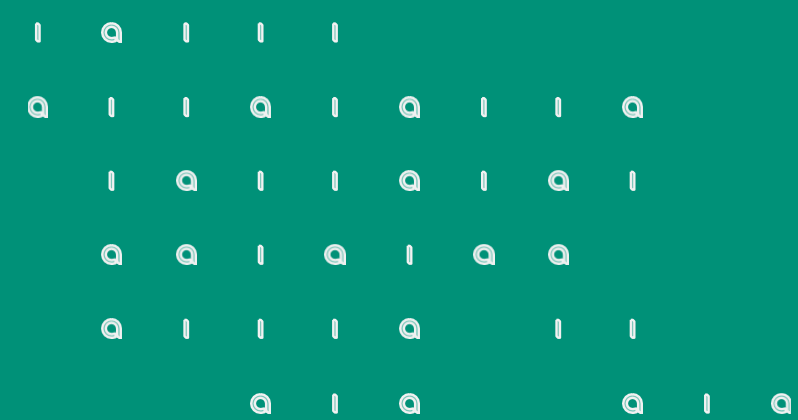
Which branch should be used for bringing forth production releases?
- main
Branch name for production releases: [main]
Branch name for "next release" development: [develop]

How to name your supporting branch prefixes?
Feature branches? [feature/]
Release branches? [release/]
Hotfix branches? [hotfix/]
Support branches? [support/]
Version tag prefix? []
```

연습문제

1. 2개 feature 시작 (login, regist)
2. login, regist 각각 코딩
3. login 작업 완료
4. login 코드 리뷰
5. login 릴리즈 & 배포 (0.1.0)
6. regist 작업완료
7. regist 릴리즈 (0.2.0)
8. **0.1.0에서 버그 발생!!** → hotfix 시작
9. hotfix 0.1.1 작업완료
10. hotfix 0.1.1 코드리뷰
11. hotfix 0.1.1 배포(0.1.1)
12. regist 릴리즈 마무리 & 배포(0.2.0)

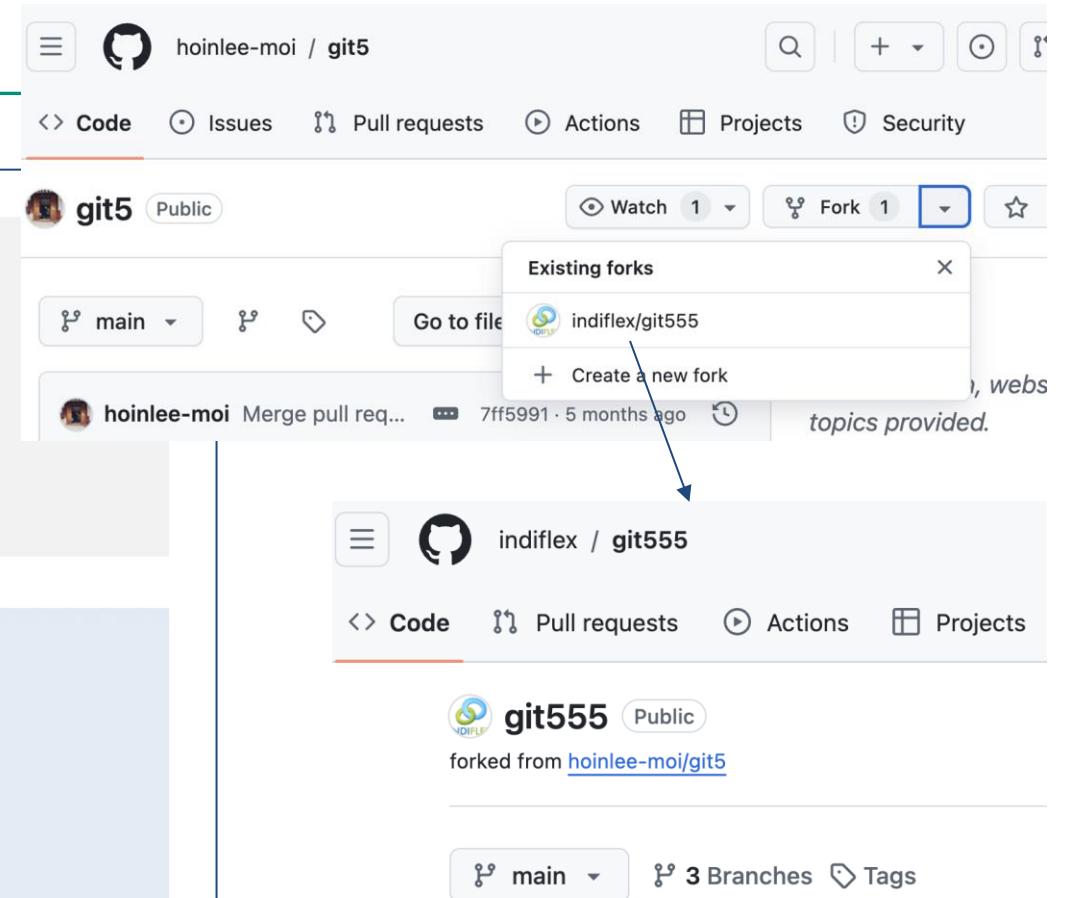
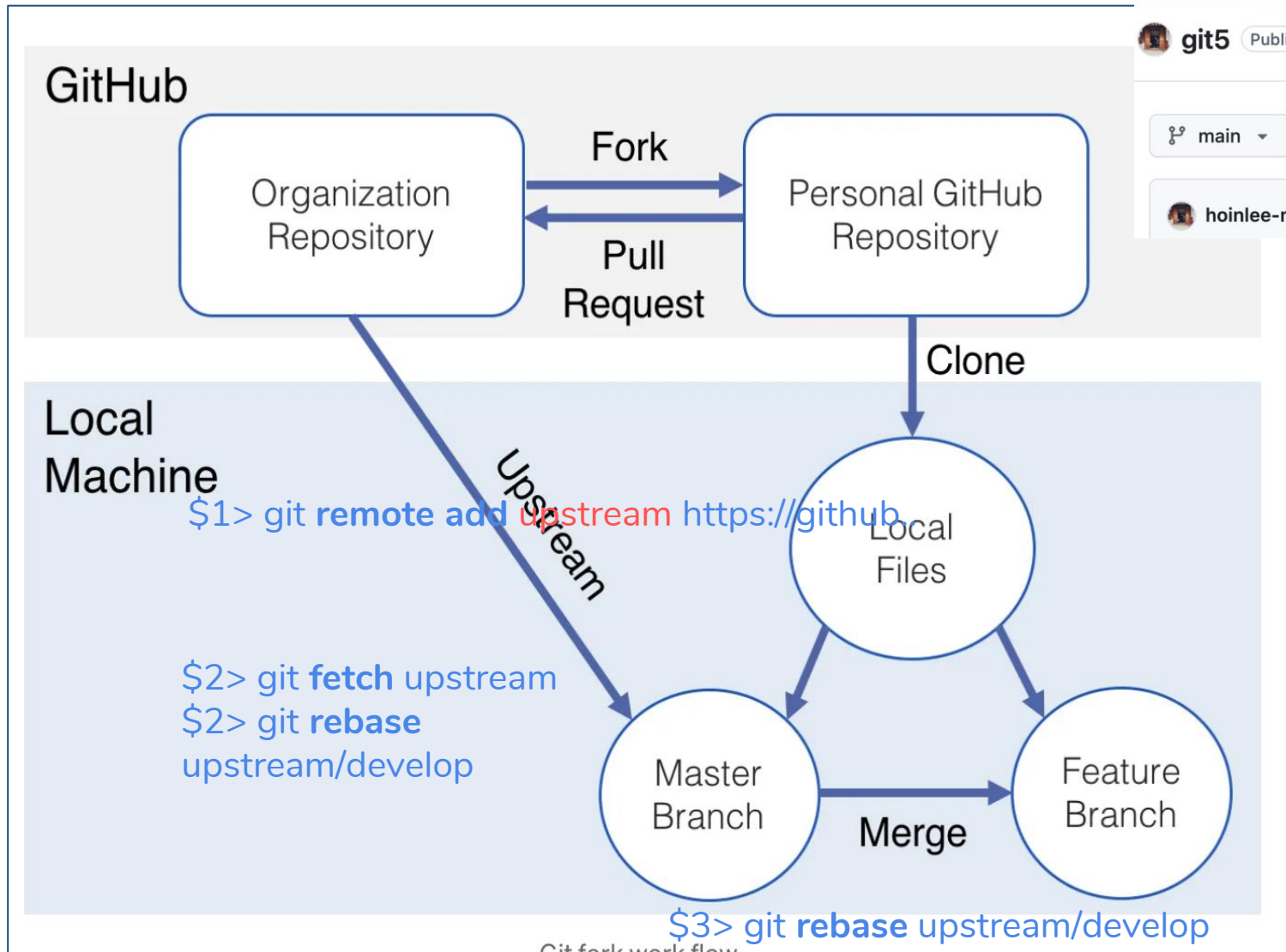




12. Fork & Pull Request



12. Fork & Pull Request



12. Fork & Pull Request

```
~/workspace/tmp/pr/git555(main) $ git remote add upstream https://github.com/hoinlee-moi/git5
~/workspace/tmp/pr/git555(main) $ git remote -vv
origin https://github.com/indiflex/git555.git (fetch)
origin https://github.com/indiflex/git555.git (push)
upstream https://github.com/hoinlee-moi/git5 (fetch)
upstream https://github.com/hoinlee-moi/git5 (push)
```

git remote add upstream https://github.com/indiflex/xxx

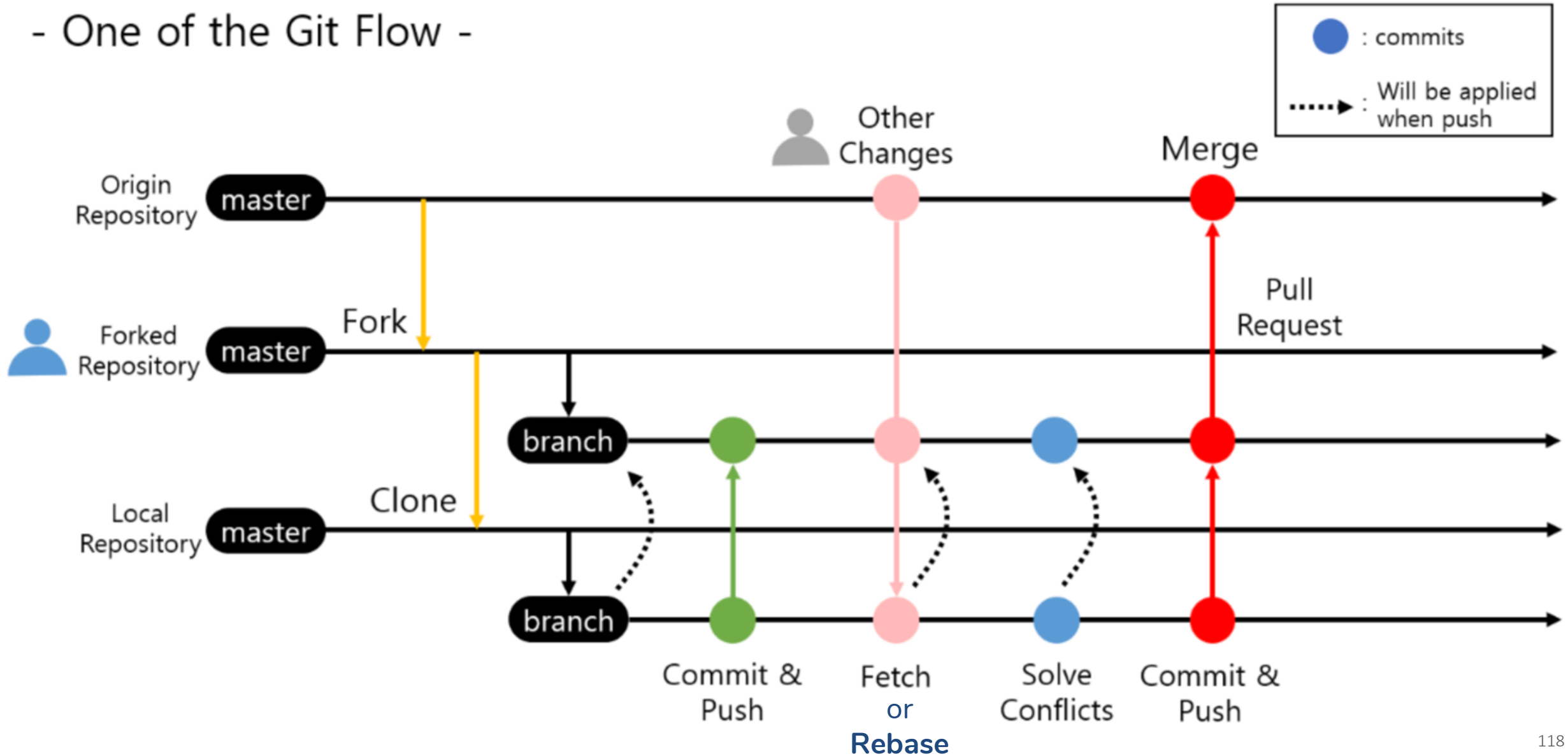
```
~/workspace/tmp/pr/git555(main) $ git fetch upstream
remote: Enumerating objects: 33, done.
remote: Counting objects: 100% (33/33), done.
remote: Compressing objects: 100% (23/23), done.
remote: Total 31 (delta 3), reused 29 (delta 1), pack-reused 0 (from 0)
Unpacking objects: 100% (31/31), 2.91 KiB | 102.00 KiB/s, done.
From https://github.com/hoinlee-moi/git5
* [new branch]      develop -> upstream/develop
* [new branch]      main    -> upstream/main
* [new branch]      test    -> upstream/test
* [new branch]      tt      -> upstream/tt
```

git fetch upstream

git rebase upstream/develop

12. Fork & Pull Request

- One of the Git Flow -



12. Fork & Pull Request

 **develop** had recent pushes 21 minutes ago

Compare & pull request

 Watch 1 ▼

 Fork 1 ▼

승희님 테스트 PR이에요 #1

 Open indiflex wants to merge 1 commit into [doSeung11:main](#) from [indiflex:fetch-ttt](#) 

Existing forks

 indiflex/test11-do

+ Create a new fork

10  2 Commits

 Conversation 0

 Commits 1

 Checks 0

 Files changed 1



indiflex commented 2 minutes ago

No description provided.



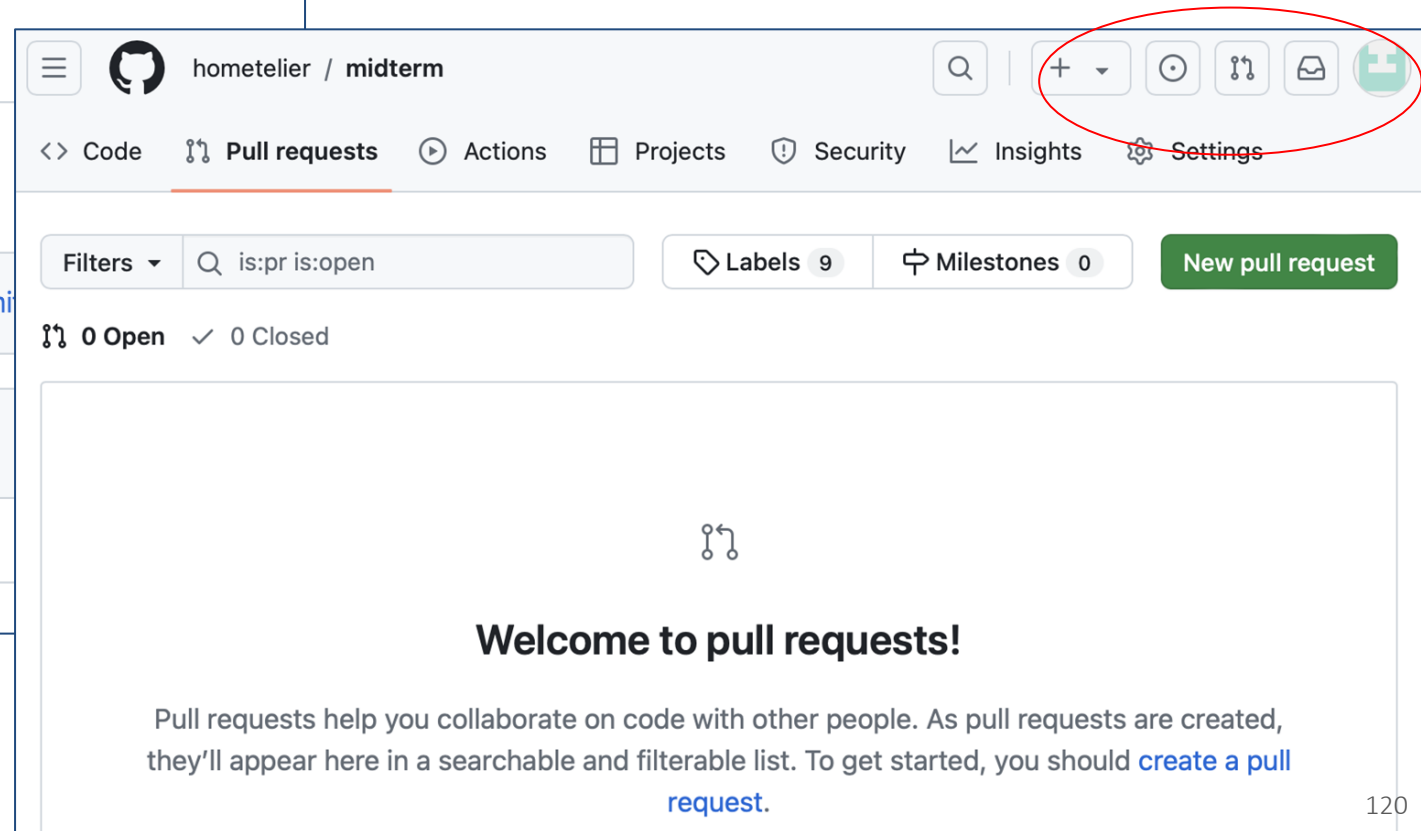
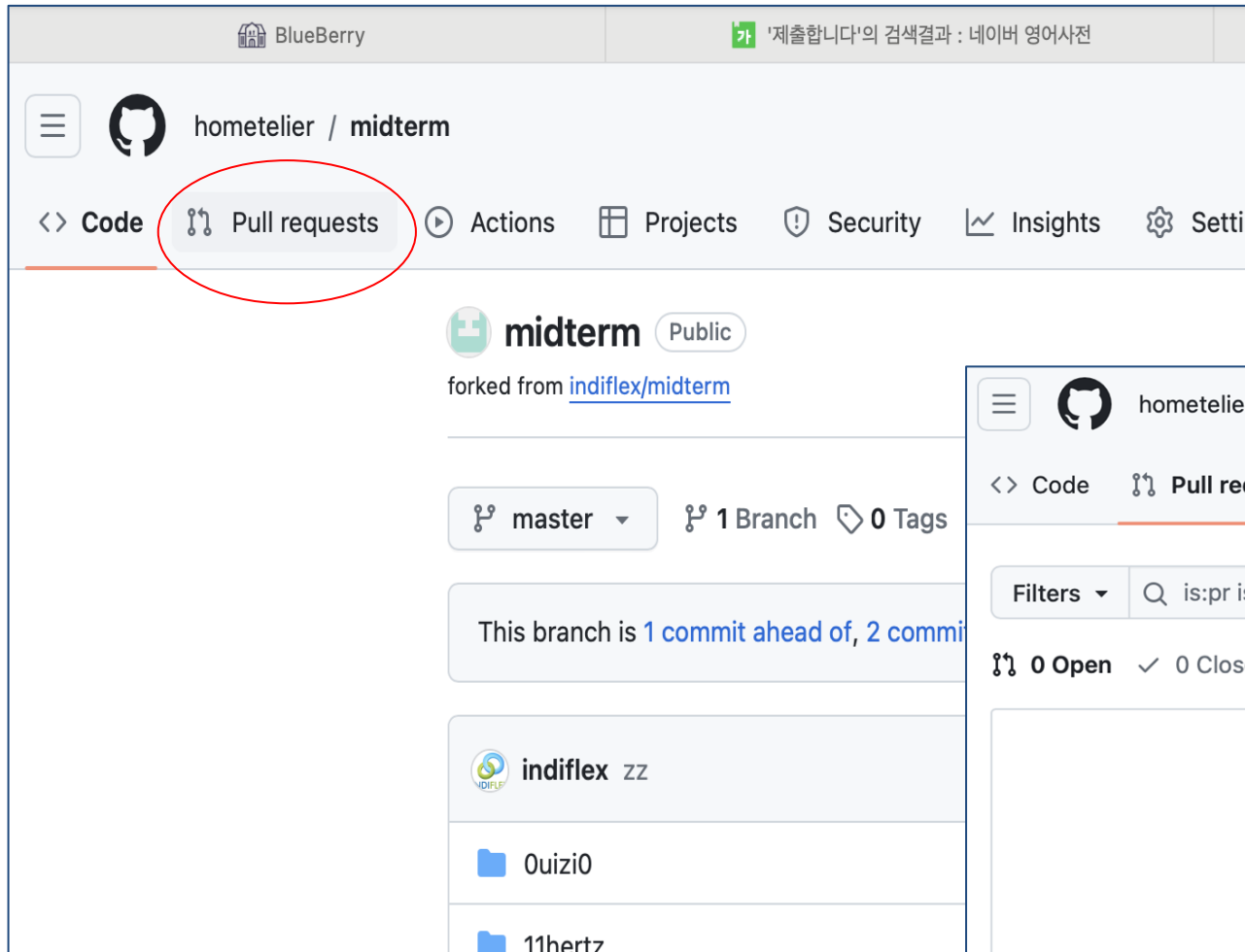
  ttt

ec8f3bf

 Markdown is supported

 Paste, drop, or click to add files

Create pull request ▼



12. Fork & Pull Request

Comparing changes

Choose two branches to see what's changed or to start a new pull request. If you need to, you can also [compare across forks](#) or [learn more about diff comparisons](#).

base repository: `indiflex/midterm`

base: `master`

head repository: `hometelier/midterm`

compare: `master`

✓ **Able to merge.** These branches can be automatically merged.

Discuss and review the changes in this comparison with others. [Learn about pull requests](#)

1 commit

1 file changed

Commits on Jul 6, 2024

zz

indiflex committed 15 minutes ago

hometelier

[indiflex/midterm] 전성호 제출합니다 (PR #2)

받는 사람: indiflex/midterm, 참조: Subscribed, 답장 받는 사람: indiflex/midterm

이 메시지는 메일링 리스트에서 옵니다.

3번 문제는 못 풀었어요

You can view, comment on, or merge this pull request online at:
<https://github.com/indiflex/midterm/pull/2>

Commit Summary

- cf8ce7c zz
- 898a14d submit

File Changes (9 files)

- M README.md (1)
- M hoinlee.test.js (4)
- M hoinlee/ProxyEmp.js (18)
- M hoinlee/cal.js (20)
- M hoinlee/collection.js (116)
- M hoinlee/index.js (11)
- M hoinlee/keyPair.js (18)
- M hoinlee/reduce.js (8)
- M hoinlee/regexp.js (21)

Patch Links:

- <https://github.com/indiflex/midterm/pull/2.patch>
- <https://github.com/indiflex/midterm/pull/2.diff>

Reply to this email directly, [view it on GitHub](#), or [unsubscribe](#).
You are receiving this because you are subscribed to this thread.

indiflex / midterm

<> Code Issues Pull requests 1 Actions Projects

Open a pull request

Create a new pull request by comparing changes across two branches. If you need to, you can also [compare across forks](#) or [learn more about diff comparisons](#).

base repository: `indiflex/midterm`

base: `master`

head repository: `hometelier/midterm`

compare: `master`

✓ **Able to merge.** These branches can be automatically merged.

Add a title

전성호 제출합니다

Add a description

3번 문제는 못 풀었어요

Write Preview

H B I

Markdown is supported Paste, drop, or click to add files

Remember, contributions to this repository should follow our [GitHub Community Guidelines](#).

indiflex / midterm

<> Code Issues Pull requests 2 Actions Projects Security Insights

전성호 제출합니다 #2

hometelier wants to merge 2 commits into `indiflex:master` from `hometelier:master`

Conversation 0 Commits 2 Checks 0 Files changed 9

hometelier commented now

3번 문제는 못 풀었어요

indiflex added 2 commits 22 minutes ago

cf8ce7c zz

898a14d submit

Add more commits by pushing to the `master` branch on `hometelier/midterm`.

This branch has no conflicts with the base branch

Only those with [write access](#) to this repository can merge pull requests.

Add a comment

Write Preview

H B I

✓ **Allow edits by maintainers** Create pull request

121

12. Fork & Pull Request

PR Receiver (받는 사람, 호스트)

indiflex / midterm

<> Code Issues **Pull requests 2** Actions

midterm Public

master 1 Branch 0

indiflex Update README.md

0uizi0

11hertz

Filter

2 Open 0 Closed

☐ **전성호 제출합니다 #2**
#2 opened 4 minutes ago by hometelier

전성호 제출합니다 #2

Open hometelier wants to merge 2 commits into `indiflex:master` from `hometelier:master`

Conversation 0 Commits 2 Checks 0 **Files changed 9**

hometelier commented 5 minutes ago

3번 문제는 못 풀었어요

indiflex added 2 commits 27 minutes ago

zz

submit

cf8ce7c

898a14d

Copy link

Quote reply

Edit

Report content

First-time contributor

Add more commits by pushing to the `master` branch on `hometelier/midterm`.

The image displays two screenshots of a GitHub Pull Request (PR) interface for a repository named 'indiflex / midterm'.

Left Screenshot (PR Overview):

- The PR title is "전성호 제출합니다 #2".
- The status is "Open", indicating that "hometelier wants to merge 2 commits into `indiflex:master` from `hometelier:master`".
- Navigation tabs include Conversation (0), Commits (2), Checks (0), and **Files changed (9)**, which is circled in red.
- Comments from "hometelier" and "indiflex" are visible.

Right Screenshot (Diff View):

- The PR title is "전성호 제출합니다 #2".
- The status is "Open", indicating that "hometelier wants to merge 2 commits into `indiflex:master` from `hometelier:master`".
- Navigation tabs include Conversation (0), Commits (2), Checks (0), and **Files changed (9)**.
- The diff view shows changes to `README.md` and `hoinlee.test.js`.
- In the `README.md` diff, a change on line 16 is highlighted with a red box, showing a new line added by "zz":
@@ -13,3 +13,4 @@
13 3. 소스코드의 완성도 및 성능 10점
14
15 * 부정행위 적발 시 0점 처리 (컨닝, CoPilot, GPT 등)
16 + * 부정행위 적발 시 0점 처리 (컨닝, CoPilot, GPT 등)
- The `hoinlee.test.js` diff shows changes to the `import` statement and the `from` path in the `reduce.js` import.

12. Fork & Pull Request

Code Review Keywords

IMO (in my opinion) < **IMHO** (in my humble opinion)

LGTM (Looks Good To Me)

SGTM (Sounds Good to Me)

PTAL (Please Take Another Look)

리뷰어가 리뷰를 해주다가 중간에 stop된 상태에서 리뷰좀 해달라고 부탁할때 쓰는것으로 보인다.

12. Fork & Pull Request

14

15 * 부정행위 적발 시 0점 처리 (컨닝, CoPilot, GPT 등)

16 + zz

Write Preview H B I

이건 의미가 뭐야?

Add a suggestion, <Ctrl+g>

drag & drop

17 +

18 +

12 + return target[prop];

Comment on lines +8 to +12

indiflex Pending Owner ...

fullName에서 firstName이나 lastName이 없으면 어떡해요?

Reply...

13 },

14

15 set(target, prop, value) {

16 + if (prop === 'fullName') {

17 + const [firstName, lastName] = value.split(' ');

Write Preview H B I

여기 정규식은 안될까요?

Markdown is supported Paste, drop, or click to add files

Cancel Add review comment

Cancel (취소)
Add Single Comment (하나의 코멘트로 리뷰 끝)
Start a Review

12. Fork & Pull Request

```
138 +   }
139 +   return node2;
```

Commenting on lines +130 to +139

Write Preview  suggestion    

이건 어때요?
```suggestion  
if (!node) break;  
}  
return err;

 Markdown is supported

Commenting on lines +130 to +139

Write Preview   

이건 어때요?

Add a suggestion, <Ctrl+g>

```
13 + const ret = {};
```

Write Preview           

Leave a comment

 Markdown is supported  Paste, drop, or click to add

-  Attach files
-  Mention @특정인 멘트
-  Reference ← 다른 코드/커밋 참조
-  Saved replies 리뷰 저장
- ☒ Slash commands

Cancel Add review comment

## 12. Fork & Pull Request

The image shows a GitHub Pull Request review interface. On the left, a suggested change is highlighted in a pink box, showing a code diff for a function. The change is from user 'indiflex' and is in a 'Pending' state. The text '이건 어때요?' (How about this?) is written above the diff. The diff shows lines 130 to 135, with changes to a function that returns an array.

On the right, the 'Finish your review' modal is open. It has a title bar with a close button. Below the title bar, there are tabs for 'Write' and 'Preview'. The 'Write' tab is active, showing a text area for leaving a comment. Below the text area, there are icons for Markdown support and file uploads. At the bottom of the modal, there are three radio button options for the review type: 'Comment', 'Approve', and 'Request changes'. Each option has a description and a red Korean annotation. At the bottom right of the modal, there are three buttons: '6 pending comments', 'Abandon review', and 'Submit review'. The 'Submit review' button is circled in red.

**Suggested change**

```
130 - if (!node) break;
131 - }
132 - return arr;
133 - }
134 - static arrayToList(arr) {
135 - let node2 = { value:
```

**Finish your review**

Write Preview H B I ≡ <> 🔗 | ½≡ ≡≡ ≡≡ | 📎 @ ↻ ↶

Leave a comment

📄 Markdown is supported | 📎 Paste, drop, or click to add files

- ☒ **Comment** 단순한 코멘트 only  
Submit general feedback without explicit approval.
- ☐ **Approve** 승인!  
Submit feedback and approve merging these changes.
- ☐ **Request changes** 수정 요함!  
Submit feedback that must be addressed before merging.

6 pending comments Abandon review **Submit review**

## 12. Fork & Pull Request

Tip. approve 된 리뷰만 merge할 수 있도록 설정!

리뷰어는 리뷰만,  
merge는 개발자가 직접.

The screenshot shows the GitHub 'Branch protection rule' configuration page. The left sidebar contains navigation links: General, Access, Collaborators, Moderation options, Code and automation, Branches (selected), Tags, Actions, Webhooks, Environments, Pages, Security, Code security and analysis, Deploy keys, Secrets, Integrations, GitHub apps, and Email notifications. The main content area is titled 'Branch protection rule' and shows settings for the 'develop' branch. The 'Branch name pattern' is set to 'develop'. Under 'Protect matching branches', the following options are configured: 'Require a pull request before merging' (checked), 'Require approvals' (checked) with a dropdown set to '1', 'Dismiss stale pull request approvals when new commits are pushed' (unchecked), 'Require review from Code Owners' (unchecked), 'Require status checks to pass before merging' (unchecked), and 'Require conversation resolution before merging' (unchecked).

General

Access

Collaborators

Moderation options

Code and automation

Branches

Tags

Actions

Webhooks

Environments

Pages

Security

Code security and analysis

Deploy keys

Secrets

Integrations

GitHub apps

Email notifications

### Branch protection rule

Branch name pattern \*

develop

Applies to 0 branches

#### Protect matching branches

- ☒ **Require a pull request before merging**  
When enabled, all commits must be made to a non-protected branch and submitted via a pull request before they can be merged into a branch that matches this rule.
- ☒ **Require approvals**  
When enabled, pull requests targeting a matching branch require a number of approvals and no changes requested before they can be merged.  
Required number of approvals before merging: 1
- ☐ **Dismiss stale pull request approvals when new commits are pushed**  
New reviewable commits pushed to a matching branch will dismiss pull request review approvals.
- ☐ **Require review from Code Owners**  
Require an approved review in pull requests including files with a designated code owner.
- ☐ **Require status checks to pass before merging**  
Choose which [status checks](#) must pass before branches can be merged into a branch that matches this rule. When enabled, commits must first be pushed to another branch, then merged or pushed directly to a branch that matches this rule after status checks have passed.
- ☐ **Require conversation resolution before merging**  
When enabled, all conversations on code must be resolved before a pull request can be merged into a branch that matches

## 12. Fork & Pull Request

## 최종 merge 하기

```
137 + node2 = { value: arr[i], rest: node2 };
138 + }
139 + return node2;
```

Commit suggestion

Add suggestion to

Reply...

Resolve conversation



전성호 제출합니다 (#2)

- \* zz
- \* submit
- \* zz 제거

Co-authored-by: indiflex <indiflex1@gmail.com>

This commit will be authored by 160228527+hometelier@users.noreply.github.com

Confirm squash and merge

Cancel

Add more commits by pushing to the **master** branch on **hometelier/midterm**.



Changes approved

1 approving review [Learn more about pull request reviews](#)

1 approval

Require approval from specific review  
[Rulesets](#) ensure specific people approve pull request

Continuous integration has not been set up  
[GitHub Actions](#) and [several other apps](#) can be configured to run on pull requests

This branch has no conflicts with the base branch  
Merging can be performed automatically.

Merge pull request

You can also [open this in GitHub Desktop](#)

Merging can be performed automatically.

Merge pull request



You can also [open this in GitHub Desktop](#)

✓ Create a merge commit **각 커밋 모두 merge**

All commits from this branch will be added to the base branch via a merge commit.

Squash and merge **하나의 커밋으로 합친 다음 merge**

The 3 commits from this branch will be combined into one commit in the base branch.

Rebase and merge **마치 master에서 작업한 것 처럼 merge**

The 3 commits from this branch will be rebased and added to the base branch.

Squash and merge

You can also [open this in GitHub Desktop](#) or view [command line instructions](#).

Add a comment

Write

Preview

80점

Markdown is supported

Paste, drop, or click to add files

Close with comment

Comment

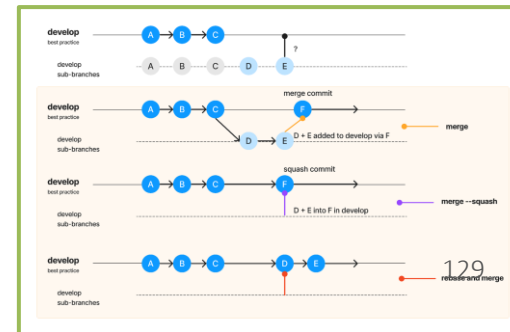


Add a comment

Write

Preview

Add your comment here...



## 12. Fork & Pull Request

hoinlee-moi / git5

<> Code

Issues

Pull requests

Actions

Projects

Security

Insights

git5

Public

Watch 1

Fork 1

main

Go to file

hoinlee-moi fork

git555 / README.md

in main

git555

Update README.md

Existing forks

indiflex/git555

Create a new fork

Commit message

Update README.md

Extended description

Add an optional extended description..

Commit Email

indiflex1@gmail.com

☐ Commit directly to the main branch

☒ Create a new branch for this commit and start a pull request

[Learn more about pull requests](#)

indiflex-patch-1

Cancel

Propose changes

ho\_in

[indiflex/secho] full request edit by hoinlee (PR #1)

받는 사람: indiflex/secho, 참조: Subscribed, 담당 받는 사람: indiflex/secho

이 메시지는 메일링 리스트에서 옵니다.

edit by hoinlee

You can view, comment on, or merge this pull request online at:

<https://github.com/indiflex/secho/pull/1>

Commit Summary

- 2f1bd8c Update c.js
- 8b25d11 Merge pull request #1 from hoinlee-moi/hoinlee-moi-patch-1

File Changes (1 file)

- M c.js (3)

Patch Links:

- <https://github.com/indiflex/secho/pull/1.patch>
- <https://github.com/indiflex/secho/pull/1.diff>

Reply to this email directly, [view it on GitHub](#), or [unsubscribe](#).  
You are receiving this because you are subscribed to this thread.

130

## 12. Fork & Pull Request

 indiflex / git555

 Code  Pull requests  Actions  Projects  Wiki  Security  Insights 

### Comparing changes

Choose two branches to see what's changed or to start a new pull request. If you need to, you can also [compare across forks](#) or [learn more about diff comparisons](#).

 base: main  compare: indiflex-patch-2

✓ **Able to merge.** These branches can be automatically merged.

Discuss and review the changes in this comparison with others. [Learn about pull requests](#) [Create pull request](#)

 1 commit  1 file changed  1 contributor

 Commits on May 23, 2024

 **Update README.md**   bb6264f 

 indiflex committed now

 Showing **1 changed file** with **1 addition** and **0 deletions**.  Split  Unified

 1  README.md 

|     |               |            |
|-----|---------------|------------|
| ... | @@ -1 +1,2 @@ |            |
| 1   | # git5        | 1 # git5   |
|     |               | 2 + second |



### Add a title

Update README.md

### Add a description

Write  Preview  H  B  I      

CL  
- aaa  
- bbb

 Markdown is supported  Paste, drop, or click to add files

[Create pull request](#)

 Remember, contributions to this repository should follow our [GitHub Community Guidelines](#).



## 12. Fork & Pull Request

 indiflex / **git555**

🔍 Type / to search

[Code](#) [Pull requests](#) 1 [Actions](#) [Projects](#) [Wiki](#) [Security](#) [Insights](#) [Settings](#)

 **git555** Public [Pin](#) [Watch](#) 0 

forked from [hoinlee-moi/git5](#)

[main](#)  [3 Branches](#) [0 Tags](#)   [+](#) [Code](#) 

This branch is up to date with [hoinlee-moi/git5:main](#).

[Contribute](#) 

[Sync fork](#) 

 **hoinlee-moi** first commit 33dadee · 2 months ago [1 Commit](#)

 README.md

first commit

2 months ago

 **README** 

# git5



## 12. Fork & Pull Request

indiflex-patch-2 3 Branches 0 Tags

Go to file

Code

This branch is 1 commit ahead of hoinlee-moi/git5:main . #1

indiflex Update README.md

bb6264f · 4 minutes ago 2 Commits

README.md

Update README.md

4 minutes ago

README

git5

second

Comparing changes

Choose two branches to see what's changed or to start a new pull request. If you need to, you can also [compare across forks](#) or [learn more about diff comparisons](#).

base repository: hoinlee-moi/git5 base: main

head repository: indiflex/git555 compare: indiflex-patch-2

✓ Able to merge. These branches can be automatically merged.

Discuss and review the changes in this comparison with others. [Learn about pull requests](#)

Create pull request

1 commit

1 file changed

1 contributor

Commits on May 23, 2024

Update README.md

indiflex committed 5 minutes ago

Verified

bb6264f

133

## 12. Fork & Pull Request

### Open a pull request

Create a new pull request by comparing changes across two branches. If you need to, you can [learn more about diff comparisons here](#).

base repository: hoinlee-moi/git5 ▾ base: main ▾ ↩  
head repository: indiflex/git555 ▾ compare: indiflex-patch-2 ▾  
✓ **Able to merge.** These branches can be automatically merged.



#### Add a title

Update README.md

Helpful  
GitHub C

#### Add a description

Write

Preview

H

B

I

≡

<>

🔗

≡

≡

...

CL  
-aaa

Markdown is supported

Paste, drop, or click to add files

☒ Allow edits by maintainers ?

Create pull request ▾

ChangeList  
cf. LGTM, SGTM

Prefix non-critical comments with “**Nit:** ”

## 12. Fork & Pull Request

### 수신(Code Owner)

The screenshot shows a GitHub Pull Request (PR) titled "Update README.md #2". The PR is open and shows a commit from the `indiflex:indiflex-patch-2` branch being merged into the `hoinlee-moi:main` branch. The PR has 1 commit, 0 checks, and 1 file changed. A comment from `indiflex` is visible, stating "CL -aaa". The PR is verified and has a commit hash of `bb6264f`. The right sidebar shows the PR's status, including "No reviews", "Still in progress? Convert to draft", "No one—assign yourself", "None yet" for labels, "None yet" for projects, "No milestone", "Development", and "None yet" for notifications. A bottom panel shows the PR's status, including "Require approval from specific reviewers before merging", "Continuous integration has not been set up", and "This branch has no conflicts with the base branch". A "Merge pull request" button is visible at the bottom.

<> Code Issues Pull requests 1 Actions Projects Wiki Security Insights Settings

### Update README.md #2

Edit <> Code

Open indiflex wants to merge 1 commit into hoinlee-moi:main from indiflex:indiflex-patch-2

Conversation 0 Commits 1 Checks 0 Files changed 1 +1 -0

indiflex commented now First-time contributor

CL  
-aaa

Update README.md Verified bb6264f

Add more commits by pushing to the `indiflex-patch-2` branch on `indiflex/git555`.

Require approval from specific reviewers before merging  
Rulesets ensure specific people approve pull requests before they're merged. Add rule

Continuous integration has not been set up  
GitHub Actions and several other apps can be used to automatically catch bugs and enforce style.

✓ This branch has no conflicts with the base branch  
Merging can be performed automatically.

Merge pull request You can also open this in GitHub Desktop or view command line instructions.

Reviewers  
No reviews  
Still in progress? Convert to draft

Assignees  
No one—assign yourself

Labels  
None yet

Projects  
None yet

Milestone  
No milestone

Development  
Successfully merging this pull request may close these issues.

None yet

Notifications Customize

Add a comment

## 12. Fork & Pull Request

# Update README.md #2

 **Merged** hoinlee-moi merged 1 commit into `hoinlee-moi:main`

 Conversation **0**  Commits **1**  Checks **0**



**indiflex** commented 2 minutes ago

CL  
-aaa



  Update README.md

**Verified**

  **hoinlee-moi** merged commit **7ff5991** into `hoinlee-moi:main` now

  indiflex / git555

[Code](#) [Pull requests](#) **1** [Actions](#) 

 **git555** Public  Pin

forked from [hoinlee-moi/git5](#)

 main   [Go to file](#)

This branch is **2 commits behind** `hoinlee-moi/git5:ma`

 [Contribute](#)  [Sync fork](#)

 **This branch is out-of-date**  
Update branch to keep this branch up-to-date by syncing 2 commits from the upstream repository.  
[Learn more about syncing a fork](#)

[Compare](#)

[Update branch](#)

## 12. Fork & Pull Request

 indiflex / **git555**

[Code](#) [Pull requests](#) [Actions](#) [Projects](#) [Wiki](#)

 **git555** Public  
forked from [hoinlee-moi/git5](#)

[main](#) 3 Branches 0 Tags

This branch is up to date with [hoinlee-moi/git5:main](#).

 **hoinlee-moi** Merge pull request [hoinlee-moi#2](#) from indiflex/indiflex-patch-2

 README.md Update README.md

 **README**

### Update README.md #2

Merged hoinlee-moi merged 1 commit into [hoinlee-moi:main](#) from [indiflex:indiflex-patch-2](#)

Conversation 0 Commits 1 Checks 0 Files changed 1

 **indiflex** commented 7 minutes ago

CL  
-aaa

 Update README.md

 **hoinlee-moi** merged commit **7ff5991** into [hoinlee-moi:main](#) 4 minutes ago

 **Pull request successfully merged and closed**  
You're all set—the [indiflex:indiflex-patch-2](#) branch can be safely deleted.

 Add a comment

## 12. Fork & Pull Request

indiflex / git555

Q Type / to search

[Code](#) [Pull requests](#) [Actions](#) [Projects](#) [Wiki](#) [Security](#) [Insights](#) [Settings](#)

**git555** Public

[Pin](#) [Watch](#) 0

forked from [hoinlee-moi/git5](#)

 main ▾

 3 Branches

 0 Tags

Go to file t +

<> Code ▾

This branch is up to date with [hoinlee-moi/git5:main](#).

 [Contribute](#) ▾

 [Sync fork](#) ▾

 **hoinlee-moi** Merge pull request [hoinlee-moi#2](#) from [indiflex/indiflex-patc...](#) 7ff5991 · 7 minutes ago

 3 Commits

 README.md

Update README.md

15 minutes ago

 README



# git5

## 12. Fork & Pull Request

indiflex / git555

<> Code Pull requests Actions Projects Wiki Security Insights

### Commits

main

Commits on May 23, 2024

Merge pull request [hoinlee-moi#2](#) from [indiflex/indiflex-patch-2](#) 

 hoinlee-moi committed 7 minutes ago [Show description for 7ff5991](#)

Update README.md

 indiflex committed 16 minutes ago

Commits on Mar 15, 2024

first commit

 hoinlee-moi committed 2 months ago

### Commits

main

Commits on May 23, 2024

Merge pull request [hoinlee-moi#2](#) from [indiflex/indiflex-patch-2](#) 

ok! Update README.md  
LGTM

 hoinlee-moi committed 7 minutes ago [Hide description for 7ff5991](#)

Update README.md

 indiflex committed 16 minutes ago

Commits on Mar 15, 2024

## 12. Fork & Pull Request

### Update README.md #1

 Open indiflex wants to merge 1 commit into `hoinlee-moi:main` from `indiflex:indiflex-patch-1` 

 Conversation 0  Commits 1  Checks 0  Files changed 1



indiflex commented 1 minute ago

No description provided.



 Update README.md

Verified

f4d256f

Add more commits by pushing to the `indiflex-patch-1` branch on `indiflex/git555`.



**This branch has no conflicts with the base branch**

Only those with [write access](#) to this repository can merge pull requests.



Add a comment

Write

Preview

H

B

I

≡

<>

🔗

≡

≡

≡

🔗

@

🗨️

↩️

📄

Add your comment here...

 Markdown is supported

 Paste, drop, or click to add files

 Close pull request

Comment

### Update README.md #1

 Closed

indiflex wants to merge 1 commit into `hoinlee-moi:main` from `indiflex:indiflex-patch-1`

 Conversation 0

 Commits 1

 Checks 0

 Files changed 1



indiflex commented 2 minutes ago

No description provided.



ho\_in

Re: [indiflex/seocho] full request edit by hoinlee (PR #1)

받는 사람: indiflex/seocho, 참조: Subscribed,

답장 받는 사람: indiflex/seocho



이 메시지는 메일링 리스트에서 옵니다.

Closed [#1](#).

—  
Reply to this email directly, [view it on GitHub](#), or [unsubscribe](#).  
You are receiving this because you are subscribed to this thread.



# 연습문제

옆 사람의 리포를 Fork하고  
수정 후 PR을 날려보자.

1. 옆 사람의 Repo로 이동
2. 해당 repo에서 create fork
3. fork한 repo를 PC에 clone
4. pc에 원본(옆사람) 리포도 remote로 등록  
`git remote add <별칭> https://github.com/yourusername/yourusername.github.io`
5. <작업 branch> 생성
6. `commit & push <작업branch>`
7. PR 생성 & 보내기
8. Owner는 Review & Merge



## 원본 Repository

Watch 1

Fork 26

Star

<> Code

### About

디지털 하나로4기 평가 문제

Readme

Activity

0 stars

1 watching

26 forks

Report repository

7 Commits

2 hours ago

2 hours ago

30 minutes ago

## Forks

Period: 2

hhbb0081 / hana4exam

0

0

0

0

Created 1 hour ago

Update

hoinlee-moi / hana4exam

0

0

0

0

Created 33 minutes ago

U

hometelier / hana4examH

0

0

0

0

Created 32 minutes ago

U

JungSungYeob / hana4exam

0

0

0

0

Created 1 hour ago

Update

Sionparadox / hana4exam

0

0

0

0

Created 1 hour ago

Update

cf. [gh](#) (github cli)

```
$> gh --version
$> gh auth login
$> gh pr create --base main --head feature-branch --title
"Add feature X" --body "message..."
```

# Take GitHub to the command line

GitHub CLI brings GitHub to your terminal. Free and open source.

brew install gh

or

Download for Mac

[View installation instructions](#) →

## Windows

gh is available via [WinGet](#), [scoop](#), [Chocolatey](#), [Conda](#), [Webi](#), and as downloadable MSI.

```
~/workspace/tmp/pr/ppp(ttt) $ gh issue list
To get started with GitHub CLI, please run: gh auth login
Alternatively, populate the GH_TOKEN environment variable with a GitHub API authentication token.
~/workspace/tmp/pr/ppp(ttt) $ gh auth login
? Where do you use GitHub? GitHub.com
? What is your preferred protocol for Git operations on this host? HTTPS
? Authenticate Git with your GitHub credentials? Yes
? How would you like to authenticate GitHub CLI? Paste an authentication token
Tip: you can generate a Personal Access Token here https://github.com/settings/tokens
The minimum required scopes are 'repo', 'read:org', 'workflow'.
? Paste your authentication token: *****
- gh config set -h github.com git_protocol https
✓ Configured git protocol
✓ Logged in as ifsico
```

```
~/workspace/hana7(master) $ gh issue list
HTTP 401: Bad credentials (https://api.github.com/graphql)
Try authenticating with: gh auth login
~/workspace/hana7(master) $ gh auth login
? Where do you use GitHub? GitHub.com
? What is your preferred protocol for Git operations on this host? HTTPS
? Authenticate Git with your GitHub credentials? Yes
? How would you like to authenticate GitHub CLI? Login with a web browser

! First copy your one-time code: 9E5F-0576
Press Enter to open https://github.com/login/device in your browser...

√ Authentication complete.
- gh config set -h github.com git_protocol https
√ Configured git protocol
√ Logged in as indiflex
~/workspace/hana7(master) $
```

```
~/workspace/hana7(master) $ gh pr checks
no pull requests found for branch "master"
```



**Congratulations, you're all set!**

Your device is now connected.

⚙️ General

Access

👤 Collaborators

💬 Moderation options ▼

Code and automation

🔗 Branches

🏷️ Tags

📄 Rules ▲

Rulesets

▶️ Actions ▼

🔗 Models

Preview

🔗 Webhooks

## Rulesets / New branch ruleset



### Protect your most important branches

[Rulesets](#) define whether collaborators can delete or force as passing status checks or a linear commit history.

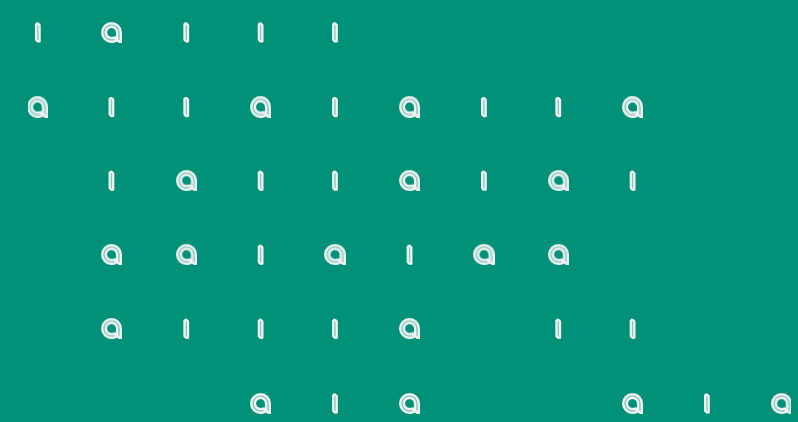
Ruleset Name \*

dont merge to master

Enforcement status

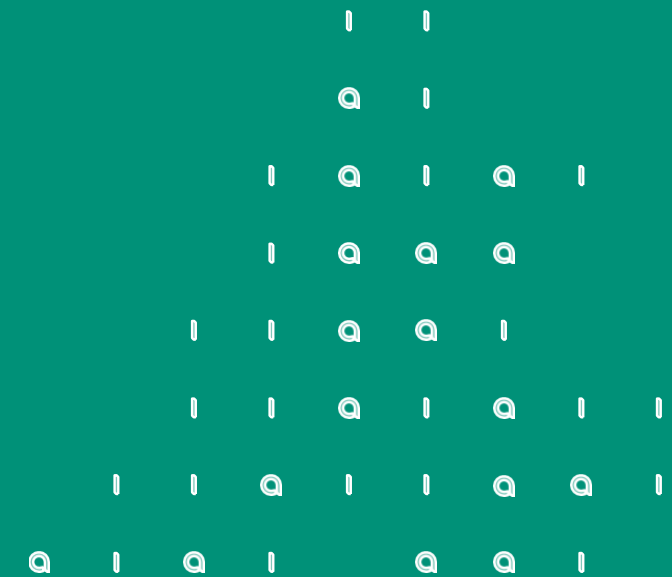
▶️ Active ▼

## Bypass list



## 13. Best Practice in a Project

---





## 13. Best Practice in a Project

---

### Getting started with Project

1. Repository 생성
2. git clone 또는 git remote add origin ...
3. master> git checkout -b develop # develop branch 생성
4. git pull origin develop # develop branch 올리기
5. github **rulesets** setting # branch 보호 및 Merge 규칙 등록
6. develop> git checkout -b feature... # feature branch 생성 (local)
7. feature> git rebase develop # feature 작업 및 rebase
8. feature> git push origin feature... # feature 작업 완료 & push
9. PR to develop
10. Code Review & Merge

(주의) 이미 push 한 feature branch는 rebase 하면 안됨! ⇒ merge 이용!  
feature> git fetch origin develop  
feature> git merge origin/develop  
cf. git push --force-with-lease origin feature # 조건부 덮어쓰기 (--force는 강제)  
PR 날린 branch는 절대 덮어쓰지 말 것! (:commit-log 및 comment 모두 변경!)

## 13. Best Practice in a Project

github - Rulesets **name & activate6**

**repository > settings**

The screenshot shows the GitHub repository settings page. On the left sidebar, the 'Rules' section is highlighted with a red circle, and 'Rulesets' is selected. The main content area is titled 'Rulesets / New branch ruleset'. It features a section 'Protect your most important branches' with a description: 'Rulesets define whether collaborators can delete or force as passing status checks or a linear commit history.' Below this, the 'Ruleset Name \*' field is labeled with a red note 'Ruleset 이름' and contains the text 'dont merge to master'. The 'Enforcement status' is set to 'Active', with a red note 'Ruleset 활성화 여부' next to it. At the bottom, there is a 'Bypass list' section and a 'Preview' button.

General

Access

Collaborators

Moderation options

Code and automation

Branches

Tags

**Rules**

Rulesets

Actions

Models

Webhooks

Rulesets / New branch ruleset

Protect your most important branches

Rulesets define whether collaborators can delete or force as passing status checks or a linear commit history.

Ruleset Name \*

Ruleset 이름

dont merge to master

Enforcement status

Active

Ruleset 활성화 여부

Bypass list

Preview



## 13. Best Practice in a Project

github - Rulesets - **bypass & target**

The screenshot displays the GitHub Rulesets configuration interface. It is divided into three main sections: **Bypass list**, **Targets**, and **Target branches**.

- Bypass list:** Contains the text "Exempt roles, teams, and apps from this ruleset by adding them to the bypass list." Below this, a list of actors is shown, including "Repository admin Role". A red annotation "Ruleset 제약을 회피할 수 있는 권한들 (ex. 배포담당자는 master에 push 가능)" points to this section. A red circle highlights the "Repository admin Role" entry, with a blue arrow pointing from the "Add bypass" button above.
- Targets:** Contains the text "Which branches do you want to make a ruleset for?".
- Target branches:** Contains the text "Branch targeting determines which branches of branches under this ruleset. Use exclusion patterns to exclude branches." Below this, a list of "Branch targeting criteria" is shown, including "master". A red circle highlights the "Include by pattern" option in the "Add target" dropdown, with a blue arrow pointing from the "Include by pattern" option in the "Add target" dropdown.

Two modal windows are open:

- Add bypass:** A modal window titled "Add bypass" with the text "Choose which roles, teams, and apps can bypass this ruleset". It features a search bar "Filter bypass actors" and a list of actors: "Deploy keys", "Repository admin Role" (checked and circled in red), and "Maintain Role".
- Include by pattern:** A modal window titled "Include by pattern" with the text "Branches that match the matching pattern will be targeted by this ruleset." It features a search bar "Branch naming pattern" with the value "master" and a list of example patterns: "main", "releases/\*\*/\*", "users/\*\*/\*". It includes a "Cancel" button and an "Add Inclusion pattern" button.

### 13. Best Practice in a Project

github - Rulesets -  
Branch rules

#### Branch rules

- ☐ Restrict creations  
Only allow users with bypass permission to create matching refs.
- ☐ Restrict updates  
Only allow users with bypass permission to update matching refs.
- ☒ Restrict deletions  
Only allow users with bypass permissions to delete matching refs.
- ☐ Require linear history  
Prevent merge commits from being pushed to matching refs.
- ☐ Require deployments to succeed  
Choose which environments must be successfully deployed to before merging.
- ☐ Require signed commits  
Commits pushed to matching refs must have verified signatures.

☒ **Require a pull request before merging**  
Require all commits be made to a non-target branch and submitted via a pull request before they can be merged.

Hide additional settings ^

**Required approvals**  
2 ▾  
The number of approving reviews that are required before a pull request can be merged.

☐ Dismiss stale pull request approvals when new commits are pushed  
New, reviewable commits pushed will dismiss previous pull request review approvals.

☐ Require review from Code Owners  
Require an approving review in pull requests that modify files that have a designated code owner.

☐ Require approval of the most recent reviewable push  
Whether the most recent reviewable push must be approved.

☐ Require conversation resolution before merging  
All conversations on code must be resolved before merging.

☐ Request pull request review from Copilot  
Automatically request review from Copilot when creating a pull request.

**Allowed merge methods**  
Merge, Squash, Rebase ▾  
When merging pull requests, you can allow or disallow certain merge methods to be enabled.

☐ Require status checks to pass  
Choose which status checks must pass before merging. All checks must pass.

☒ **Block force pushes**  
Prevent users with push access from force pushing to the target branch.

☐ Require code scanning results  
Choose which tools must provide code scanning results. All checks must be enabled and have results for both the commit and the pull request.

Create

## 13. Best Practice in a Project

Tip. github - Projects

github (or repository) > Projects

The collage illustrates the GitHub Projects interface. The top screenshot shows the repository 'indiflex / hana7' with the 'Projects' tab highlighted in a red circle. The middle screenshot shows the 'indiflex' profile with the 'Projects' tab highlighted in a red circle. The bottom screenshot shows the 'Welcome to projects' modal with a '+ New project' button highlighted in a red circle. The modal also displays a project table for 'OctoArcade Invaders'.

| Title                                   | Team      | Assignees       | Status |
|-----------------------------------------|-----------|-----------------|--------|
| <b>Prototype</b>                        |           |                 |        |
| 1 Game brief and go-no-go               | Producers | jannespeters    |        |
| 2 Engine prototype (physics, rendering) | Engine    | mariorod and pm |        |

Welcome to projects

Built like a spreadsheet, project tables give you a live canvas to filter, sort, and group issues and pull requests. Tailor them to your needs with custom fields and saved views.

[Learn more](#) [Jump right in](#)

13. Best Practice in a Project

Tip. github - Projects (Cont'd)

Create project

Project templates

Featured

Start from scratch

Table

Board

Roadmap

Board

Board

New view

Filter by keyword or by field

Planning

25

Estimate: 66

We're working through the details

Not Started

18

Estimate:

This work hasn't started yet

github/OctoArcade

21

Estimate: 33

OctoArcade #40

Scoreboard

epic

Squad 2

High

OctoArcade #44

Design the website

Squad 4

Medium

OctoArcade #20

Schedule social media posts

epic

Squad 4

Medium

OctoArcade #46

Code the frontend

Squad 4

Medium

OctoArcade #43

Develop an amazing website

Squad 4

Medium

OctoArcade #28

Finalize the content strategy

Squad 4

Medium

OctoArcade #47

Code the backend

Squad 4

Medium

OctoArcade #21

Break out the evaluation team to determine final

Squad 4

Medium

Table

Table

New view

Filter by keyword or by field

|   | Title                                          | Assignees    | Status   |
|---|------------------------------------------------|--------------|----------|
| 1 | Updates and bug fixes to sprites #827          | cmwinters    | Review   |
| 2 | Updates and bug fixes to engine from Beta #823 | keisaacson   | Planning |
| 3 | Updates to collision logic #814                | ohiosveryown | Done     |
| 4 | Create user account and subscription #820      | j0siepy      | Building |
| 5 | Update ProjectsSLOs.md #374                    | mariorod     | Building |
| 6 | Save score across levels #800                  | Mattamorphic | Planning |

Roadmap

Roadmap

New view

Filter by keyword or by field

October 2023

GA Launch

Squad 1

7

Fri, Oct 6 - Mon, Oct 30

1

Updates to velocity of the ship and all... #802

Oct 6, 2023

Oct 24, 2023

Updates to velocity of the ship and alien movements #802

2

Create user account and subscription #820

Oct 9, 2023

Oct 16, 2023

Create user account and subscription #820

3

Finalize upsell model #1323

Oct 12, 2023

Oct 30, 2023

Finalize upsell model #1323

4

Blowing in to the cartridge does not fix the bl...

Oct 14, 2023

Oct 18, 2023

Blowing in to the cartridge does not fix the blinking screen

5

Add a leaderboard for Europe #1378

Oct 16, 2023

Oct 23, 2023

Add a leaderboard for Europe #1378

6

Create the design for login screen #39

Oct 16, 2023

Create the design for login screen #39

7

Support light and dark themes

Oct 17, 2023

Support light and dark themes

Squad 2

14

Tue, Jan 31 - Thu, Oct 26

Team planning

Manage your team

## 13. Best Practice in a Project

github - Projects

The screenshot illustrates the workflow for creating a new issue in a GitHub project. The main interface shows the project overview for '@indiflex's untitled project'. A sidebar on the left contains a 'Todo' section with 0 items and a '+ Add item' button highlighted with a red box. A modal window titled 'Create new issue' is open, showing the 'Repository' dropdown set to 'indiflex/hana7'. Below this, the 'Templates and forms' section is visible. A second modal window, 'Create new issue in indiflex/hana7', is shown in the foreground, detailing the issue creation process. It includes a title field with the text 'regist & login', a description field with the text 'credentials and SNS(kakao, naver, google) regist & login', and a 'Create' button. A third modal window, 'regist & login #3', is also visible, showing the issue details, including the title, description, and a 'Create sub-issue' button. Arrows indicate the flow from the '+ Add item' button to the 'Create new issue' modal, and from the 'Create new issue' modal to the 'regist & login #3' modal.

@indiflex's untitled project

View 1 + New view

Filter by keyword or by field

Todo 0

This item hasn't been started

Create new issue

Repository \*  
indiflex/hana7

Templates and forms

regist & login #3

Open indiflex/hana7 Public

indiflex opened 2 minutes ago

credentials and SNS(kakao, naver, google) regist & login

Create sub-issue

Assignees  
No one - [Assign yourself](#)

Labels  
No labels

Cancel Create

## 13. Best Practice in a Project

issue로 branch까지 한번에 만들 수 있다!

Assignees

indiflex

Labels

No labels

Projects

@indiflex's untitled project

Status In Progress

Assignees

indiflex

### Create a branch for this issue

#### Branch name

3-regist-login

#### Repository destination

indiflex/hana7

#### Branch source

master

#### Branch source

develop

### Development

[Create a branch](#) for this issue or link a pull request.

### Checkout in your local repository

Run the following commands in your local clone.

```
git fetch origin
git checkout 3-regist-login
```

### Todo 1

This item hasn't been started

Draft

aaaaaaa

Convert to issue

Copy link in project

### Todo 1

This item hasn't been started

Draft

#### Select an item

Search repositories

hana7

ydp7

kc5

spclecture

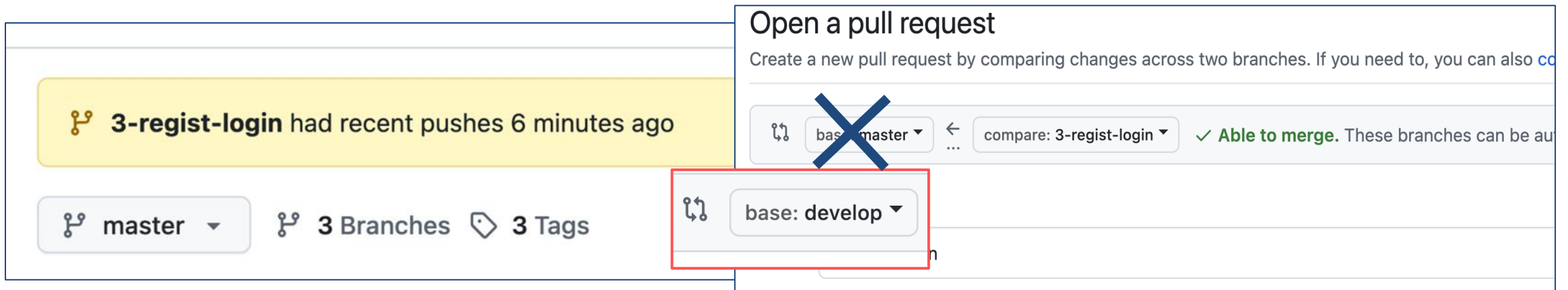


## 13. Best Practice in a Project

Working in a feature branch

1. local에서 feature branch 생성 (또는 github projects 생성 → fetch)  
\$> git checkout -b feature
2. feature> working...
3. feature> git rebase develop # feature 작업 및 rebase
4. feature> git push origin feature... # feature 작업 완료 & push
5. PR to develop & code review & merge to develop

```
~/workspace/hana7/next/my-app(sss) $ git fetch
From https://github.com/indiflex/hana7
* [new branch] 3-regist-login -> origin/3-regist-login
```



## 13. Best Practice in a Project

### Merge & conflict

conflict 발생 시,  
feature에서 다시 develop을 merge한 후  
다시 push 하면 자동 해제됨! (refresh)

The screenshot shows a GitHub Pull Request (PR) titled "Update README.md #2". The PR is from the branch "indiflex:indiflex-patch-2" and is targeting the "hoinlee-moi:main" branch. The PR status is "Open".

Below the PR title, there is a comment from "indiflex" with the content "CL -aaa".

Below the comment, there is a list of checks:

- Require approval from specific reviewers before merging (Rule sets ensure specific people approve pull requests before they're merged.)
- Continuous integration has not been set up (GitHub Actions and several other apps can be used to automatically catch bugs and enforce style.)
- This branch has no conflicts with the base branch (Merging can be performed automatically.)

The "Merge pull request" button is highlighted with a red circle.

On the right side of the PR, there are sections for "Reviewers", "Assignees", "Labels", "Projects", "Milestone", "Development", and "Notifications".



## 13. Best Practice in a Project

### Tip. Commit Message Template

1. .gitmessage 파일 생성
2. `$> git config commit.template .gitmessage`
3. `$> git commit` 하면 아래와 같이 에디터 열림!

```
🎯 [타입]: 간단한 제목
1
2 # 📝 상세 설명 (선택사항)
3 # - 변경 사항에 대한 설명
4 # - 왜 이 변경이 필요한지
5
6 # 🔗 관련 이슈
7 # Fixes #이슈번호
8 # Closes #이슈번호
9
10 # 📋 체크리스트
11 # - [] 테스트 작성/수정
12 # - [] 문서 업데이트
13 # - [] 코드 리뷰 요청
```

#### sample of .gitmessage

# 🛠 [타입] 제목 (50자 이내)

# 📄 설명 (선택사항):

# 🔗 이슈: #

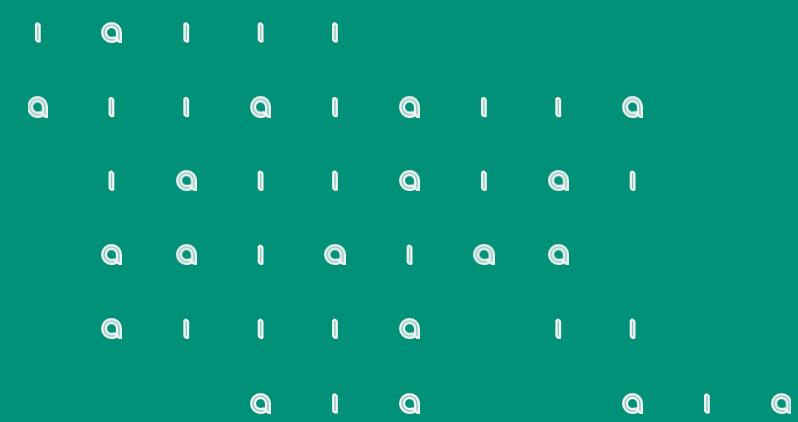
# 타입:

# feat - 새 기능   fix - 버그 수정   docs - 문서

# style - 스타일   refactor - 리팩토링   test - 테스트

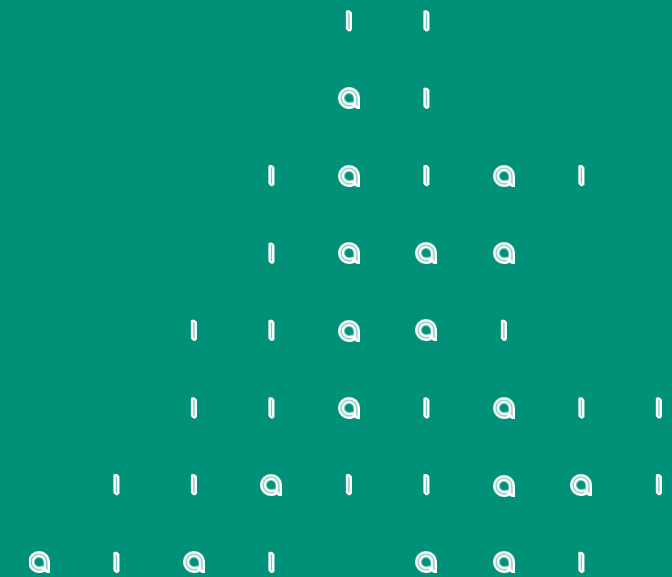
# chore - 기타   hotfix - 긴급 수정

작성한 내용만 commit 메시지로  
등록되고, 기존에 있던 Template  
내용은 포함 안됨!



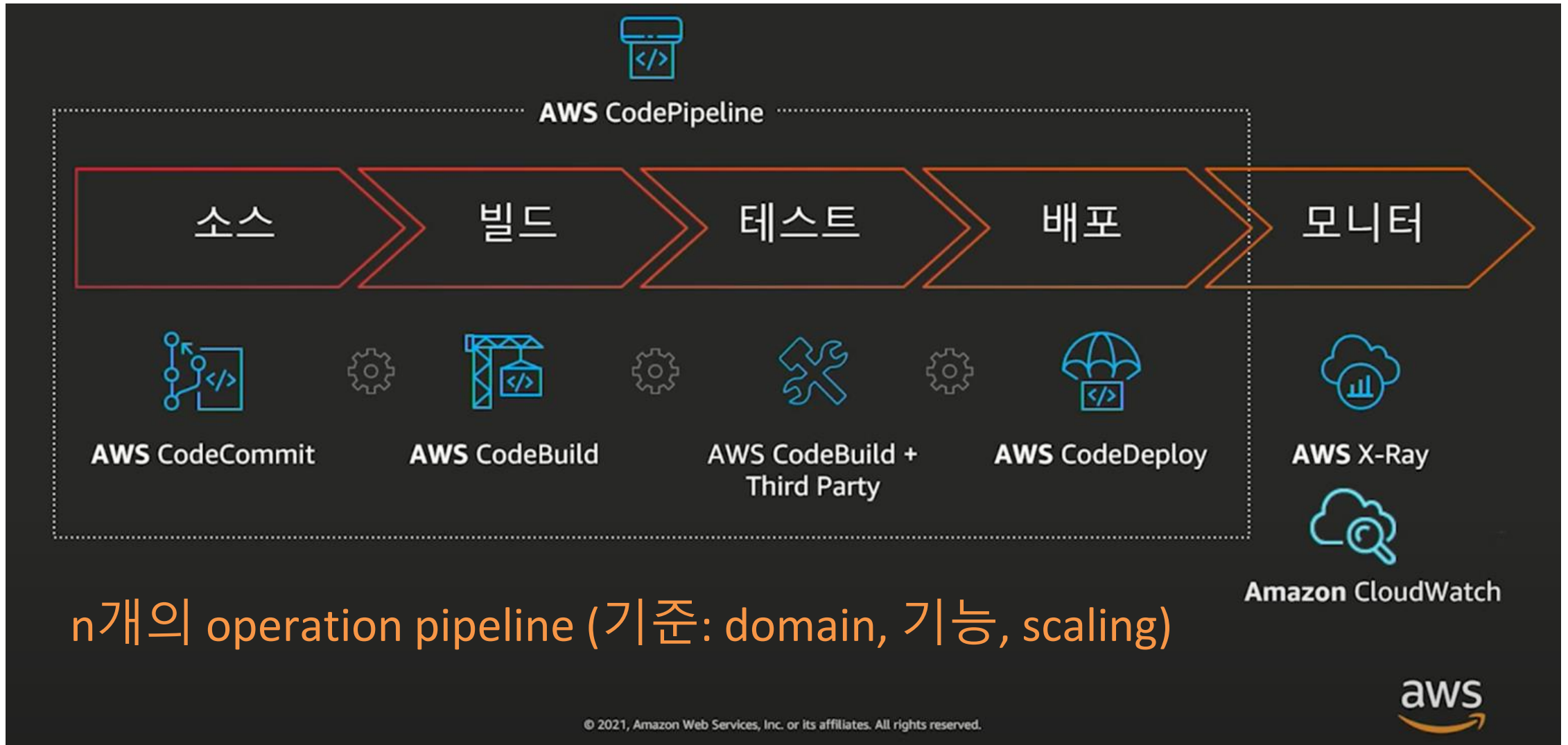
## 14. Github Action

---



## 14. Github Action

cf. DevOps & CI/CD



n개의 operation pipeline (기준: domain, 기능, scaling)

## 14. Github Action

---

### Github Action 시작하기

#### ❖ Github Action의 코어 개념

- 워크플로(Workflow)
  - ✓ 워크플로는 정의된 작업들의 집합 워크플로는 YAML 형식으로 `.github/workflows/` 폴더에 저장
  - ✓ 워크플로는 이벤트에 의해 트리거되며, 각 워크플로는 하나 이상의 Job으로 구성
- 잡(Job)
  - ✓ Job은 하나 이상의 단계(Step)로 구성. Job은 독립적으로 실행되며, 실행되는 환경도 설정
  - ✓ Job은 다양한 환경에서 실행할 수 있으며, 예를 들어 `ubuntu-latest`, `windows-latest`, `macos-latest` 등의 운영 체제에서 실행될 수 있음
- 단계(Step)
  - ✓ 각 단계는 하나의 작업을 수행하는 명령어입니다. 단계 내에서는 셸 스크립트 명령어를 실행하거나, GitHub에서 제공하는 액션(Action)을 호출할 수 있음
- 액션(Action)
  - ✓ GitHub Actions 워크플로에서 실행할 수 있는 개별 작업
  - ✓ 예를 들어, 코드 빌드, 테스트 실행, AWS에 배포 등이 액션으로 구현될 수 있음
  - ✓ GitHub에서는 공식 액션뿐만 아니라 사용자가 만든 커스텀 액션도 사용할 수 있음
- 이벤트(Event)
  - ✓ 이벤트는 워크플로를 트리거하는 조건. 예) `push`, `pull_request`, `issue` 생성 등
- Runner
  - ✓ Github Action Runner 어플리케이션이 설치된 머신으로, Workflow가 실행될 인스턴스

## 14. Github Action

### workflow 작성법

```
name: Workflow 이름
on: push: branches: [master]
jobs:
 start_build:
 runs-on: ubuntu-latest
 steps:
 - name: echo
 run: echo "Start Ci/CD"
 - name: pwd
 run: pwd
 - name: ls
 run: ls -al
```

```
on:
 push:
 branches:
 - main
 - release/*
```

```
on:
 pull_request:
 branches:
 - main
```

```
on:
 schedule:
 - cron: "0 2 * * 1-5"
```

```
jobs:
 my_job:
 name: deploy to staging
 runs-on: ubuntu-22.04
```

```
- name: Setup Node
 uses: actions/setup-node@v4
 with:
 node-version: '20.x'
```

```
- name: Install Dependencies
 run: npm install
```

- ubuntu-latest, ubuntu-22.04, or ubuntu-20.04
- windows-latest, windows-2022, or windows-2019
- macos-latest, macos-13, or macos-12

## 14. Github Action

cf. push 안될 때

Github Profile

- > Settings
- > Developer Settings
- > Personal access tokens

or Repo. Setting > SSH Key 등록

GitHub Apps

OAuth Apps

Personal access tokens ^

Fine-grained tokens Preview

Tokens (classic)

### Edit personal access token (classic)

If you've lost or forgotten this token, you can regenerate it, but be aware that scripts or applications using this token will need to be updated.

**Note**

indiflex\_token1

What's this token for?

**Expiration**

This token has no expiration date. To set a new expiration date, you must

**Select scopes**

Scopes define the access for personal tokens. [Read more about OAuth scopes](#)

|                                                     |                                      |
|-----------------------------------------------------|--------------------------------------|
| <input checked="" type="checkbox"/> repo            | Full control of private repositories |
| <input checked="" type="checkbox"/> repo:status     | Access commit status                 |
| <input checked="" type="checkbox"/> repo_deployment | Access deployment status             |
| <input checked="" type="checkbox"/> public_repo     | Access public repositories           |
| <input checked="" type="checkbox"/> repo:invite     | Access repository invitations        |
| <input checked="" type="checkbox"/> security_events | Read and write security events       |
| <input checked="" type="checkbox"/> workflow        | Update GitHub Action workflows       |
| <input checked="" type="checkbox"/> write:packages  | Upload packages to GitHub Packages   |

## 14. Github Action

-Dspring.profiles.active=prod

cf. spring에서 application.properties 분리하기

application.properties 에서 민감한 부분이나 prod와 다른 부분은 파일 분리

case 1) application.properties, application-dev.properties, application-**prod**.properties 와 **application-secret.properties**로 분리  
(**prod에는 secret 내용도 함께 담고 github 환경변수로 저장!**)

case 2) application.properties 와 **application-secret.properties** 2개로 분리

```
application.properties
spring.application.name=spring-demo
server.port=8081
...
```

```
spring.profiles.include=secret
```

```
application-secret.properties
```

```
S3
```

```
cloud.aws.region.static=ap-northeast-2
```

```
cloud.aws.s3.bucket=sico2025-uploads
```

```
aws.accessKeyId=AKIA2AAGCF523BUMWNUZ
```

```
aws.secretAccessKey=j1MIUMEYv8sVRLigZM8apX/+5qAWtV6myBLBWLC6
```

```
MySQL
```

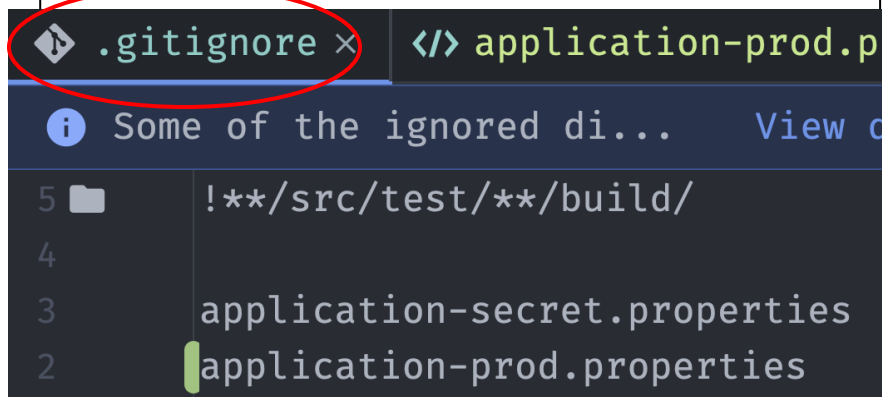
```
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver
```

```
spring.datasource.url=jdbc:mysql://localhost:3309/tdb?serv...
```

```
spring.datasource.username=root
```

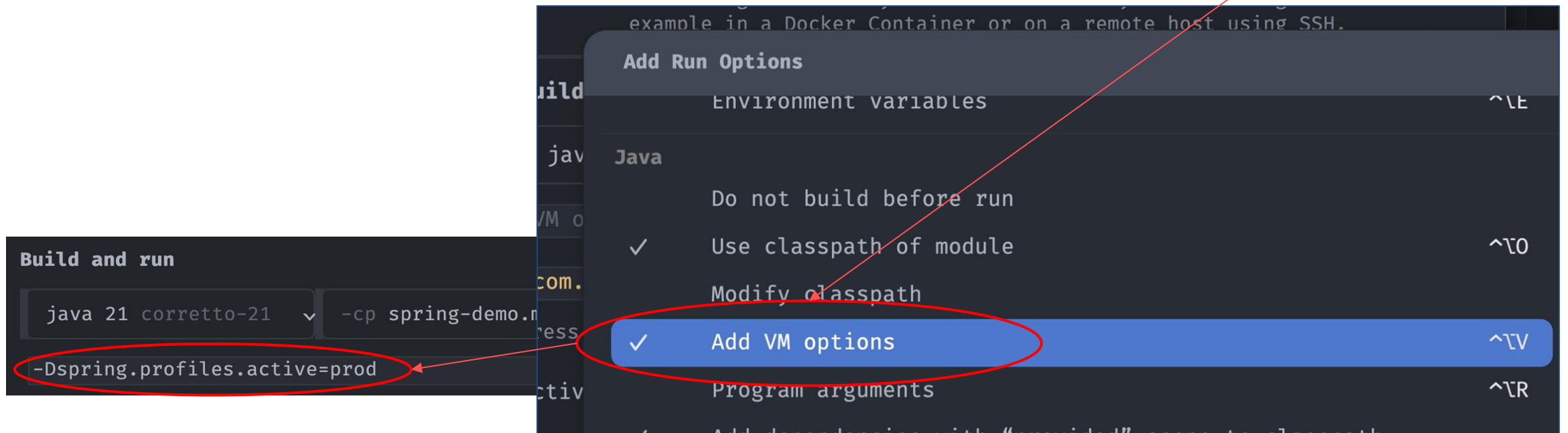
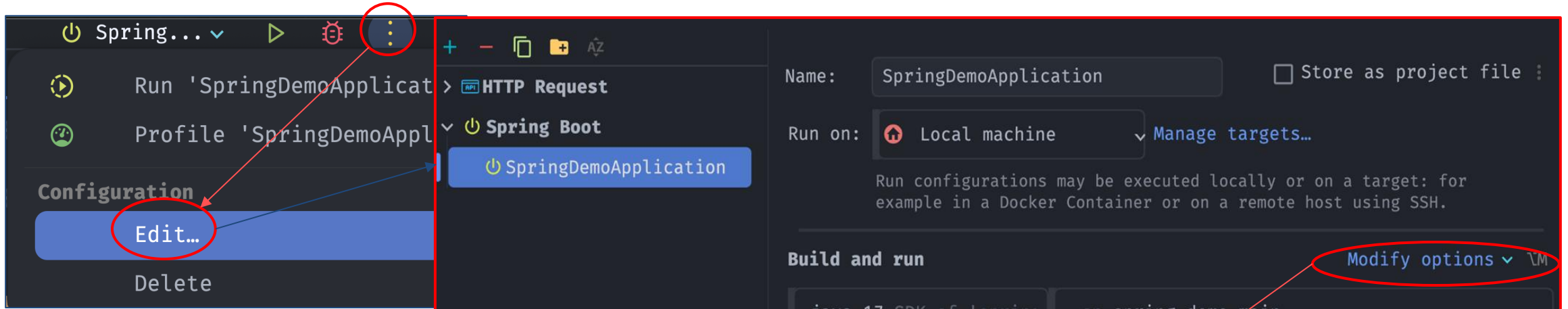
```
spring.datasource.password=TestdbRoot
```

(주의) application-secret.properties은 .gitignore에 추가!  
(application-prod.properties에 실DB 정보 있다면 함께 추가!)



## 14. Github Action

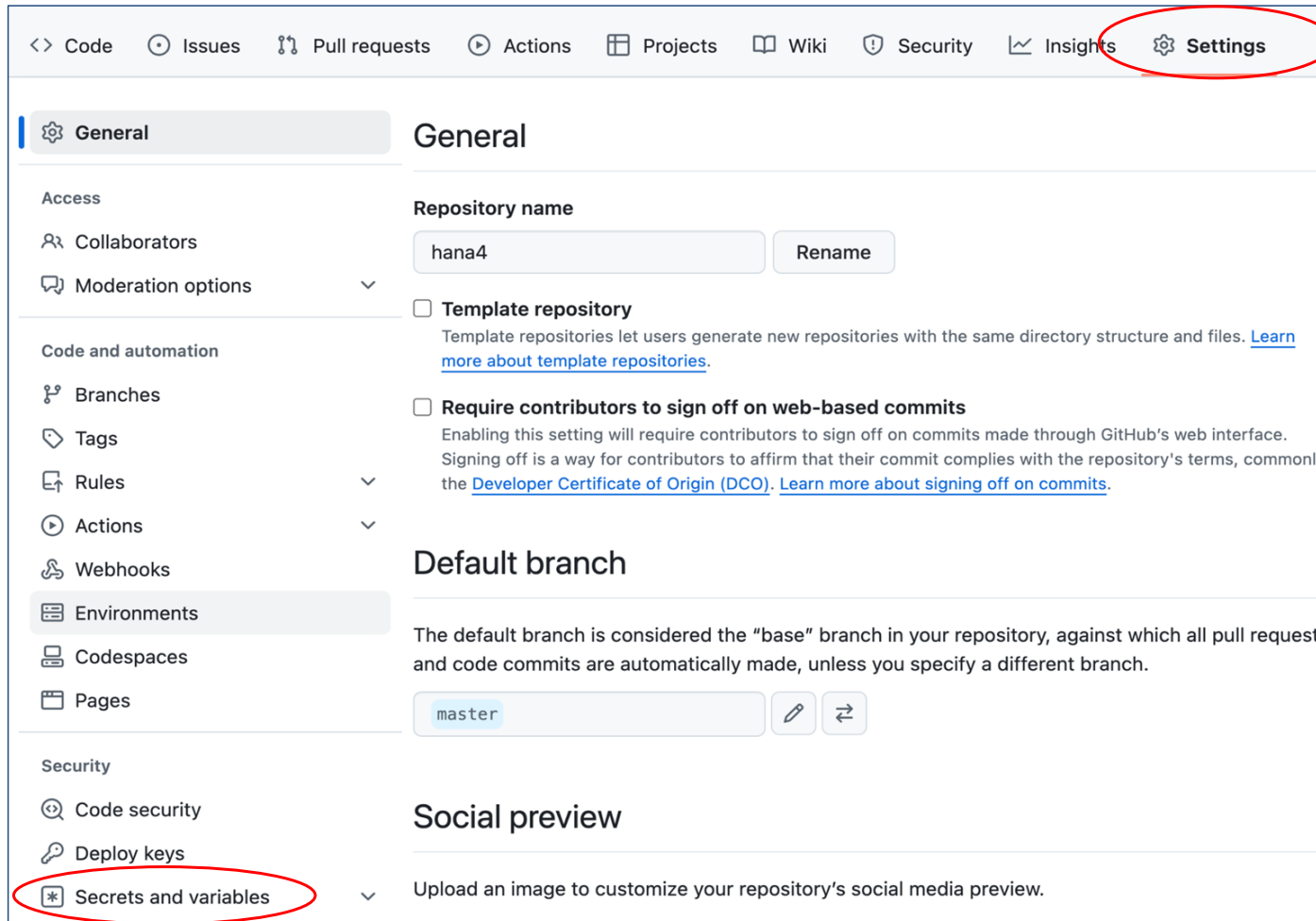
cf. IntelliJ에서 application-prod.properties 로 실행해보기+





## 14. Github Action

### 준비 작업: Github 환경변수 설정



#### Repository secrets

This repository has no secrets.

New repository secret

```
application-prod.properties
JPA on prod
spring.jpa.hibernate.ddl-auto=validate
spring.jpa.show-sql=false
S3
cloud.aws.region.static=ap-northeast-2
cloud.aws.s3.bucket=sico2025-uploads
aws.accessKeyId=AKIA2AAGCFProdKey
aws.secretAccessKey=j1MlUMEYv8sVRLigZM8apX/+5qA
MySQL
spring.datasource.driver-class-
name=com.mysql.cj.jdbc.Driver
spring.datasource.url=jdbc:mysql://sico2025db.c9s...rds
.amazonaws.com/sico2025db?serverTimezone=Asia/Seoul&ch
aracterEncoding=UTF-8
spring.datasource.username=hanadm
spring.datasource.password=xxxxxxxxx
```

data 옮기기(local ⇒ EC2) 또는 JPA create!

```
$> mysqldump -u root -p -h 127.0.0.1 -P 3309 testdb > dump.sql
```

```
$> scp -i ~/.ssh/sico2025.pem dump.sql ec2-user@ec2-43-203-200-38.ap-northeast-2.compute.amazonaws.com:/home/ec2-user/workspace/
```

```
ec2> mysql -u root -p -h sico2025db.c9s2oiuk8geh.ap-northeast-2.rds.amazonaws.com sico2025db < dump.sql
```

## 14. Github Action

ex) SSH key 등록하기

### Actions secrets / New secret

Name \*

SSH\_KEY

Secret \*

Add secret

### Repository secrets

New repository secret

Name 

Last updated

 SSH\_KEY

3 minutes ago



```
~/workspace/hana7(develop) $ cat ~/.ssh/sico2025.pem
-----BEGIN RSA PRIVATE KEY-----
MIIEowIBAAKCAQEA46Px0zy7INb0aGd0403Rms36TLjyMzLFjcu1WDqweYm3J3q3
DzFcppccKQ2RFRfjSuS7yYv1BykItByz4W7VWxCHX1ZB6uhXnfUC7zdHb1FCw92Z
mk0ggVOLJFxmZ45EEeLZwtKEbtrCkUIOm5oHLeoMHuU0cDU+19cU3J6Xvb96K/zO
aD8s0kf6tqLlзмаPGSuNyxDCBktf8dJYkCQqn/ShVZ03JW9bI5Gs6YMxnqko2664
a4PAOG0W5K0aK9cVmmkYWxcRJ/l33RcZQPYbfFtXSYK5rHnMQsoPLa2GZUzLuZcF
XFRYvIoIYRtl+H99c6z3JKIEsRm4R8NnjPqGSwIDAQABAoIBAQCoRZ9T89er/dmr
d3piYMoJcettXWWEHSrVl1jVAIVFQlZGbTr2mxm097LUUfgGLKNS0GRUJwYYpeeO
JI1/vgsvBrfa7ItLlZZ1SBFEPYurqNj+qkeKNiVmLBgN5903Bv4gbp1kIEHbpBAh
ij/8G87kzNbq241huqpaqN39+6nVk1lM3nHINN9gccXXD77qqcVopd+1ympLosv
t7gFsMlm0SpWBJdd+xEUwbBD8DBkJzyI40+dQBz8tgnMsXEWPrkaBwyWZNoS5HIr
b7G1/KaJftHlKglWiR895XtQsPaqbNOYzYxU91cQzrQBiGpCYBiHVWpwTbP//vk
BODhoHYxAoGBAPVxJAXhaNNDw89a05cG3nP/uZTwasDCC0wtfZrcegvM088ChuFm
0eSxSCImbJUvSsbKf1E9HLGnPUS9PeIKz8Utn9tE+FQurdPQZj0a9Cmog3DpQLfx
SMGirLXY+asQFr8RDE41/G0g2jDXEZVrhiX+Y+TE5b0eUndHctl4gOB5AoGBA01u
xPwWf9A1MBP/710j3dwbmYdei4XQ4qvqvlAg+MacZkgEOAPrNnBVWJzlBaAdzTEu
VvbaszP51nnPyXUE/wK4UddGoiAoMPcSj/7v3hp8Brjvn1dV5KWHtLQjSPbnf/z2
oZKK0y3mScI0eXwsuAUHmugCSHWWtaA8uyvYvpPjAoGAfrgHTTHvmQ0nmOqRUYwu
QpyKUfa/Rz5jm7P4nYUBVQpkpqHuTZhUUrV4RbvleDtAmZX48wiF
-----END RSA PRIVATE KEY-----~/workspace/hana7(develop)
```

## 14. Github Action

### Github 환경변수 설정 (Cont'd)

Pages

Security

- Code security
- Deploy keys
- Secrets and variables**
  - Actions**
  - Codespaces
  - Dependabot
- Integrations

Repository secrets

New repository secret

| Name ↕      | Last updated |
|-------------|--------------|
| APPLICATION | 2 weeks ago  |
| ENV         | 2 weeks ago  |
| SSH_KEY     | 2 weeks ago  |
| WAS         | 2 weeks ago  |

.env

.ssh/ec2-user.pem

server host(ip)

application-prod.properties

```
spring.application.name=demo
#server.port=8090
MySQL
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver
spring.datasource.url=jdbc:mysql://hana4db.c3eyu4ewuz63.ap-northeast-2.rds.a
.com:3306/testdb?serverTimezone=Asia/Seoul&characterEncoding=UTF-8
spring.datasource.username=hana4
```

```

jar {
 enabled = false
}

```

## 14. Github Action

### CI setting (feat. GithubAction)

repository root에 .github 폴더 생성 > 그 속에 workflows 폴더 생성 > 그 속에 ci.yml 파일 생성

name: Demo CI

on:

push:

branches: [ master ]

jobs:

build:

runs-on: ubuntu-latest

steps:

- uses: actions/checkout@v4

- uses: actions/setup-java@v5.0.0

with:

distribution: 'temurin'

java-version: '21'

- name: grant exe to gradlew

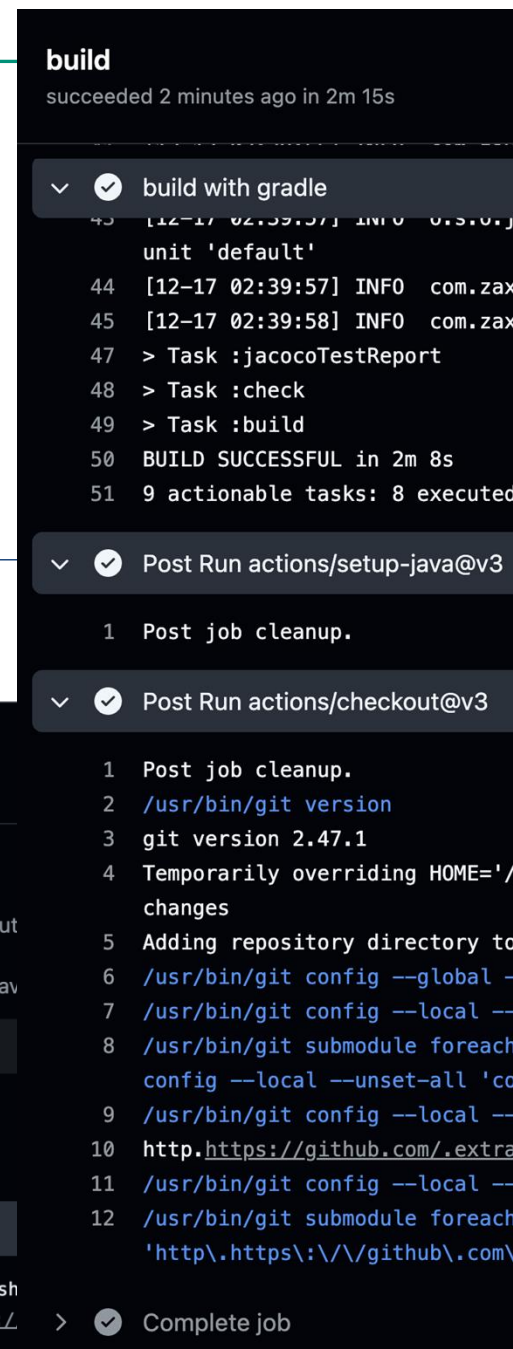
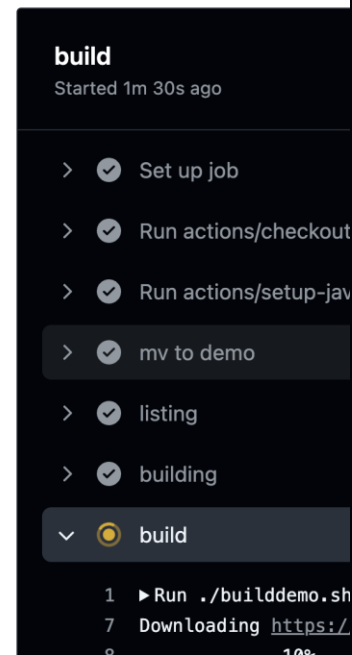
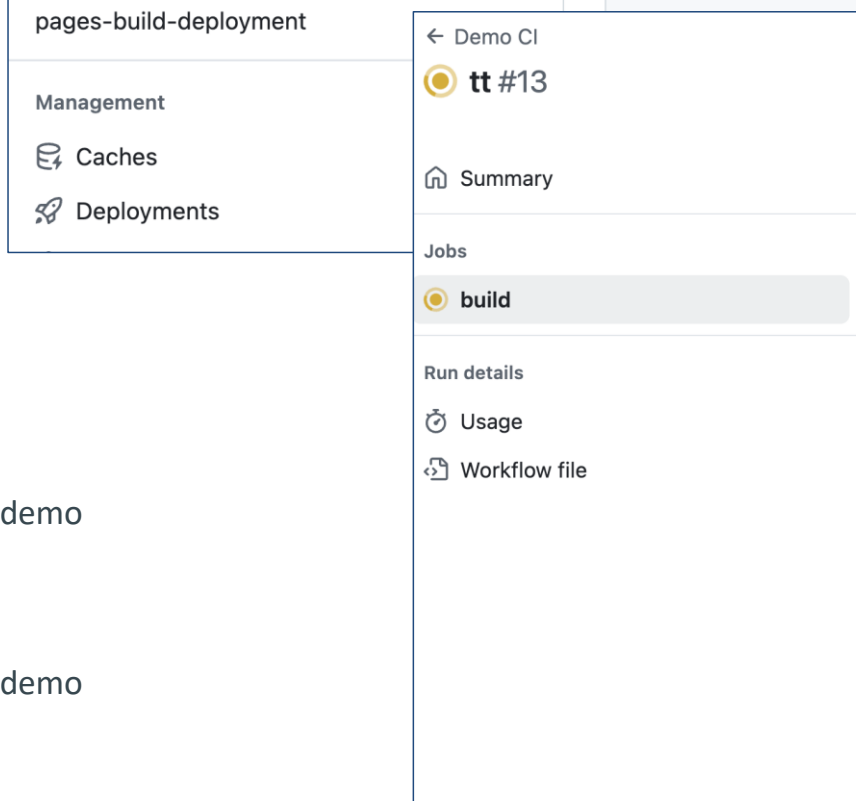
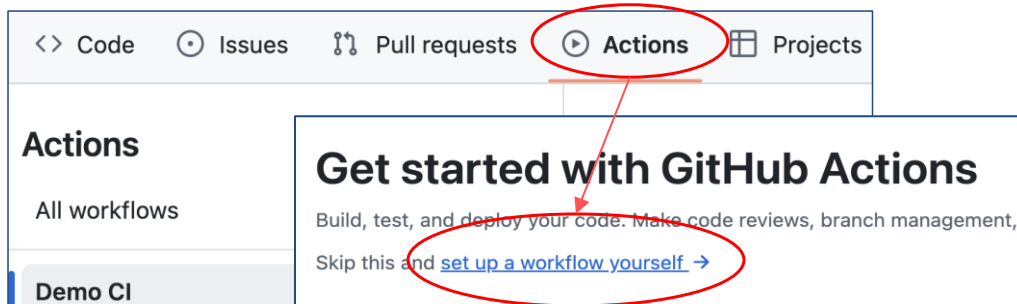
working-directory: spring/spring-demo

run: chmod +x gradlew

- name: build with gradle

working-directory: spring/spring-demo

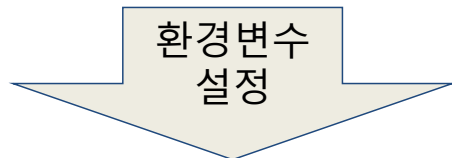
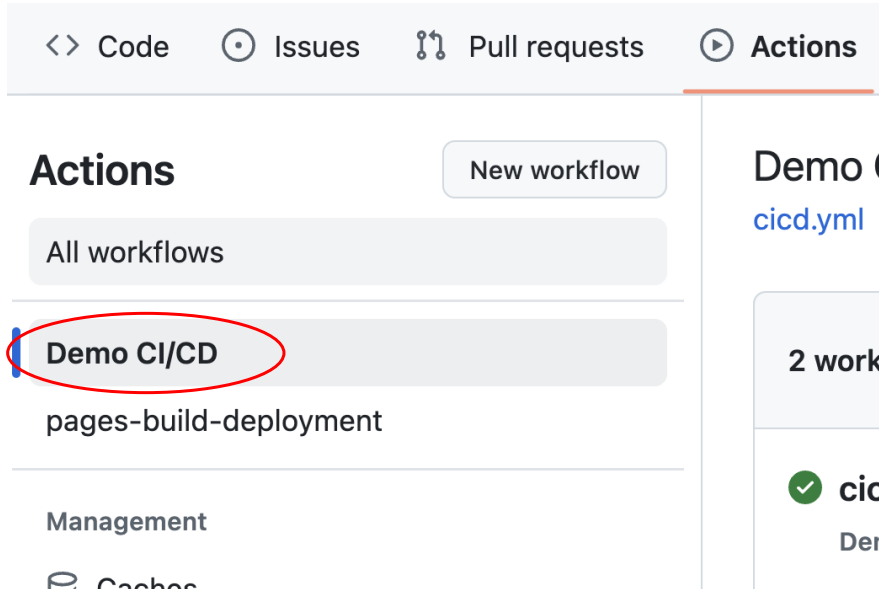
run: ./gradlew clean build -x test





## 14. Github Action

### CD setting (springboot)



.github/workflows/cicd.yml

```
name: Demo CI/CD
on:
 push:
 branches: [master]
```

```
jobs:
 build:
 runs-on: ubuntu-latest
```

```
steps:
 - uses: actions/checkout@v3

 - uses: actions/setup-java@v5.0.0
 with:
 distribution: 'temurin'
 java-version: '21'
```

```
- name: set props
 working-directory: spring/spring-demo
 run: echo "${{ secrets.APPLICATION }}" > ./src/main/resources/application.properties
```

```
- name: grant exe
 working-directory: spring/spring-demo
 run: chmod +x gradlew
```

```
- name: build with gradle
 working-directory: spring/spring-demo
 run: ./gradlew clean build -x test
```

```
- name: setup ssh
 run: |
 mkdir -p ~/.ssh
 echo "${{ secrets.SSH_KEY }}" > ~/.ssh/id_rsa
 chmod 400 ~/.ssh/id_rsa
 ssh-keyscan ${{ secrets.WAS }} >> ~/.ssh/known_hosts
 chmod 600 ~/.ssh/known_hosts
```

```
- name: scp jar to was
 run: |
 scp spring/spring-demo/build/libs/*.jar ec2-user@${{ secrets.WAS }}:/home/ec2-user/workspace/hana.jar
```

build.gradle - plain.jar(묶음 jar) 제거!

```
jar {
 enabled = false
}
```

## 14. Github Action

### 기타 (java-systemctl)

# cf. java 실행을 system service로 등록하기

\$> sudo vi /etc/systemd/system/springboot.service

\$> sudo systemctl daemon-reload

\$> sudo systemctl (start | stop | restart) springboot

[Unit]  
Description=hana7 springboot

[Service]  
SyslogIdentifier=**springboot**  
WorkingDirectory=/home/ec2-user/workspace  
ExecStart=/bin/bash -c "exec java -jar /home/ec2-user/workspace/hana.jar"

# cf. gradle 설치

mkdir ttt && cd \$\_

wget https://services.gradle.org/distributions/gradle-8.14.3-bin.zip

sudo unzip -d /opt/gradle gradle-8.14.3-bin.zip

sudo ln -s /opt/gradle/gradle-8.14.3 /opt/gradle/latest

echo 'export PATH=\$PATH:/opt/gradle/latest/bin' >> ~/.bash\_profile

source ~/.bash\_profile

gradle -v

```
[07:53:37:hana4 ~/workspace]$ tail -f logs/logback2.log
[2024-12-19 07:30:40:85901][http-nio-8080-exec-4] ERROR c.h.demo.controller.eee-postcontroller!!!
[2024-12-19 07:37:27:37944][http-nio-8080-exec-1] DEBUG c.h.demo.controller.dee-postcontroller!!!
[2024-12-19 07:37:27:37946][http-nio-8080-exec-1] INFO c.h.demo.controller.ddd-postcontroller!!!
[2024-12-19 07:37:27:37946][http-nio-8080-exec-1] ERROR c.h.demo.controller.eee-postcontroller!!!
[2024-12-19 07:37:32:43105][http-nio-8080-exec-2] DEBUG c.h.demo.controller.dee-postcontroller!!!
[2024-12-19 07:37:32:43109][http-nio-8080-exec-2] INFO c.h.demo.controller.ddd-postcontroller!!!
[2024-12-19 07:37:32:43109][http-nio-8080-exec-2] ERROR c.h.demo.controller.eee-postcontroller!!!
[2024-12-19 07:37:39:49674][http-nio-8080-exec-3] DEBUG c.h.demo.controller.dee-postcontroller!!!
[2024-12-19 07:37:39:49676][http-nio-8080-exec-3] INFO c.h.demo.controller.ddd-postcontroller!!!
[2024-12-19 07:37:39:49676][http-nio-8080-exec-3] ERROR c.h.demo.controller.eee-postcontroller!!!
```

## 14. Github Action

---

simple test

`.github/workflows/cicd.yml`

```
name: SpringDemo CI/CD
on:
 push:
 branches: [develop]

jobs:
 build:
 runs-on: ubuntu-latest

 steps:
 - uses: actions/checkout@v4

 - name: test pwd
 run: pwd

 - name: test ls
 run: ls -al

 - name: set props
 working-directory: spring/spring-demo
 run: echo "${{ secrets.APPLICATION }}"
```

next.js standalone (^next15)

next.config.ts

```
const nextConfig: NextConfig = {
 /* config options here */
 output: 'standalone',
 images: {
 remotePatterns: [
 ...
]
 }
};
```

cicd.yml

```
- name: make .env file
 working-directory: next/my-app
 run: |
 cat > .env <<'EOF'
 ${ secrets.ENV }
 EOF
```

```
- name: Install pnpm dependencies
 working-directory: next/my-app
 run: |
 pnpm install
 pnpm prisma generate

- name: Build with Next.js
 working-directory: next/my-app
 run: pnpm build
```



## 14. Github Action

cf. next.js @ 문제 해결

next.config.ts

```
const nextConfig: NextConfig = {
 /* config options here */
 webpack: (config) => {
 config.resolve.alias['@'] = __dirname;
 return config;
 },
 images: {
 remotePatterns: [
 ...
]
 }
};
```

```
$ pnpm add -D dotenv-cli
```

```
- name: generate prisma
 working-directory: next/my-app
 run: pnpm db:gen
```

## 14. Github Action

---

cf. styled-jsx, @swc/helpers, @next/env 문제 해결

cicd.yml

```
- name: Install dependencies
 working-directory: next/my-app
 run: |
 pnpm install
 pnpm add styled-jsx
 pnpm add @swc/helpers
 pnpm add @next/env
 pnpm prisma generate
```

pnpm i --prod

## 14. Github Action

### .github/workflows/cicd.yml 전체 (frontend)

```
name: SpringDemo CI/CD
on:
 push:
 branches: [master]
```

```
jobs:
 build:
 runs-on: ubuntu-latest
```

```
steps:
 - uses: actions/checkout@v4
```

```
 - uses: actions/setup-java@v5.0.0
 with:
 distribution: 'temurin'
 java-version: '21'
```

```
 - name: set pring props
 working-directory: spring/spring-demo
 run: echo "${{ secrets.APPLICATION }}" > ./src/main/resources/application.p
 properties
```

```
 - name: grant exe to gradlew
 working-directory: spring/spring-demo
 run: chmod +x gradlew
```

```
 - name: build with gradle
 working-directory: spring/spring-demo
 run: ./gradlew clean build -x test
```

```
 - name: setup ssh
 run: |
 mkdir -p ~/.ssh
 echo "${{ secrets.SSH_KEY }}" > ~/.ssh/id_rsa
 chmod 400 ~/.ssh/id_rsa
 ssh-keyscan ${{ secrets.WAS }} >> ~/.ssh/known_hosts
 chmod 600 ~/.ssh/known_hosts
```

```
 - name: Install Node
 uses: actions/setup-node@v4
 with:
 node-version: '22'
```

```
 - name: setup node pnpm
 uses: pnpm/action-setup@v4
 with:
 version: 10
 run_install: false
```

```
 - name: Install dependencies
 working-directory: next/my-app
 run: |
 pnpm install
 pnpm prisma generate
```

```
 - name: make .env file
 working-directory: next/my-app
 run: |
 cat > .env <<'EOF'
 ${{ secrets.ENV }}
 EOF
```

```
Downloading next@15.3.2: 27.78 MB/27.78 MB, done
Progress: resolved 987, reused 20, downloaded 71, added 23
```

```
 - name: Build with Next.js
 working-directory: next/my-app
 run: pnpm next build
```

```
 - name: zip next
 working-directory: next/my-app
 run: zip -rq next.zip .next
```

```
 - name: scp next to was
 run: |
 scp next/my-app/next.zip ec2-
 user@${{ secrets.WAS }}:/home/ec2-user/workspace/
```

```
 - name: unzip & restart next
 run: |
 ssh ec2-user@${{ secrets.WAS }} '
 cd /home/ec2-user/workspace
 rm -rf my-app/.next
 unzip -q next.zip -d my-app
 pm2 reload my-next-app
 '
```

```
 - name: scp jar to was
 run: |
 scp spring/spring-demo/build/libs/*.jar ec2-
 user@${{ secrets.WAS }}:/home/ec2-user/workspace/hana.jar
```

```
 - name: restart spring
 run: |
 ssh ec2-user@${{ secrets.WAS }} '
 cd /home/ec2-user/workspace &&
 sudo systemctl restart springboot
 '
```

## 14. Github Action

### .github/workflows/cicd.yml (1/3)

```
name: SpringDemo CI/CD
on:
 push:
 branches: [master]

jobs:
 build:
 runs-on: ubuntu-latest

 steps:
 - uses: actions/checkout@v4

 - uses: actions/setup-java@v5.0.0
 with:
 distribution: 'temurin'
 java-version: '21'

 ...
```

```
- name: setup node pnpm
 uses: pnpm/action-setup@v4
 with:
 version: 9
 run_install: false
```

```
- name: setup pages
 uses: actions/configure-pages@v5
 with:
 static_site_generator: next
```

#### Run actions/setup-java@v5.0.0

```
1 ▶ Run actions/setup-java@v5.0.0
13 ▶ Installed distribution
24 Creating settings.xml
25 Writing to /home/runner
```

#### set props

```
1 ▶ Run echo "***
```

#### setup node pnpm

```
1 ▶ Run pnpm/action-setup@v4
11 ▶ Running self-installer...
21 Installation Completed!
```

#### setup pages

```
1 ▶ Run actions/configure-pages@v5
10 Using default blank configuration
11 Injecting property=output and value=export
12 // Default Pages configuration for Next
13 const nextConfig = {}
14 module.exports = nextConfig
15
16 Found configuration object in indirect module
17 Injection successful, new configuration:
```

#### Set up job

```
1 Current runner version: '2.321.0'
2 ▶ Operating System
6 ▶ Runner Image
11 ▶ Runner Image Provisioner
13 ▶ GITHUB_TOKEN Per
17 Secret source: Actions
18 Prepare workflow
19 Prepare all required
20 Getting action details
21 Download action repository
22 Download action repository
23 Download action repository
24 Download action repository
25 Download action repository
26 Complete job name
```

#### Run actions/checkout@v3

```
1 ▶ Run actions/checkout@v3
14 Syncing repository: indiflex/hana4
15 ▶ Getting Git version info
19 Temporarily overriding HOME='/home/runner/work/_temp
changes
20 Adding repository directory to the temporary git glo
21 /usr/bin/git config --global --add safe.directory /h
22 Deleting the contents of '/home/runner/work/hana4/ha
23 ▶ Initializing the repository
37 ▶ Disabling automatic garbage collection
39 ▶ Setting up auth
45 ▶ Fetching the repository
401 ▶ Determining the checkout info
402 ▶ Checking out the ref
406 /usr/bin/git log -1 --format='%H'
407 '58c10b150a63909fd6119cb95808663ef5fdc852'
```

## 14. Github Action

### .github/workflows/cicd.yml (2/3)

```
- name: Install Node
 uses: actions/setup-node@v4
 with:
 node-version: '22'
```

```
- name: Install dependencies
 working-directory: my-app
 run: npm install
```

```
- name: set .env
 working-directory: my-app
 run: |
 echo "${{ secrets.ENV }}" > .env
```

```
- name: grant exe
 working-directory: spring/demo
 run: chmod +x gradlew
```

```
- name: build with gradle
 working-directory: spring/demo
 run: ./gradlew clean build -x test
```

```
- name: setup ssh
 run: |
 mkdir -p ~/.ssh
 echo "${{ secrets.SSH_KEY }}" > ~/.ssh/id_rsa
 chmod 400 ~/.ssh/id_rsa
 ssh-keyscan ${{ secrets.WAS }} >> ~/.ssh/known_hosts
 chmod 600 ~/.ssh/known_hosts
```

#### Install Node

```
1 ▶ Run actions/setup-node@v4
12 Attempting to download 21...
13 Not found in manifest. Fal
14 Acquiring 21.7.3 - x64 fro
15 Extracting ...
16 /usr/bin/tar xz --strip 1
 0e28ed6b6e41 -f /home/runn
17 Adding to the cache ...
18 Done
19 ▶ Environment details
```

#### Install dependencies

```
1 ▶ Run npm install
9 Lockfile is up to date, re
10 Progress: resolved 1, reus
11 Packages: +434
12 ++++++
13 Progress: resolved 434, re
14 Progress: resolved 434, re
15 Progress: resolved 434, re
16 Progress: resolved 434, re
17
18 dependencies:
19 + @heroicons/react 2.1.5
20 + @radix-ui/react-checkbox 1.1.2
21 + @radix-ui/react-icons 1.3.0
22 + @radix-ui/react-label 2.1.0
23 + @radix-ui/react-slot 1.1.0
24 + @radix-ui/react-toast 1.2.2
25 + class-variance-authority 0.7.0
26 + clsx 2.1.1
```

#### set .env

```
1 ▶ Run echo "***
```

#### grant exe

```
1 ▶ Run chmod +x gradlew
```

#### build with gradle

```
1 ▶ Run ./gradlew clean build -x test
9 Downloading https://services.gradle.org/di
1010%.....20%.....
 .80%.....90%.....100%
11
12 Welcome to Gradle 8.10.2!
13
14 Here are the highlights of this release:
15 - Support for Java 23
16 - Faster configuration cache
17 - Better configuration cache reports
```

#### setup ssh

```
1 ▶ Run mkdir -p ~/.ssh
39 # ***:22 SSH-2.0-OpenSSH_8.7
```

## 14. Github Action

.github/workflows/cicd.yml (3/3)

```
- name: Build with Next.js
 working-directory: my-app
 run: npm run build
```

```
- name: zip next
 working-directory: my-app
 run: zip -r next.zip .next
```

```
- name: scp next to was
 run: |
 scp my-app/next.zip ec2-user@${{ secrets.WAS }}:/home/ec2-user/workspace/
```

```
- name: scp jar to was
 run: |
 scp spring/demo/build/libs/*.jar ec2-user@${{ secrets.WAS }}:/home/ec2-user/workspace/hana4.jar
```

```
- name: restart next
 run: |
 ssh ec2-user@${{ secrets.WAS }} '
 cd /home/ec2-user/workspace
 rm -rf hana7/my-app/.next
 unzip -q next.zip -d hana7/my-app &&
 pm2 reload hana-next
 '
```

```
- name: restart spring
 run: |
 ssh ec2-user@${{ secrets.WAS }} '
 cd /home/ec2-user/workspace &&
 sudo systemctl restart springboot
 '
```

```
Build with Next.js
1 ▶Run npm run build
9 ▲ No build cache found. Please
 cache
10 Attention: Next.js now coll
11 This information is used to
12 You can learn more, includi
 following URL:
13 https://nextjs.org/telemetr
14
15 ▲ Next.js 14.2.14
16 - Environments: .env
17
18 Creating an optimized pr
19 ▲ Compiled with warnings
20
38
39 Route (app)
40 | f /
41 | f /_not-found
42 | f /about
43 | ● /about/[id]
44 | | /about/1
45 | | /about/2
46 | | /about/3
47 | | [+17 more paths]
48 | f /about/me
49 | f /api/auth/[...next_auth]
50 | f /api/auth/error
51 | f /api/books
52 | f /api/books/[bookId]
53 | f /books
54 | f /books/[bookId]
55 | f /books/[bookId]/edit
56 | f /hello
57 | f /hello/afternoon
58 | f /hello/evening
59 | f /hello/morning
60 | f /hi
61 | ● /hi/[time]
62 | | /hi/morning
63 | | /hi/afternoon

94 f Middleware
95
96 ○ (Static) prerendered as
97 ● (SSG) prerendered as
98 f (Dynamic) server-rendered
99

zip next
1 ▶Run zip -r next.zip .next

scp next to was
1 ▶Run scp my-app/next.zip ec2

scp jar to was
1 ▶Run scp spring/demo/build/L

restart next
1 ▶Run ssh ec2-user@*** '
14 Use --update-env to update en
15 [PM2] Applying action reload
16 [PM2] [hana4-next](3) ✓

restart spring
1 ▶Run ssh ec2-user@*** '

Post 부분 178
```

## 14. Github Action

▼ ✓ restart spring

```
1 ▼Run ssh ec2-user@*** '
2 ssh ec2-user@*** '
3 cd /home/ec2-user/workspace &&
4 sudo systemctl restart
5 '
6 shell: /usr/bin/bash -e
7 env:
8 JAVA_HOME: /opt/hosted
9 JAVA_HOME_17_X64: /opt
10 Pnpm_HOME: /home/runner
11 GITHUB_PAGES: true
```

▼ ✓ Post Install Node

```
1 Post job cleanup.
```

▼ ✓ Post setup node pnpm

```
1 Post job cleanup.
2 Pruning is unnecessary.
```

▼ ✓ Post Run actions/setup-java@v3

```
1 Post job cleanup.
```

▼ ✓ Post Run actions/checkout@v3

```
1 Post job cleanup.
2 /usr/bin/git version
3 git version 2.47.1
4 Temporarily overriding HOME='/home/runner/work'
5 changes
6 Adding repository directory to the temporary
7 /usr/bin/git config --global --add safe.directory
8 /usr/bin/git submodule foreach --recursive sh
9 config --local --unset-all 'core.sshCommand'
10 /usr/bin/git config --local --name-only --get
11 http.https://github.com/.extraheader
12 /usr/bin/git config --local --unset-all http.
13 /usr/bin/git submodule foreach --recursive sh
14 'http\https\:\\github\\.com\\\\.extraheader'
```

▼ ✓ Complete job

```
1 Cleaning up orphan processes
2 Terminate orphan process: pid (2013) (java)
```



## 14. Github Action

---

nginx & start next server

```
location ~/(api|members|posts|swagger-ui)(/.*)?$ {
 proxy_pass http://localhost:8081;
}

#location ~/(hi|hello|_next|css|todos)(/.*)?$ {
location / {
 index index.html;
 proxy_pass http://ip-172-31-3-82.ap-northeast-2.compute.internal:3000;
 proxy_http_version 1.1;
 proxy_set_header Upgrade $http_upgrade;
 proxy_set_header Connection 'upgrade';
 proxy_set_header Host $host;
 proxy_cache_bypass $http_upgrade;
}
```

```
[ec2-user@ip-172-31-3-82 my-app]$ node .next/standalone/server.js
▲ Next.js 15.3.2
- Local: http://ip-172-31-3-82.ap-northeast-2.compute.internal:3000
- Network: http://ip-172-31-3-82.ap-northeast-2.compute.internal:3000

✓ Starting...
✓ Ready in 72ms
```



## 14. Github Action

```
drwxrwxr-x. 4 ec2-user 33 Sep 7 23:19 my-app
-rw-r--r--. 1 ec2-user 88M Sep 7 23:18 next.zip
[ec2-user@ip-172-31-3-82 workspace]$ date
Sun Sep 7 23:20:56 UTC 2025
```

```
[ec2-user@ip-172-31-3-82 workspace]$ cd my-app/
[ec2-user@ip-172-31-3-82 my-app]$ ll
total 0
drwxrwxr-x. 4 ec2-user ec2-user 33 Sep 7 23:19 .
drwxr-xr-x. 7 ec2-user ec2-user 119 Sep 7 23:19 ..
drwxrwxr-x. 3 ec2-user ec2-user 24 Sep 7 23:19 .next
drwxr-xr-x. 3 ec2-user ec2-user 107 Sep 7 23:17 public
```

```
$ node .next/standalone/server.js
```

```
pm2 start .next/standalone/server.js --name my-next-app --time
```

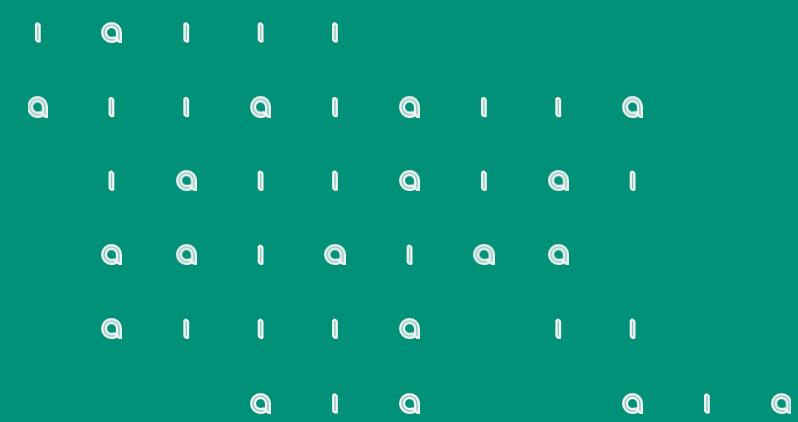
| id | name        | mode | ↺ | status | cpu | memory  |
|----|-------------|------|---|--------|-----|---------|
| 3  | my-next-app | fork | 1 | online | 0%  | 121.3mb |

- name: restart next

run: |

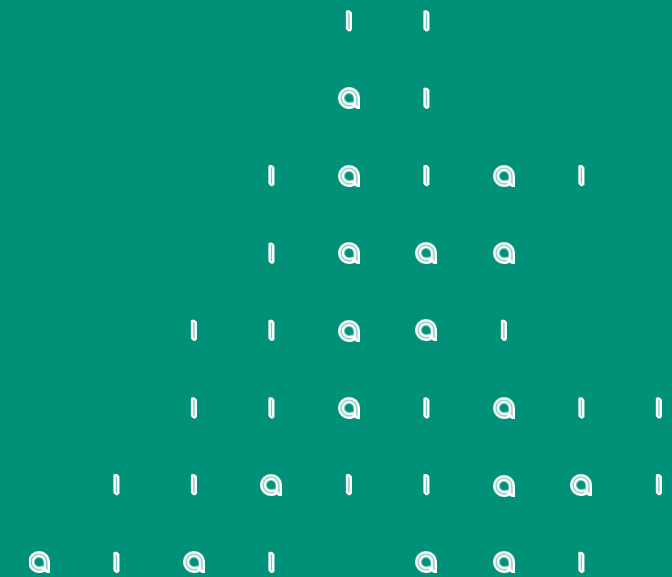
```
ssh ec2-user@${{ secrets.WAS }} '
 cd /home/ec2-user/workspace
 rm -rf my-app/.next
 unzip -q next.zip -d my-app
 pm2 reload my-next-app
'
```

```
middleware - pathname: /hi/night
middleware - pathname: /hi
middleware - pathname: /intercept
middleware - pathname: /photos
middleware - pathname: /todos
middleware - pathname: /board
middleware - pathname: /todos
middleware - pathname: /todos/100
middleware - pathname: /todos/1
```



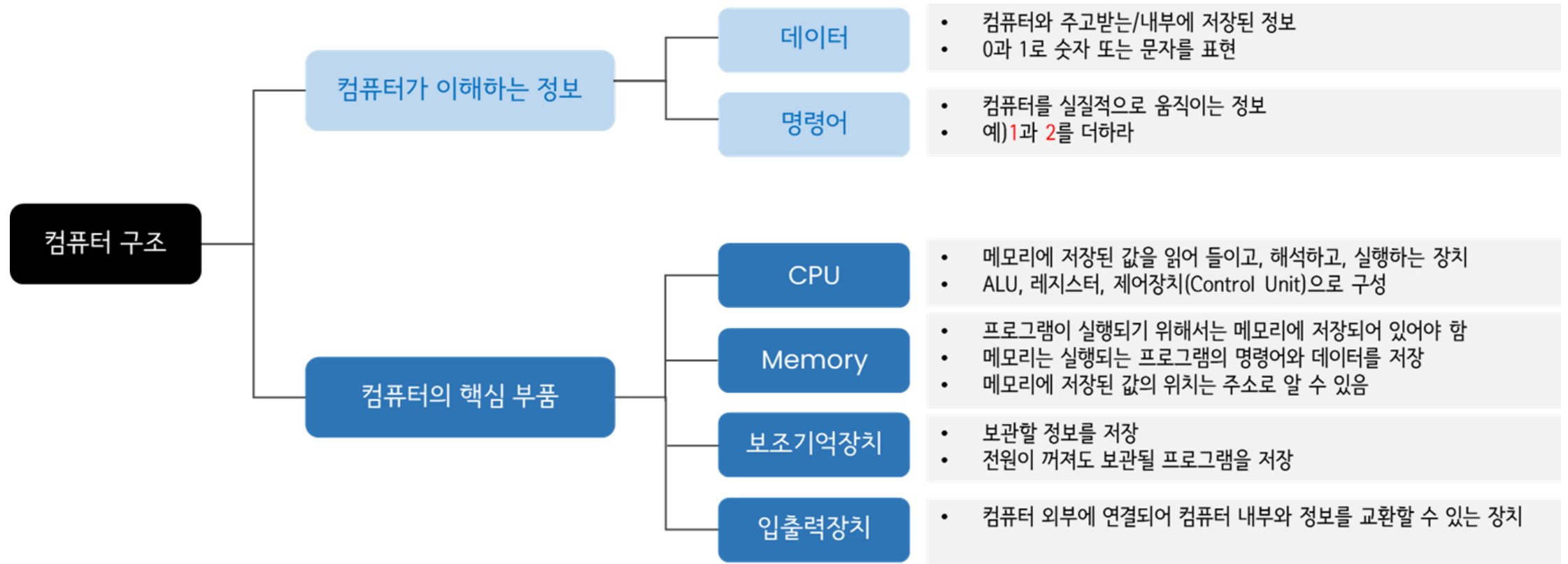
## 15. ICT Architecture의 이해

---



## 15. ICT Architecture의 이해

### 컴퓨터 구조



## 15. ICT Architecture의 이해

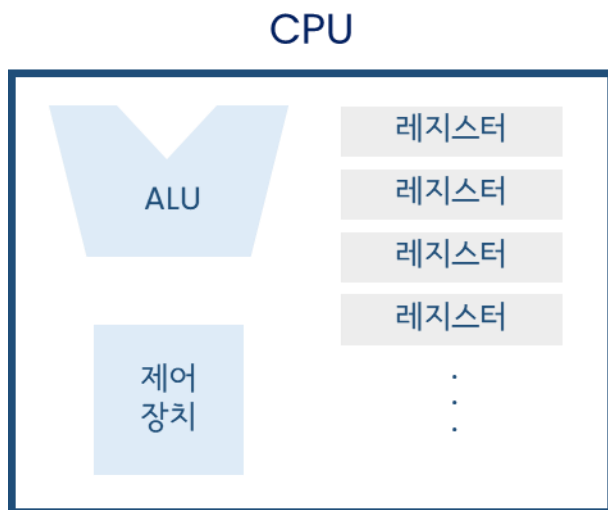
### CPU

#### ❖ ALU(Arithmetic Logic Unit)

- 산술 논리 연산 장치로, 컴퓨터의 중앙 처리 장치(CPU) 내에서 산술 연산(덧셈, 뺄셈 등)과 논리 연산(AND, OR, NOT 등)을 수행

#### ❖ 제어장치(Control Unit, CU)

- 컴퓨터의 전체 작업을 제어하고 조정하는 역할
- 명령어의 해석, 명령어 실행 흐름의 제어, 다른 하드웨어 구성 요소들 간의 데이터 흐름 조정 등을 담당



#### ❖ 레지스터 (Register)

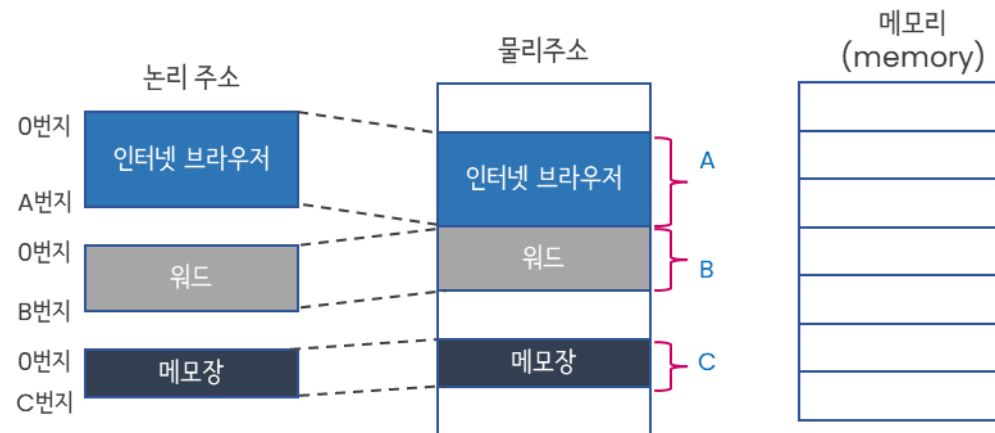
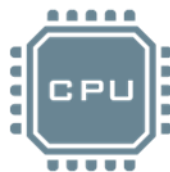
- 중앙 처리 장치(CPU) 내에 존재하는 고속 메모리 장치로, 데이터나 명령어를 일시적으로 저장하고 처리하는 데 사용
- 종류
  - ✓ 프로그램 카운터(PC): 메모리에서 가져올 명령어의 주소를 저장
  - ✓ 명령어 레지스터(IR): 현재 실행중인 명령어를 저장
  - ✓ 메모리 주소 레지스터: 메모리의 주소를 저장
  - ✓ 메모리 버퍼 레지스터: 메모리와 주고받을 값 (데이터와 명령어)
  - ✓ 스택 포인터(SP): 함수 호출 시 호출 스택의 위치를 관리
  - ✓ 플래그 레지스터(Flag Register): 연산의 결과에 따라 설정되는 여러 플래그를 저장
  - ✓ 범용 레지스터: 연산을 수행하는데 필요한 데이터나 중간 결과를 저장
  - ✓ 베이스 레지스터: 기준 주소 저장

## 15. ICT Architecture의 이해

### 메모리(Memory)

#### ❖ 물리 주소

- 메모리 입장에서 바라본 주소
- 말 그대로 정보가 실제로 저장된 하드웨어상의 주소

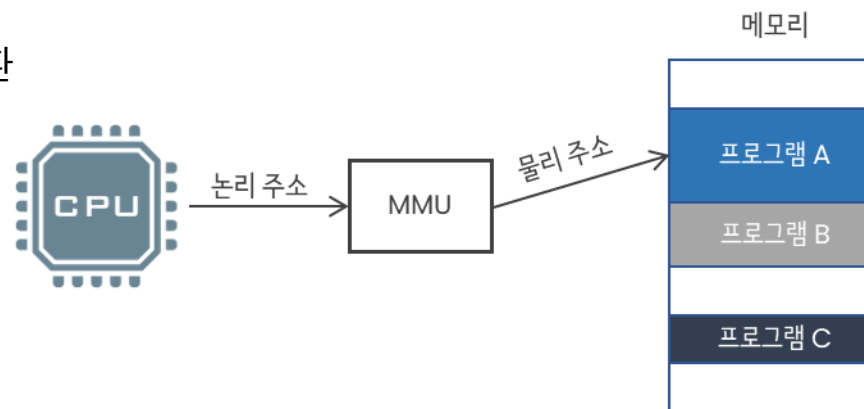


#### ❖ 논리 주소

- CPU와 실행 중인 프로그램 입장에서 바라본 주소
- 실행 중인 프로그램 각각에게 부여된 0번지부터 시작하는 주소

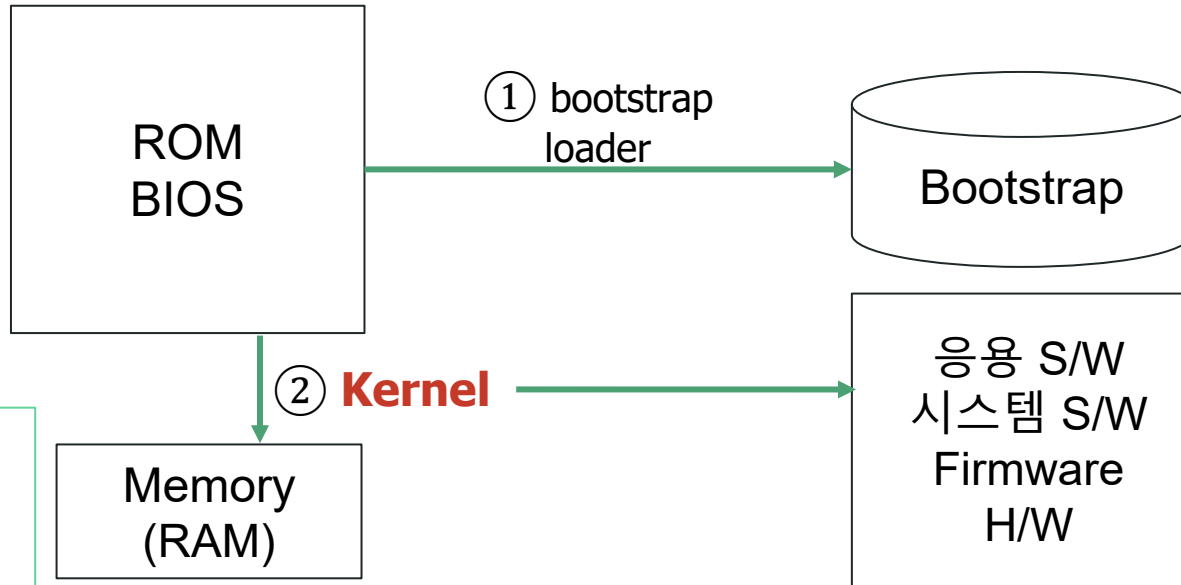
#### ❖ 메모리 관리 장치(MMU: Memory Management Unit)

- 논리 주소와 베이스 레지스터 값을 더하여 논리 주소를 물리 주소로 변환



## 15. ICT Architecture의 이해

### OS (Operating System)



Virtual Memory/Swap  
Fragmentation/Paging  
cf. GarbageCollector

#### <OS>

**Unix** (1970)

- Bell KennethThompson/DennisRitch

**Linux** (1991) LinusTorvalds, [배포판\(계열\)](#)

**Windows** (1985) MS PaulAllen/BillGates

Partitioning (논리)  
Formating(FS결정)  
Index Block  
(FAT:Cluster, Unix:i-node)  
Mirroring(Raid1/Raid5)  
NFS, HDD/SSD  
SATA/SCSI/SAS

<Memory> 1 Byte, 1 Word (cf. nibble, quarter)

**DRAM**(Dynamic RAM): **RAM** 고집적/저속/단위시간-휘발

**SRAM**(Static RAM): **Cache**저집적/고속

**SDRAM**(SyncDynamic RAM) clock신호와 동기

**DDR SDRAM** 최근 대중적은 **DDR4** SDRAM(16배속)

(Double Data Rate SDRAM) 2배 대역폭/속도

### Process vs Thread

**H/W** Multi-Thread & **S/W** Multi-Thread cf. Hyper-Thread

코드/데이터/힙은 공유, 레지스터/스택/프로그램카운터는 각 스레드별

cf. **IPC**(Inter-Process Communication) File | SharedMem.

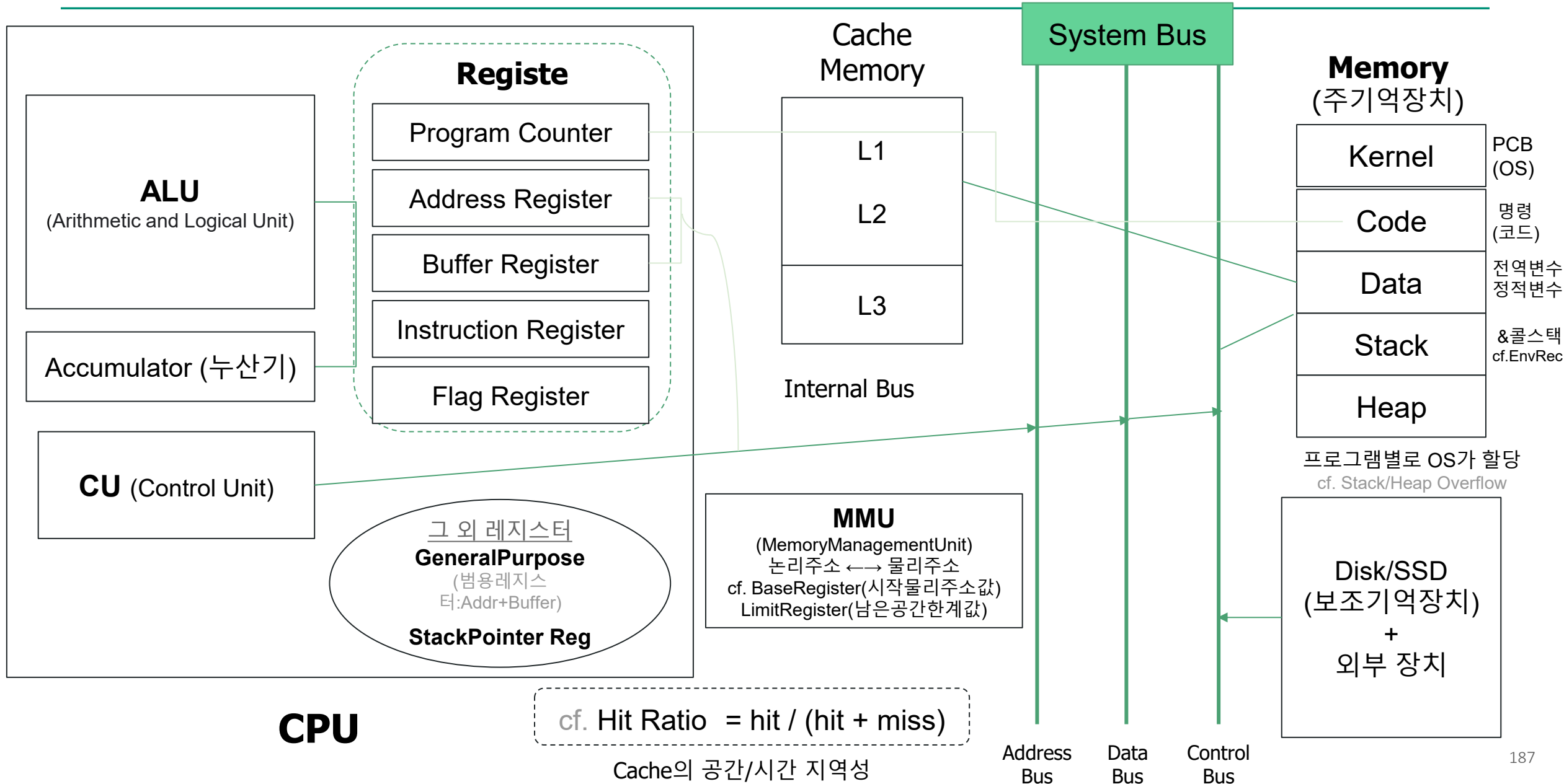
⇒ MutexLock, Semaphore(영속속 각 자원별), Monitor(큐) 방식

⇒ DeadLock발생조건: 상호배제, 점유와대기, 비선점, 원형대기

**PCB**(Process Control Block) for **Context Switching**

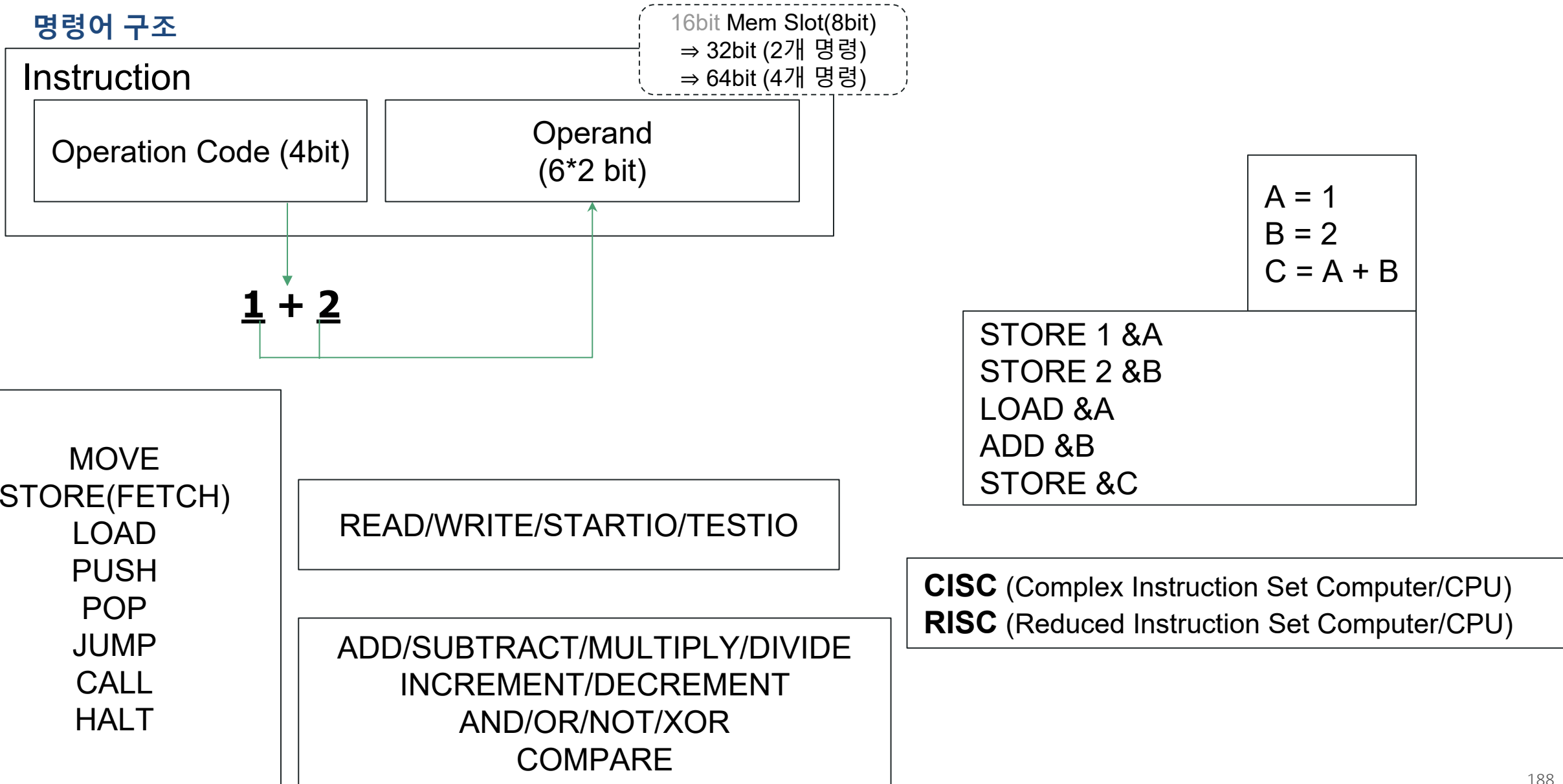
# 15. ICT Architecture의 이해

## 컴퓨터 구조



## 15. ICT Architecture의 이해

### 명령어 구조



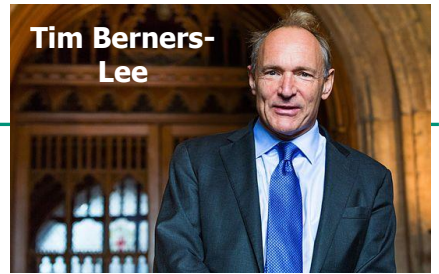


## 15. ICT Architecture의 이해 연대별 언어 chronological programming language

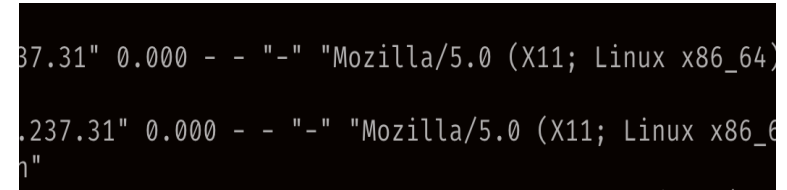
- 1943: Plankalkül (MachineLang)  
ENIAC CS(10진수) → UNIVAC (1949)
- 1951: **Assembly** (2GL)
- 1954: **Fortran**
- 1958: Algol(A) 58
- 1958: Lisp
- 1959: **COBOL** ← FACT (1959)
- 1962: APL, Simular, SNOBOL
- 1964: **BASIC**, PL/1(IBM)
- 1969: **Pascal** (교육용), **B** KennethThompson
- 1970: Forth
- 1970~1984: CommonLisp
- 1972: **C** ← **B** ← CPL(1963,Cambridge)
- 1975: Scheme ← Lisp
- 1978: **SQL**
- 1978~1983: Ada (미국방부, F22)
- 1980: Self ← CommonLisp
- 1983: **C++** ← C with Classes(1980)
- 1984: MATLAB, Eiffel(1985)
- 1986: **Object-C**, ErLang → Elixir
- 1987: **Perl**, Mathematica(1988)
- 1989~1998: **XML** ⇒ + HTML = **XHTML** → JSX
- 1989~1991: **HTML** ← XML ← Enquire(1980)
- 1990: Haskell
- 1991: **Python** ← Perl, VisualBASIC
- 1993: **Ruby**, Lua, **R** → Factor(2003)
- 1991~1995: **Java**(SUN) ← Oak ← C++
- 1994: CLOS ← Part of CommonLisp(1970)
- 1995: JS, **PHP**, **Delphi** ← Pascal IDE/Lang
- 1996: OCaml → **Rust**(2015), **CSS1**
- 1999: D ← C++, XSLT → **CSS3**(2005)
- 2000: **ActionScript** ← ECMAScript
- 2001: **C#**(Delphi+Java), VS.Net → F# (2002)
- 2009: Clojure ← Lisp
- 2003: **Groovy**, **Go**lang, Scala, Factor
- 2011: **Xamarin**, **Dart**(Google), Elixir
- 2011~2016: Kotlin(JetBrains), TS(2012)
- 2014: **Swift**, Rust(2015)

## 15. ICT Architecture의 이해

### JS의 탄생 배경 (feat. Browser)



- 1989, **URL, HTTP, HTML** → **HyperText Browser** (by Sir Tim Berners-Lee)
- 1993, **NCSA *Mosaic* Graphic Browser** (모자이크)
  - 일리노이 대학교 ([UIUC](#)), NCSA(National Center for Supercomputing Applications)
- 1994, Mosaic Communications's ***Mosaic* Netscape 0.9**
  - Jim Clark이 NCSA의 Marc Andreessen 영입하여 전면 재개발. (**Mozilla** = Mosaic + Godzilla)
- 1995, **Mosaic Netscape** → **Netscape Navigator**
  - 브라우저 점유율 75%, NCSA와 상표권 문제로 이름 변경.  
(Mosaic Communications → Netscape Communications)
- "브라우저는 준비되었는데.. 사이트가 별로 없네?"  
"Java Applet은 너무 무겁고 Sandbox 문제도 있고..."



## 15. ICT Architecture의 이해

### JS의 탄생과 발전

- '모든 사용자가 편하게 웹 사이트를 만들게 하자!'
  - ⇒ Scheme Runtime을 **Netscape**에 Embed하자!
  - ⇒ Self, Scheme의 전문가 **Brendan Eich**을 영입!
  - ⇒ LiveScript (코드명 **Mocha**) 완성! (1995년 9월)
- (Applet이 표준, 2011년까지 Java가 JS 보다 빨랐다!)
  - ⇒ 'Live를 Java로 바꿔야 사람들이 인정하고 사용하겠군'
  - ⇒ **LiveScript** ⇒ **JavaScript**
- 이름을 바꿔도 사람들은 인정하지 않아! (cf. MS's JScript, 1996 IE3)
  - ⇒ **ECMA에 찾아가 보자!**



cf. jQuery(2006), CSS3(2005), JSON(2001)

TypeScript(2012), React(2011), ReactTS(2020), Dart/Kotlin/...

## 15. ICT Architecture의 이해

### Compiler

cf. AST

#### 1. Lexical Analyzer

Tokenizer(Scanner) & Lexer  $\Rightarrow$  Symbol

#### 2. Syntax Analyzer (Parser)

$\Rightarrow$  AST(Abstract Syntax Tree) cf. Annotated AST

$\Leftrightarrow$  Token(Parsing) Tree cf. TS's Symbol Table

#### 3. Intermediate Code Generator (Semantic Analyzer)

Type Checking  $\Rightarrow$  Byte Code (Object file)

#### 4. Code Optimizer (Local & Global Optimizer)

Scheduling & Register(Memory Address) Assignment

#### 5. Target Code Generator (exe file, Machine Code)

**Interpreter** Language

**Compiler** Language

```
var a = 1;
var b = 2;
var c = a + b;
if (c >= 3)
 console.log('Big!');
else
 console.log('Small!')
```

## 15. ICT Architecture의 이해

Transpiler (TypeScript Compiler )

(\$> tsc)

Token ⇒ Tree

```
import * as ts from "typescript";

const program = ts.createProgram(files, opts);
const checker = program.getTypeChecker();
program.emit();
```

- 1) **Read TSConfig (tsconfig.json)** setup the program & gets starting files
- 2) **Pre-process Files** follow imports (all possible files)
- 3) **Tokenize(Scan) and Parse** convert text to syntax-tree
- 4) **Binder** convert identifiers in syntax-tree to symbols
- 5) **Type Check** checks the types use Binder and syntax-tree
- 6) **Transform** changes syntax-tree to match tsconfig target options
- 7) **Emit** prints syntax-tree to \*.js, \*.d.ts and other files

code to data  
Annotated AST

type  
checking

creating  
files

## 15. ICT Architecture의 이해

### And Other Things

#### 1. DBMS(DataBase Management System)

RDBMS(MySQL, Postgresql, Oracle, MS-SQL, etc)

DocumentDB(MongoDB, DynamoDB, MySQL XDev/API, etc)

Session, Index, Partitioning, Clustering vs Replication

#### 2. Network

Router(Routing Table), DNS(Domain Name System), Switch, Hub

TCP(Transmission Control Protocol), UDP(User Datagram Protocol)

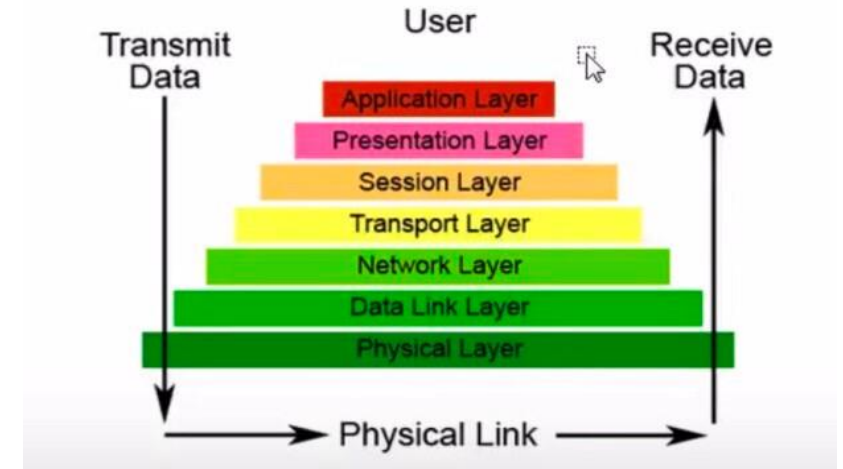
Firewall, Load-Balancer, WASWebApplicationServer vs WSWebServer

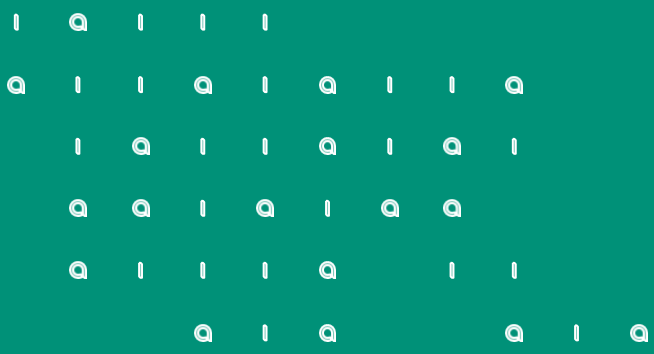
#### 3. Cloud (OnDemand/OnPremise)

InfraaaS, PlatformaaSFirebase/DevOps/Kubernetes/S3/BigQuery, SoftwareaaSGoogleApps/SalesForce

#### 4. Library/Framework, Clean Architecture, Messaging Queue, etc

### The Seven Layers of OSI





THANK YOU

